



天数智芯
Iluvatar CoreX

天数智芯

IGIE 推理框架使用指南

版本：V4.3.0 (x86)|V4.3.6 (ARM)

日期：2025.6.30

适用产品：天垓 150 | 智铠系列

声明

版权与保密声明

本文档（属未发表作品）版权归属于天数智芯全权所有，且本声明不构成对本文档已发表的承认。本文档所载全部信息均为天数智芯保密信息（包括但不限于天数智芯技术信息、经营信息、专属于接收方的未公开资料等）。未经天数智芯事先书面许可，严禁任何形式的使用、复制、传播、转录或翻译。

免责声明

天数智芯可以随时对本文档或本文档中描述的产品进行改进和/或更改。本文档包括与天数智芯产品有关的信息，作为说明典型应用的一种方式，因此，不一定提供足以进行生产设计的完整信息。对于本文档中内容的准确性或完整性，天数智芯不做任何陈述或保证。

联系方式

地址：上海闵行区陈行公路 2168 号 3 幢

电话：021-68886607

网址：www.iluvatar.com

目录

1 修订记录	7
2 IGIE 推理框架功能说明	8
2.1 版本升级说明	8
2.2 IGIE 推理框架模块	8
2.3 使用 IGIE 部署模型步骤	9
2.4 IGIE 功能特性	9
2.4.1 框架模型的支持	9
2.4.2 精度支持	9
2.4.3 自动图优化支持	10
2.4.4 INT8 和 FP16 量化支持	10
2.4.5 Runtime 支持	10
2.4.6 模型多线程多 stream 并发推理	10
2.4.7 Pipeline 支持	10
2.4.8 IGIE Framework Integrations	11
2.4.9 IxRT 后端和 Fallback 机制支持	11
2.4.10 igie-exec 命令行工具	11
2.4.11 IGIE ModelZoo 支持的模型列表	11
3 IGIE 快速上手	12
3.1 使用 igie-exec 命令行工具快速验证性能	12
3.2 使用 Python 脚本端到端推理	12
3.2.1 1. 准备测试模型和测试图片	12
3.2.2 2. FP32/FP16 推理	14
3.2.3 3. INT8 推理	15
4 快速上手：IGIE 推理框架使用教程	17
4.1 教程 0：如何开始深度学习模型推理部署	17
4.1.1 接口支持情况	17
4.2 教程 1：如何导入一个模型到 IGIE	18
4.2.1 前端框架模型导入 IGIE 的支持情况	18
4.2.2 不同框架的模型导入 IGIE 接口使用介绍	19
4.2.2.1 导入 ONNX 模型到 IGIE	19
4.2.2.2 导入 PyTorch 模型到 IGIE	19
4.2.2.3 导入 Tensorflow 模型到 IGIE	20
4.2.3 统一导入接口	20
4.3 教程 2：如何对模型进行量化	22
4.3.1 1. 为什么要做量化？	22
4.3.2 2. 使用 IGIE 对模型进行 FP16 量化	22
4.3.3 3. 对模型进行 INT8 量化	23
4.3.4 4. 量化总结	25
4.3.5 附录：量化术语介绍	25
4.4 教程 3：如何自动优化模型并导出引擎文件	26
4.4.1 1. 自动优化模型并编译成引擎文件	27

4.4.2	2. 导出或导入引擎文件	28
4.5	教程 4: 如何加载引擎文件进行推理 (Python)	28
4.6	教程 5: 如何加载引擎文件进行推理 (C++)	29
4.6.1	步骤 1: 引入必要的头文件	29
4.6.2	步骤 2: 创建 Target 和 Device	29
4.6.3	步骤 3: 创建 Tensor	30
4.6.4	步骤 4: 加载引擎文件	31
4.6.5	步骤 5: 获取模型推理函数	31
4.6.6	步骤 6: 设置模型输入	32
4.6.7	步骤 7: 创建计时器	32
4.6.8	步骤 8: 运行模型	32
4.6.9	步骤 9: 获取模型输出并统计耗时	32
4.6.10	IGIE C++ Inference 完整代码示例	32
4.7	教程 6: 如何使用 igie-exec 生成引擎并推理	36
4.7.1	示例 1: 运行 PyTorch ResNet50 模型	36
4.7.2	示例 2: 运行 YOLOv5 ONNX 模型	36
4.7.3	示例 3: 运行 ONNX BERT-Base Squad 模型	37
4.8	教程 7: IGIE 完整推理说明及示例	37
4.8.1	1. IGIE Fallback 机制说明	37
4.8.2	2. 如何进行推理	38
4.8.2.1	2.1 导入模型到 IGIE	38
4.8.2.2	2.2 优化和编译模型引擎文件	40
4.8.2.3	2.3 推理模型	41
4.8.2.4	2.4 完整推理代码示例	43
4.9	教程 8: 快速上手 Relax 进行模型推理	45
4.10	教程 9: 如何使用 IGIE 多进程推理	47
4.10.0.1	实现推理进程	47
4.10.0.2	开启多进程推理	47
5	igie-exec 模型快速部署和验证工具使用指南	51
5.1	igie-exec 功能说明	51
5.2	igie-exec 的使用指南	51
5.2.1	开始之前: 准备环境	51
5.2.2	测试模型推理性能	52
5.2.2.1	示例 1: 运行 PyTorch ResNet50	52
5.2.2.2	示例 2: 运行 PyTorch TorchVision	52
5.2.2.3	示例 3: 运行 TensorFlow VGG16	53
5.2.2.4	示例 4: 运行 YOLOv3 ONNX	53
5.2.2.5	示例 5: 运行 YOLOv5 ONNX	54
5.2.2.6	示例 6: 运行 YOLOv7 ONNX	54
5.2.2.7	示例 7: 运行 ONNX BERT-Base Squad	54
5.2.2.8	示例 8: 运行 ONNX BERT-Large Squad	54
5.2.2.9	示例 9: 运行 ONNX Electra	55
5.2.2.10	示例 10: 运行 ONNX ViT	55
5.2.2.11	示例 11: 运行 ONNX Swin Transformer	56
5.2.2.12	示例 12: 运行 ONNX Conformer	56
5.2.2.13	示例 13: 使用自定义数据集进行 INT8 量化	56

5.3	igie-exec 测试工具的参数定义	57
6	从 TensorRT 迁移到 IGIE	60
6.1	使用 TensorRT 推理 ResNet50 模型	60
6.2	使用 IGIE 推理 ResNet50 模型	61
6.3	总结	62
7	IGIE 自动图优化介绍	64
7.1	算子融合	64
7.2	Layout 转换和自动 padding	67
7.3	内存复用	67
7.4	常量折叠和常量预计算	67
8	高级教程：IGIE 推理框架高级功能使用教程	69
8.1	教程 0：如何使用 IGIE 给模型添加 NMS	69
8.1.1	Vision ixinfer_nms OP 定义	74
8.1.2	为 RetinaFace 添加 NMS	75
8.1.3	为 YOLOv5 添加 Batched_NMS	77
8.2	教程 1：如何使用 Relax	78
8.2.1	如何使用 TVMScript	78
8.2.2	如何构建 Relax 模型	82
8.2.2.1	Relax.BlockBuilder	82
8.2.2.2	TVMScript	86
8.2.2.3	Relax.Function	87
8.2.2.4	Relax.nn	88
8.2.3	如何优化 Relax 模型	89
8.2.4	如何使用 Relax 前端	90
8.2.5	关于动态 Shape	93
9	IGIE C++ API 说明	95
9.1	引入必要的头文件	95
9.2	Target 相关类	95
9.2.1	Target()	95
9.2.2	Target::Current()	96
9.3	Device 相关类	96
9.3.1	DLDevice 结构体	96
9.3.2	DeviceAPI 类	97
9.3.3	DeviceAPI::GetAttr()	97
9.4	Device 内存、流相关操作和设置	99
9.4.1	SetDevice()	99
9.4.2	CreateStream()	99
9.4.3	SetStream()	99
9.4.4	StreamSync()	100
9.4.5	DeviceSync()	100
9.4.6	FreeStream()	101
9.4.7	AllocDataSpace()	102
9.4.8	CopyDataFromTo()	103
9.4.9	FreeDataSpace()	104

9.5	Tensor 相关类和函数	104
9.5.1	DLTensor 结构体	105
9.5.2	NDArray 类	106
9.5.3	NDArray::Empty()	106
9.5.4	NDArray::Shape()	106
9.5.5	NDArray::DataType()	107
9.5.6	NDArray::CopyFromTo()	108
9.5.7	NDArray::CopyToBytes()	108
9.5.8	NDArray::FromDLPack()	109
9.6	加载模型引擎相关类和函数	110
9.6.1	Module 类	110
9.6.1.1	Module::LoadFromFile()	110
9.6.1.2	Module::GetFunction("default")()	111
9.7	模型输入输出信息相关函数	111
9.7.1	GraphExecutor::GetFunction()	111
9.7.2	get_num_inputs()	113
9.7.3	get_num_outputs()	113
9.7.4	get_input_names()	114
9.7.5	get_output_names()	115
9.7.6	get_input_index()	115
9.7.7	get_output_index()	116
9.7.8	get_input()	116
9.7.9	set_input()	117
9.7.10	set_input_zero_copy()	118
9.7.11	get_output()	119
9.8	模型运行相关函数	120
9.8.1	run()	120
9.9	性能统计相关函数	120
9.9.1	GetTimer()	120
9.9.2	Start()	121
9.9.3	Stop()	121
9.9.4	SyncAndGetElapsedNanos()	122
9.10	推理完整参考示例	122
10	常见问题	128
11	附录: IGIE 框架算子支持列表	135
11.1	IGIE 算子支持 - Relay 算子	135
11.2	IGIE 算子支持 - Relax 算子	146
11.3	框架模型算子支持	146
11.3.1	ONNX 模型算子的支持	147
11.3.2	PyTorch 模型算子支持	151
11.3.3	TensorFlow 模型算子支持	156
11.3.4	Paddle 模型算子支持	160
11.3.5	MXNet 模型算子支持	163

1 修订记录

- COREX01-BI430-UG10-00: 2025/6/30
 - 更新本次发布的 IGIE 版本为 v0.18.1
 - 新增“教程 9：如何使用 IGIE 多进程推理”
- COREX01-BI420-UG10-00: 2025/1/24
 - 更新本次发布的 IGIE 版本为 v0.18.0
 - 新增“教程 10：快速上手 Relax 进行模型推理”
 - 新增“高级教程：IGIE 推理框架高级功能使用教程”
 - 更新“igie-exec 模型快速部署和验证工具使用指南”，补充安装 igie-exec 需要的 Python 依赖包
 - 更新“附录：IGIE 框架算子支持列表”：
 - * 更新“IGIE 算子支持 - Relay 算子”
 - * 新增“IGIE 算子支持 - Relax 算子”
 - * 更新各“框架模型算子支持”
- COREX01-BI411-UG10-00: 2024/8/26

无内容更新
- COREX01-BI410-UG10-00: 2024/6/30

无内容更新
- COREX01-BI340-UG10-00: 2024/3/25

文档本次发布内容与 V3.3.0-BI150 文档相比 (COREX01-BI330-UG10-01)，有以下更新：

 - 更新本次发布的 IGIE 版本为 v0.9.1
 - 更新 IGIE 多后端支持，新增 IxRT 兼容 TensorRT API
 - 更新“IGIE ModelZoo 支持的模型列表”
 - “常见问题”章节新增“问题 14：在 ARM 设备上编译模型出现 Compilation Error，如何解决？”
 - 更新“附录：IGIE 框架算子支持列表”章节中“框架模型算子支持”列表

2 IGIE 推理框架功能说明

IGIE (Iluvatar GPU Inference Engine) 是一个针对天数智芯加速卡研发的高性能、高通用、全流程的神经网络推理框架。可为推理场景提供易部署、高吞吐量、低延迟的完整方案。当前提供的 IGIE 版本是 v0.18.1。

2.1 版本升级说明

- 本次发布 IGIE 版本升级为 v0.18.1。
 - 相比 IGIE v0.9.1, v0.18.0 版本升级涉及如下功能：
 - 升级适配 TVM v0.18.0
 - 新增更灵活的 Relax IR 及配套机制
 - 兼容升级前基于 Relay 的功能
 - 新增对 CV/NLP 场景下的 Dynamic Shape 模型推理支持
 - 提升 tvmscript 易用性, 提供 Python 语法形式的 round-trippable TVM IR
 - 扩展 MetaSchedule 模块, 提供 Relay/Relax/tir 层调优方法
 - 重构 tir buffer 实现, 支持 N-d buffer 形式
 - 更新 Relay 算子, 增加 Relax 算子介绍, 以及各前端 OpConverter 类型, 详情请参考“附录: IGIE 框架算子支持列表”
- Relax 算子在命令以及分组上更倾向于向 Python array API 看齐, 因此您可以通过访问 [API specification -Python array API standard v2023.12](#) 进一步了解 Relax 算子。

2.2 IGIE 推理框架模块

IGIE 主要提供以下功能模块:

- 模型导入: 支持将 PyTorch、Tensorflow、ONNX、PaddlePaddle、MXNet 等框架模型导入到 IGIE 中。
- 模型量化: 支持 FP16 和 INT8 量化, 支持 KL 和 Percentile 量化校准算法, 也支持客户使用任何开源量化框架 (例如 PyTorch、ONNXRuntime 量化), 只要能将量化后模型导出为 ONNX 模型文件即可。
- 自动图优化: 对模型进行自动优化, 包括自动算子融合, 冗余算子消除, 常量折叠, 内存重用, layout 转换, 自动 padding 等, 最终转换为性能更优的 IR Module。
- 编译: 将优化好的模型编译成可执行的引擎文件 (engine file), 供模型部署使用。引擎文件 (engine file) 是指通过模型构建的, 经过 IGIE 优化的可部署的二进制文件。
- 运行时: 用户使用 IGIE 轻量 Runtime, 加载并运行引擎文件。
- AutoTune: IGIE 通过 AutoTune 可以自动寻找 ixInfer、ixDNN、ixBLAS、IGIE 算子等生成各种融合算子的最优实现, 并应用于模型。
- 多后端支持: 支持 CPU、天数智芯加速卡 (Native)、IxRT 等后端的支持。当前 IxRT 兼容 TensorRT API, IGIE 增加了 IxRT 后端的支持, 导入模型时可以指定 IxRT 后端, 来使用 IxRT 推理引擎编译和运行模型。
- Fallback 机制: 支持当 IxRT 后端模型编译失败后自动 Fallback 到原有的 IGIE Backend 执行。

2.3 使用 IGIE 部署模型步骤

当您得到训练好的模型后，使用 IGIE 推理框架的步骤如下：

1. 安装 IGIE 环境
2. 将模型导入 IGIE
3. 对导入模型进行量化，IGIE 还会对量化后的模型自动进行转换和优化
4. 将优化后的模型序列化为引擎文件 (engine file)，用于部署
5. 加载引擎文件，执行推理

2.4 IGIE 功能特性

2.4.1 框架模型的支持

您可直接把主流框架模型直接导入 IGIE。前端框架及算子支持情况如下：

框架	支持算子数量
ONNX	191
PyTorch	230
Tensorflow	174
Caffe 和 Caffe2	53
Paddle	163
TFLite	102
MXNet	255
Keras	70
DarkNet	28

详细算子支持信息请查看“附录：IGIE 框架算子支持列表”。

2.4.2 精度支持

IGIE 支持以下数据类型：

- FP32
- FP16
- INT8

2.4.3 自动图优化支持

IGIE 图优化模块能够自动地对导入模型进行优化，优化手段主要包括自动算子融合、冗余算子消除、常量折叠、内存重用、layout 转换、自动 padding 等，最终转换为符合天数智芯硬件特性且性能更优的 IR Module，用于最终推理。

具体内容请参考“IGIE 自动图优化介绍”。

2.4.4 INT8 和 FP16 量化支持

目前 IGIE 支持 FP16 量化和 INT8 量化：

- FP16 量化是在 FP32 的基础上对 FP32 数据直接进行截断，无需提供校准数据，因此量化较为简单。
- INT8 量化主要分为两类：训练后量化（Post-Training Quantization, 简称 PTQ）和量化感知训练（Quantization-Aware Training, 简称 QAT）。
 - 训练后量化 PTQ，是指将模型中的权重由浮点数量化到低比特整数（当前支持 INT8/INT4），并通过少量校准数据对数据（activation）进行校准量化。
 - 量化感知训练 QAT，是指借助用户完整训练数据集，在训练过程中引入量化和反量化算子，通过在训练前向计算中对数据和权重进行伪量化（Q/DeQ），引入量化误差损失，从而在训练过程中提高模型对量化效应的适应能力，提高最终的量化模型精度。

IGIE 内置了训练后量化 PTQ，提供了 FP16 和 INT8 类型的量化，而对于量化感知训练 QAT 可以通过任何其他量化框架（PyTorch、ONNXRuntime quantization）进行 QAT 训练后，再导入到 IGIE。

2.4.5 Runtime 支持

IGIE 提供了一个非常轻量的 runtime 用于模型部署。目前支持 C++ 和 Python 两种 API 接口用于模型部署。

2.4.6 模型多线程多 stream 并发推理

得益于天数智芯加速卡的 Stream 特性，IGIE 可以为每个模型设置不同的 stream，以支持模型在单卡上并发推理。

2.4.7 Pipeline 支持

IGIE 可以作为 pipeline 中的推理组件，嵌入到 pipeline 中运行。

Pipeline 中 IGIE GPU 内存持久化的支持：

- 在视频分析 pipeline 应用中，一般会希望数据尽可能放在 GPU 内存中，即解码后的图像数据一直保存在 GPU 内存中，持续到该帧在 pipeline 的整个生命周期，解码后的数据直接用于推理，以避免内存拷贝带来的开销。
- IGIE 支持将解码后的视频帧通过零拷贝的方式，直接构造 IGIE Tensor 进行模型推理，无需数据搬运，该特性也使得 IGIE 可以作为框架后端进行集成。

2.4.8 IGIE Framework Integrations

将 IGIE 作为框架的后端，部分算子可以使用 IGIE 进行加速。例如：

- Tensorflow-igie
- Torch-igie
- onnxruntime-igie
- paddle-igie

2.4.9 IxRT 后端和 Fallback 机制支持

- 当前 IxRT 兼容 TensorRT API，IGIE 增加了 IxRT 后端的支持，可以指定 IxRT 后端编译和运行模型。
- IGIE Fallback 机制：当使用 ONNX 模型进行推理时，默认使用 IxRT Backend 进行推理。如果 IxRT 不支持该模型，则自动 Fallback 到原有的 IGIE Backend 执行，该过程将自动化完成。

2.4.10 igie-exec 命令行工具

igie-exec 是 IGIE 提供的一套基于命令行调用的模型快速部署和验证工具，方便用户一键运行 FP32/FP16/INT8 模型推理和生成相应的引擎文件。

igie-exec 的具体使用方法请参考“igie-exec 模型快速部署和验证工具使用指南”。

2.4.11 IGIE ModelZoo 支持的模型列表

IGIE ModelZoo 目前支持的模型列表分类如下：

Task	ModelZoo	Nums
Classification	Classification-ModelZoo	89
ObjectDetection	ObjectDetection-ModelZoo	20
SemanticSegmentation	SemanticSegmentation-ModelZoo	3
Trace	Trace-ModelZoo	3
NLP	NLP-ModelZoo	6
Audio	Audio-ModelZoo	2
TotalNum	/	123

Note

IGIE ModelZoo 支持的模型列表将持续更新。

3 IGIE 快速上手

本章节将从使用 **igie-exec** 命令行工具快速验证性能 和使用 **Python** 脚本端到端推理 两方面的操作说明指导用户快速上手 IGIE。

3.1 使用 igie-exec 命令行工具快速验证性能

igie-exec 工具提供了基于命令行调用的方式，使用一行命令可以直接生成 FP32/FP16/INT8 (使用随机数据集或用户提供的校准数据集进行量化) 的引擎文件，并进行性能测试，可以帮助用户快速验证模型的性能。

Note

引擎文件 (engine file) 是指通过模型构建的、经过 IGIE 优化的可部署的二进制文件。

通过.whl 包安装完 IGIE 后，即可在任意目录下执行 **igie-exec** 命令。

下面是使用 igie-exec 进行 ResNet18 模型的 INT8 推理的示例，模型将从 TorchVision 自动下载，INT8 量化使用随机数据进行校准：

```
$ igie-exec --model_path resnet18 --input input:64,3,224,224 --precision int8

# 推理结果 (BI150 的设备)
[2023-03-22 12:52:39,986 igie-exec line:64] INFO: Evaluate inference igie time cost...
[2023-03-22 12:52:48,832 igie-exec line:67] INFO: Mean inference time (std dev): 2.709 ms
↳ (0.011 ms)
[2023-03-22 12:52:48,833 igie-exec line:69] INFO: Mean fps: 23621.54
```

关于 igie-exec 的参数介绍和更多示例，请参考“igie-exec 模型快速部署和验证工具使用指南”。

3.2 使用 Python 脚本端到端推理

IGIE 提供了非常简单且友好的 Python/C++ 的调用方式，基本上遵循 导入模型 -> INT8 量化 (可选) -> 编译 -> 导出引擎文件 -> 推理模型等推理步骤。

下面以 ResNet50 模型为例，展示在 Python 中使用 IGIE 生成 FP32/FP16/INT8 的引擎文件并进行推理的示例。

3.2.1 1. 准备测试模型和测试图片

IGIE 支持导入 PyTorch/ONNX/TensorFlow/PaddlePaddle 等各种框架的模型。此处以 PyTorch 举例，从 TorchVision 下载预训练的 ResNet50 模型：

```
import numpy as np
from PIL import Image
```

```
import torch
import torchvision
from torchvision import transforms
from tvn.contrib.download import download_testdata

import ssl
ssl._create_default_https_context = ssl._create_unverified_context

model_name = "resnet50"
model = getattr(torchvision.models, model_name)(pretrained=True).eval()
input_data = torch.randn([1, 3, 224, 224])
scripted_model = torch.jit.trace(model, input_data).eval()

img_url = "https://github.com/dmlc/mxnet.js/blob/main/data/cat.png?raw=true"
img_path = download_testdata(img_url, "cat.png", module="data")
img = Image.open(img_path)

img_preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])])
img = img_preprocess(img)
img = np.expand_dims(img, 0)
print("image shape: ", img.shape)
```

测试图片如下：



图 1: 推理测试图片

3.2.2 2. FP32/FP16 推理

IGIE 对 FP32/FP16 的切换非常简单，通过 `precision` 参数就可以自动切换模型的推理精度，并且在编译时选择对应的图优化策略。

对于上述步骤中下载的 ResNet50 模型，IGIE 只需如下步骤就可以完成 FP32/FP16 的引擎文件生成与推理。

```
import tvm
from tvm import relay
from tvm.contrib import graph_executor

# 设置 GPU 设备
target = tvm.target.iluvatar(options="-libs=cudnn,cublas,ixinfer")
dev = tvm.device(target.kind.name, 0)

# 导入模型
input_name = "input0"
shape_list = [(input_name, img.shape)]
mod, params = relay.frontend.from_pytorch(scripted_model, shape_list)

# 编译
precision = "fp16" # 设置 precision = "fp32" 就会使用 FP32 推理
engine = relay.build(mod, target=target, params=params, precision=precision)

# 导出引擎文件
engine.export_library("resnet50.so")
# engine = tvm.runtime.load_module("resnet50.so") # 也可直接导入之前生成的引擎文件

# 推理
m = graph_executor.GraphModule(engine["default"](dev)) # 加载引擎文件到 GPU Device

igie_input = tvm.nd.array(img.astype("float32"))
m.set_input(input_name, igie_input)
m.run()
igie_output = m.get_output(0)
print(igie_output.numpy().shape)
```

验证 IGIE 推理结果，和 PyTorch 推理结果进行比较：

```
# 使用 PyTorch 推理该图片
with torch.no_grad():
    torch_img = torch.from_numpy(img)
    torch_output = model(torch_img)

# Get top-1 result
top1_igie = np.argmax(igie_output.numpy()[0])
top1_torch = np.argmax(torch_output.numpy())

print("IGIE top-1 id: {}".format(top1_igie))
print("Torch top-1 id: {}".format(top1_torch))
```

最终输出，IGIE 和 Torch 输出结果一致 (label: 282='tiger cat'):

IGIE top-1 id: 282.

Torch top-1 id: 282.

3.2.3 3. INT8 推理

INT8 推理区别于 FP32 和 FP16 推理，需要对模型进行 INT8 量化。IGIE 内置了量化接口，同时也支持使用第三方量化框架。关于量化的更多详细信息，请参考“教程 2: 如何对模型进行量化”。

在此示例中，我们使用 IGIE 内置量化接口对导入模型进行 INT8 量化，具体代码如下：

```
import tvm
from tvm import relay
from tvm.contrib import graph_executor

# 设置 GPU 设备
target = tvm.target.iluvatar(options="-libs=cudnn,cublas,ixinfer")
dev = tvm.device(target.kind.name, 0)

# 导入模型
input_name = "input0"
shape_list = [(input_name, img.shape)]
mod, params = relay.frontend.from_pytorch(scripted_model, shape_list)

# 设置校准数据集
def calibrate_dataset(batched_image, input_name):
    calibration_data_list = []
    data = tvm.nd.array(batched_image)
    # 为了方便演示，这里重复使用一张图片组成校准数据集，进行校准
    for i in range(16):
        calibration_data_list.append({input_name: data})
    return calibration_data_list

# 使用 IGIE 内置量化接口量化模型
with relay.quantize.qconfig(calibrate_mode="percentile", weight_scale="max"):
    mod = relay.quantize.quantize(mod, params, dataset=calibrate_dataset(img, input_name))

# 编译
precision = "int8" # 设置 int8，编译时会进行针对 INT8 的专用的图优化
engine = relay.build(mod, target=target, params=params, precision=precision)

# 导出引擎文件
engine.export_library("resnet50.so")
# engine = tvm.runtime.load_module("resnet50.so") # 也可直接导入已生成的引擎文件

# 推理
m = graph_executor.GraphModule(engine["default"](dev)) # 加载引擎文件到 GPU Device
```

```
igie_input = tvn.nd.array(img.astype("float32"))  
m.set_input(input_name, igie_input)  
m.run()  
igie_output = m.get_output(0)  
print(igie_output.numpy().shape)
```

4 快速上手：IGIE 推理框架使用教程

本教程提供了一系列场景式的使用文档来帮助用户快速熟悉 IGIE，加速深度学习模型推理部署。

4.1 教程 0：如何开始深度学习模型推理部署

开始之前，我们来简单了解下 IGIE 推理部署深度学习模型的基本流程：

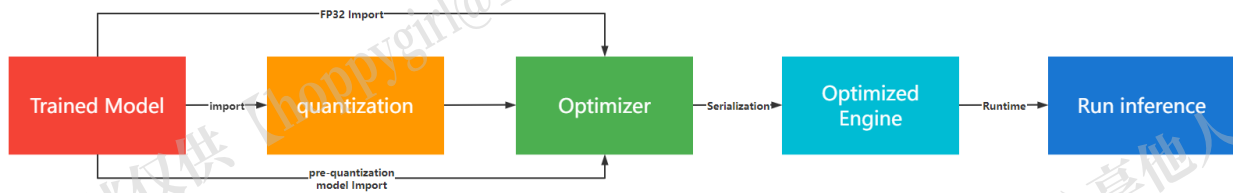


图 2: IGIE 推理模型流程

如上图所示，主要经过以下流程：

阶段	描述	教程
模型导入	将训练好的深度学习模型导入到 IGIE。	教程 1: 如何导入一个模型到 IGIE
模型量化	对模型进行量化（可选，只对 INT8 推理适用）。	教程 2: 如何对模型进行量化
自动图优化	对模型进行自动图优化（IGIE 在编译时会自动完成）。	教程 3: 如何自动优化模型并导出引擎文件
导出引擎文件	将优化后的模型导出为引擎文件用于部署。引擎文件 (engine file) 是指通过模型构建的，经过 IGIE 优化的可部署的二进制文件。	教程 3: 如何自动优化模型并导出引擎文件
推理部署	导入引擎文件进行推理部署。	教程 4: 如何加载引擎文件进行推理 (Python) 教程 5: 如何加载引擎文件进行推理 (C++)

4.1.1 接口支持情况

阶段	Python 接口	C++ 接口
模型导入	支持	开发中
模型量化	支持	开发中
自动图优化	支持	整理中
导出引擎文件	支持	整理中
推理部署	支持	支持

4.2 教程 1: 如何导入一个模型到 IGIE

IGIE (Iluvatar GPU Inference Engine) 是针对天数智芯加速卡研发的高性能深度学习推理引擎，通过对模型进行一系列图优化，算子融合、量化、集成多个加速算子库、AutoTune 等优化手段。您可使用 IGIE 进行深度学习模型推理，在天数智芯加速卡上可以达到低延迟、高吞吐的最佳性能。

首先，我们需要将训练好的深度学习模型导入到 IGIE 中，使用 IGIE 对模型做加速优化。

4.2.1 前端框架模型导入 IGIE 的支持情况

目前主流深度学习框架的模型（TensorFlow、PyTorch、ONNX）都可以直接导入 IGIE，无需额外转换到 ONNX 模型。

各个前端框架使用的导入接口如下：

支持框架	支持算子数量	使用导入 API
ONNX	191	relay.frontend.from_onnx()
PyTorch	230	relay.frontend.from_pytorch()
Tensorflow	174	relay.frontend.from_tensorflow()
Caffe/Caffe2	53	relay.frontend.from_caffe()
Paddle	163	relay.frontend.from_paddle()
TFLite	102	relay.frontend.from_tflite()
MXNet	255	relay.frontend.from_mxnet()
Keras	70	relay.frontend.from_keras()
DarkNet	28	relay.frontend.from_darknet()

IGIE 详细的算子支持情况请查看“附录：IGIE 框架算子支持列表”。

4.2.2 不同框架的模型导入 IGIE 接口使用介绍

下面针对不同框架的模型，介绍如何导入模型到 IGIE。

4.2.2.1 导入 ONNX 模型到 IGIE

导入 ONNX 模型时，需要使用 `onnx.load` 加载 ONNX 模型，然后使用 `relay.frontend.from_onnx` 将模型导入进 IGIE。

```
import tvn
from tvn import relay
import onnx

# 输入模型信息
model_path="./resnet50.onnx"
input_name="input0"
input_shape=[32, 224, 224, 3]

onnx_model = onnx.load(model_path)
shape_dict = {input_name: input_shape}
mod, params = relay.frontend.from_onnx(onnx_model, shape_dict)
print(" 查看模型 graph op: ", mod)
```

参数说明：

- `model_path`: 模型的路径。
- `input_name`: 模型输入节点的名称。
- `input_shape`: 模型输入的 shape。
- `shape_dict`: `input_name` 和 `input_shape` 组成的 list。

4.2.2.2 导入 PyTorch 模型到 IGIE

导入 PyTorch 模型时，需要使用 `torch.jit.load` 加载 pt 模型，然后使用 `relay.frontend.from_pytorch` 将模型导入进 IGIE。

以导入 `resnet50.pt` 模型为例：

```
from tvn import relay
import torch

scripted_model = torch.jit.load("./ResNet50.pt").eval()
shape_list = [("input0", ([32, 224, 224, 3], "float32"))]
mod, params = relay.frontend.from_pytorch(scripted_model, shape_list)
print(mod)
```

参数说明：

- `scripted_model`: 导入的 PT 模型。
- `shape_list`: 模型输入名和模型输入的 shape 组成的 list。

4.2.2.3 导入 Tensorflow 模型到 IGIE

导入 TensorFlow 模型时,需要使用 graph_def 加载 pb 模型,然后使用 relay.frontend.from_tensorflow 将模型导入进 IGIE。

```
# 导入 Tensorflow 包
import tensorflow as tf
import tvn.relay.testing.tf as tf_testing

# 使用 graph_def 加载 pb 模型
with tf.io.gfile.GFile(model_path, "rb") as tf_graph:
    graph_def = tf.compat.v1.GraphDef()
    graph_def.ParseFromString(tf_graph.read())
    graph_def = tf_testing.ProcessGraphDefParam(graph_def)

# 导入 TF graph 到 IGIE
shape_dict = {input_name: input_shape}
mod, params = relay.frontend.from_tensorflow(graph_def, shape_dict)
print(mod)
```

参数说明:

- model_path: 模型的路径。
- input_name: 模型输入节点的名称。
- input_shape: 模型输入的 shape。
- shape_dict: input_name 和 input_shape 组成的 list。

4.2.3 统一导入接口

为了方便地导入模型,我们整合了所有框架地导入接口,您仅需使用一个 import_model_to_igie 函数即可导入 ONNX, PyTorch, TensorFlow, PaddlePaddle 等模型到 IGIE 中。因此,您可以使用不同框架的模型导入 IGIE 接口使用介绍 中介绍的 from_onnx 导入接口指定导入的 ONNX 模型,也可以直接使用 import_model_to_igie 导入任意类型的模型。

import_model_to_igie 使用方法

```
# 首先导入函数
from tvn.relay.import_model import import_model_to_igie

### 设置模型信息
batch_size = 32
model_path_or_name = "./resnet50.onnx"
input_dict = {"input": ([batch_size, 3, 224, 224], "float32")}
output_dict = {"output": "float32"}
precision = "fp16"
mod, params = import_model_to_igie(model_path_or_name, inputs_info=input_dict,
    outputs_info=output_dict, precision=precision, backend="default")
```

import_model_to_igie 函数说明

使用该接口可以直接导入 ONNX, TensorFlow, PyTorch, TorchVision, PaddlePaddle 模型到 IGIE 中。

```
def import_model_to_igie(model_path_or_name, inputs_info, outputs_info=None,
    ↪ precision=None, backend="default", dynamic_info=None)
```

参数说明

- `model_path_or_name`: 根据自己的模型类型设置 模型路径或 模型目录或 模型名称等字符串。详细示例如下:
 - 模型文件路径:
 - * ONNX 模型: 例如 /path/to/resnet50.onnx
 - * PyTorch 模型: 例如 /path/to/resnet50.pt
 - * TensorFlow 模型: 例如 /path/to/resnet50.pb
 - 模型文件目录: 以 目录/模型名称的形式指定 `model_path_or_name` 参数。例如:
 - * PaddlePaddle 模型文件: resnet50/ model。resnet50 目录下需要包含 resnet50/model.pdiparams、model.pdiparams.info 和 model.pdmodel 文件。
 - 模型名称: 如果传入参数为模型名称, 如 resnet50、vgg16, 该接口会自动从 TorchVision 中寻找并加载 torch pre-train 模型。

- `inputs_info`: 模型的输入信息。

期望以 `inputs_info={'name': (shape, dtype)}` 的形式设置 `inputs_info`。例如:

```
inputs_info = {
    'input0': ([32, 3, 224, 224], "float32"),
    'input1': ([32, 256], "int32")
}
```

- `outputs_info`: 仅在使用 IxRT Backend 时需要设置, 设置模型的输出信息, 默认值为 None。

期望以 `outputs_info={'name': dtype}` 的形式设置 `outputs_info`。例如:

```
outputs_info={'output0': "float32", 'output1': "int32"}
```

- `precision`: 仅在使用 IxRT Backend 时需要设置, 设置模型推理精度, 默认值为 None。

例如: `precision="fp16"`。

- `dynamic_info`: 仅在使用 IxRT Backend 时需要设置, 设置模型的输入为动态 shape, 默认值为 None。

例如:

```
dynamic_info={
    'min_shape': {'input0': ([1, 3, 224, 224], "float32"), 'input1': ([1,
    ↪ 256], "int32")},
    'opt_shape': {'input0': ([32, 3, 224, 224], "float32"), 'input1':
    ↪ ([32, 256], "int32")},
    'max_shape': {'input0': ([128, 3, 224, 224], "float32"), 'input1':
    ↪ ([128, 256], "int32")},
}
```

- backend：直接指定模型推理后端，默认值为 default。
 - backend="default"，表示默认使用 IxRT Backend，IxRT 运行失败后，Fallback 到 IGIE Backend 执行。
 - backend="ixrt"，表示使用 IxRT Backend 执行模型。
 - backend="igie"，表示使用 IGIE Backend 执行模型。

返回值

返回 mod 和 params 两个对象，用来下一步进行优化和编译引擎文件。

- mod：模型结构对象。
- params：模型参数对象。

4.3 教程 2: 如何对模型进行量化

之前我们介绍了“教程 1: 如何导入模型到 IGIE”，现在我们介绍如何量化模型。

4.3.1 1. 为什么要做量化？

量化是推理引擎常用的性能加速手段，量化是指对模型的权重（weight）和数据（activation）进行低比特处理，让最终生成的网络模型更加轻量化，从而达到节省网络模型存储空间、降低传输时延、提高计算效率，达到性能提升与优化的目标。

一般训练好的模型输入和权重都是 FP32 类型的，将模型量化到更低比特（INT8）后，能够有以下好处：

- 减少 GPU 内存占用量（FP32 4Byte -> INT8 1Byte，内存占用量减少 3/4）。
- 目前很多模型的算子都是 memory bound，量化到 INT8，能大幅度提升访存带宽。
- 天数智芯加速卡为不同数据类型配置了不同的算力，例如，在智铠加速卡中，INT8 算力是 FP32 算力的 16 倍。使用 INT8 推理能充分利用 GPU 的算力资源。

4.3.2 2. 使用 IGIE 对模型进行 FP16 量化

FP16 量化是直接对 FP32 数据进行截断，在编译构建时，通过指定 precision=fp16 参数可以直接将模型编译为 FP16 的引擎文件。

```
import tvn
from tvn import relay
import onnx

# 输入模型信息
model_path="./resnet50.onnx"
input_name="input0"
input_shape=[32, 224, 224, 3]

# 1) 导入模型到 IGIE
```

```
onnx_model = onnx.load(model_path)
shape_list = {input_name: input_shape}
mod, params = relay.frontend.from_onnx(onnx_model, shape_list)

# 2) 设置模型使用 FP16 推理
precision = "fp16"

# 3) 自动优化模型，编译模型为引擎文件
print("Strat build engine...")
engine = relay.build(mod, target=target, params=params, precision=precision)
```

4.3.3 3. 对模型进行 INT8 量化

INT8 量化分为对权重（weight）量化和数据（activation）量化。

- 因为模型的权重已知，可以直接对权重进行量化，统计量化系数。
- 而模型的输入 (feature map) 是未知的，因此需要使用校准数据集：使用一部分校准数据集来代表模型整个推理过程中数据，统计数据分布情况，计算激活数据的量化系数。

为了更好满足用户需求，IGIE 支持两种量化方式：

- 支持使用 IGIE 自身内置的量化框架，对导入后的模型进行量化，对于 CV 类模型有较好效果，但有一定的局限性。目前内置的量化框架暂不支持 Per Channel 量化。
- 支持直接导入已量化好的 ONNX 模型，即：将模型量化好之后再导入到 IGIE。用户可以选择使用任何量化框架，包括 ONNXRuntime Quantization、PyTorch Quantization、TFLite 等开源量化框架，只要能将量化后模型导出为 ONNX 格式即可。

下图是 IGIE 支持的两种量化方式流程图：

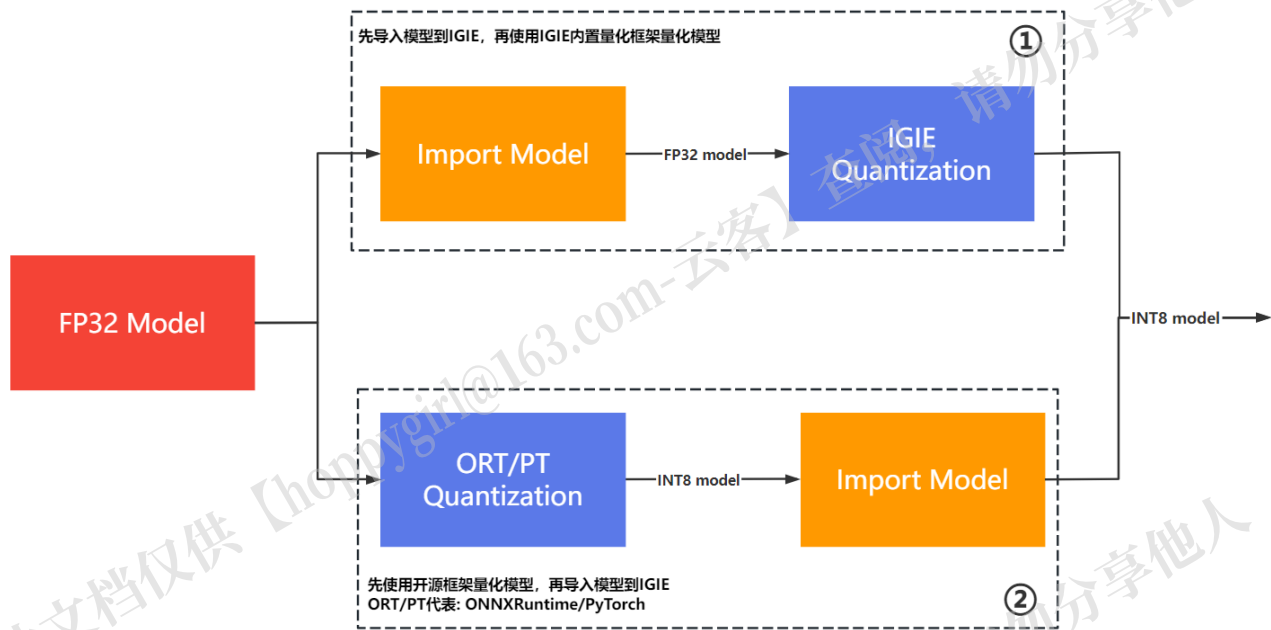


图 3: IGIE 支持的量化方式

1) 使用 IGIE 自身内置的量化框架进行 INT8 量化:

```
import tvn
from tvn import relay
import onnx

# 1) 导入模型到 IGIE
onnx_model = onnx.load(model_path)
shape_dict = {input_name: input_shape}
mod, params = relay.frontend.from_onnx(onnx_model, shape_dict)

# 2) 对模型进行 INT8 量化
# 使用 16 张图片，作为 ResNet50 INT8 量化校准数据集，
def calibrate_dataset(batched_image, input_name):
    calibration_data_list = []
    data = tvn.nd.array(batched_image)
    for i in range(16):
        calibration_data_list.append({input_name: data})
    return calibration_data_list

# 量化模型
with relay.quantize.qconfig(calibrate_mode="percentile", weight_scale="max"):
    mod = relay.quantize.quantize(mod, params, dataset=calibrate_dataset(batched_image,
    ↪ input_name))
```

通过对 relay.quantize.qconfig 进行配置，可以控制 IGIE 的量化行为。比如：

- calibrate_mode: 选择校准算法“percentile”或者“kl_divergence”。

- weight_scale: 选择权重的量化方式 max 或者 power2。
- skip_conv_layers=[]: 选择哪些 conv layer 跳过, 不进行 INT8 量化。
- skip_dense_layer=False: 是否跳过所有的 dense 量化。
- skip_group_conv_layers=False: 是否跳过所有的 group conv2d 量化。
- import_scale_file=scale_file_path: 导出并加载量化系数到指定文件。

2) 使用深度学习框架自带的开源量化框架将模型量化好后, 再导入 IGIE, (框架量化工具例如: ONNXRuntime 的 quantization, PyTorch 的 quantization tools)。

推荐使用 ONNXRuntime 进行量化, IGIE 封装了 ONNXRuntime 的量化实现, 可参考 [ONNXRuntime 量化官方网站](#)。

4.3.4 4. 量化总结

IGIE 提供了两套量化方案可供用户灵活选择:

- 一个是 IGIE 支持直接导入已量化好的 ONNX 模型, 即将模型量化好之后再导入到 IGIE。用户可以选择使用任何量化框架, 包括 ONNXRuntime Quantization、PyTorch Quantization、TFLite 等开源量化框架, 只要能将量化后模型导出为 ONNX 格式即可。
- 另一个是使用 IGIE 自身内置的量化框架, 对导入后的 FP32 模型进行量化, 对于 CV 类模型有较好效果, 但有一定的局限性, 不支持 Per Channel 量化, 量化的控制粒度较大。

对于一些检测模型、NLP 模型, 对精度要求更高, 需要对模型中的部分算子进行更细粒度的量化控制, 甚至需要进行 QAT (量化感知训练)。IGIE 支持第三方开源量化框架, 只要量化后的模型能够以 ONNX 格式导出, 便可以导入到 IGIE, 通过这种方式支持更细粒度、更全面的量化控制。

比如: 使用 ONNXRuntime 可以控制哪些 op node 做量化, 哪些 op node 不做量化。当发现模型 INT8 量化导致精度损失时, 推荐 ONNXRunTime 进行量化排查。

总结:

- FP16 量化, 建议使用 IGIE 量化, 指定 precision=fp16 即可。
- INT8 量化, ONNX 模型推荐使用 ONNXRuntime 量化, 对 PyTorch/TensorFlow/PaddlePaddle 等框架的模型, 推荐使用 IGIE 内置量化。

4.3.5 附录: 量化术语介绍

量化是指对模型的权重 (weight) 和数据 (activation) 进行低比特处理, 让最终生成的网络模型更加轻量化, 从而达到节省网络模型存储空间、降低传输时延、提高计算效率, 达到性能提升与优化的目标。

量化根据是否需要重训练, 分为训练后量化 (Post-Training Quantization) 和量化感知训练 (Quantization-Aware Training)。

量化	训练后量化 PTQ	量化感知训练 QAT
概念	训练后量化 (Post-Training Quantization, 简称 PTQ), 是指将训练后模型中的权重由浮点数 (当前支持 float32) 量化到低比特整数 (当前支持 int8), 并通过少量校准数据基于推理过程对数据 (activation) 进行校准量化。	量化感知训练 (Quantization-Aware Training, 简称 QAT), 是指借助用户完整训练数据集, 在训练过程中引入量化操作, 通过在训练前向计算中对数据和权重进行伪量化 (Q/DeQ), 引入量化误差损失, 从而在训练过程中提高模型对量化效应的适应能力, 提高最终的量化模型精度。
对于激活数据 (activation) 的量化	需要校准。对于激活数据的量化, 只有在推理过程中才知道输入数据的分布情况, 因此需要校准的过程: 使用一部分推理数据来代表推理过程中整体的数据分布情况, 计算激活数据的量化系数。	在模型中加入伪量化层, 将待量化层 (如卷积) 的数据做伪量化处理后, 再进行对应的计算 (如卷积计算)。在训练过程中, 训练伪量化层的参数和模型原有的参数, 使得模型在量化的处理下也能达到一个较好的精度。此方法耗时长, 但量化精度损失小, 适用于做训练后量化精度损失大的场景。
对于权重 (weight) 的量化	由于是训练后的模型, 权重是固定的, 因此可以根据权重数据的分布情况直接计算出权重的量化系数。	一样需要在模型中加入伪量化层。

校准

训练后量化场景中, 做前向推理获取数据量化因子的过程。由于是训练后的模型, 权重是固定的, 因此可以根据权重数据的分布情况直接计算出权重的量化因子。对于激活数据的量化, 只有在推理过程中才知道输入数据的分布情况, 因此需要校准的过程: 使用一部分推理数据来代表推理过程中的数据分布情况, 进行计算激活数据的量化因子或者系数。

校准数据集

训练后量化场景中, 做前向推理使用的数据集。该数据集的分布代表着所有激活数据集的分布, 获取校准集时应该具有代表性, 推荐使用测试集的子集作为校准数据集。如果数据集不是模型匹配的数据集或者代表性不够, 则根据校准集计算得到的量化因子, 在全数据集上表现较差, 量化损失大, 量化后精度低。

4.4 教程 3: 如何自动优化模型并导出引擎文件

之前我们介绍了“教程 1: 如何导入模型到 IGIE”和“教程 2: 如何进行模型量化”。

现在介绍如何使用 IGIE 自动优化模型并导出引擎文件。

IGIE 拥有自动图优化机制, 能够自动地对导入模型进行优化。主要包括自动算子融合, 冗余算子消除, 常量折叠, 内存重用, layout 转换, 自动 padding 等, 编译得到符合天数智芯加速卡硬件特性, 性能更优的引擎文件, 用于最终部署推理。

而这一切的优化都是由 IGIE 自动完成的，用户只需调用 `relay.build` 将导入的模型编译成引擎文件即可。

4.4.1 1. 自动优化模型并编译成引擎文件

```
# 输入模型信息
model_path="./resnet50.onnx"
input_name="input0"
input_shape=[32, 224, 224, 3]

# 1) 导入模型到 IGIE
onnx_model = onnx.load(model_path)
shape_dict = {input_name: input_shape}
mod, params = relay.frontend.from_onnx(onnx_model, shape_dict)

# 2) 设置运行模型的设备信息
target = tvn.target.iluvatar(options="-libs=cudnn,cublas,ixinfer")
device = tvn.device(target.kind.name, 0)

# 3) 自动优化模型，编译模型为引擎文件
precision = "fp16"
engine = relay.build(mod, params=params, target=target, precision=precision,
    verbose=False)

# 4) 查看编译后的 graph function
print(engine.function_metadata["__tvm_main__"])
```

我们通过 `tnv.target.iluvatar` 设置 IGIE 运行的天数智芯目标设备信息。使用 `device = tvn.device(target.kind.name, 0)` 创建 device，指定了 IGIE 运行在第几个 GPU 设备上，这里设置为设备 0。接下来将以上参数传递给 `relay.build` 函数，开始优化和编译模型的引擎文件。

“relay.build” 函数说明

编译和优化模型为引擎文件。

```
def relay.build(mod, parmas=None, target=None, precision="", device=None, verbose=False)
```

参数说明

- mod: `import_model_to_igie` 接口返回的模型 module 对象。
- params: `import_model_to_igie` 接口返回的模型参数对象。
- target: IGIE 运行的天数智芯目标设备信息。
一般固定设置为 `target = tvn.target.iluvatar(options="-libs=cudnn,cublas,ixinfer")`。
- precision: 模型运行精度，可设置 fp16 和 int8。
- device: IGIE 运行的 GPU 设备信息，可以指定运行在第几个设备上。
一般固定设置为 `device = tvn.device(target.kind.name, 0)`。

- verbose: 输出编译 log 信息。如果设置为 True, 则保存编译过程 log 文件到当前 module 目录。默认为 False。

返回值

返回优化和编译好的引擎文件对象, 用于下一步推理。

engine: 模型引擎文件对象。

4.4.2 2. 导出或导入引擎文件

```
# 导出引擎文件
engine.export_library("./resnet50_engine.so")

# 导入引擎文件
engine = tvn.runtime.load_module("./resnet50_engine.so")
```

4.5 教程 4: 如何加载引擎文件进行推理 (Python)

在之前“教程 3: 如何自动优化模型并导出引擎文件”中, 介绍了如何自动优化模型并导出引擎文件。现在我们可以直接使用引擎文件用于部署推理。

```
import tvn
import numpy as np
from tvn.contrib import graph_executor

input_name="input0"
input_shape=[32, 224, 224, 3]

# 设置测试 target 和 GPU 设备
target = tvn.target.iluvatar(options="-libs=cudnn,cublas,ixinfer")
dev = tvn.device(target.kind.name, 0)

# 加载引擎文件
engine_file = "./resnet50_engine.so"
engine = tvn.runtime.load_module(engine_file)
m = graph_executor.GraphModule(engine["default"] (dev)) # 加载引擎文件到 GPU Device

# 设置模型输入
image = np.random.uniform(size=input_shape).astype("float32")
image = tvn.nd.array(image, dev)
m.set_input(input_name, image)

# 推理模型
m.run()

# 输出推理结果
```

```
igie_output = m.get_output(0)
print(igie_output.numpy())
```

4.6 教程 5: 如何加载引擎文件进行推理 (C++)

回顾之前几节教程，使用 Python 开发完整的模型推理流程为：

1. 模型导入
2. 模型量化
3. 自动图优化
4. 导出引擎文件
5. 推理部署

目前 IGIE 的 C++ 接口更倾向于用于推理部署。即：

- 先使用 Python 对模型进行量化、导入、自动图优化等操作，最终生成可部署的引擎文件（即第 1 步到第 4 步）。
- 再使用 C++ 接口直接加载生成好的引擎文件进行模型推理（第 5 步）。

这样仅需要一个引擎文件便能使用 C++ 接口快速部署模型，一是方便开发者在 Python 侧调试模型，二是简化模型部署开发的代码量。

C++ 接口使用示例如下：

4.6.1 步骤 1：引入必要的头文件

```
#include <dlpack/dlpack.h>
#include <tvm/runtime/module.h>
#include <tvm/runtime/packed_func.h>
#include <tvm/runtime/registry.h>
#include <cstdio>
```

4.6.2 步骤 2：创建 Target 和 Device

创建一个 Iluvatar Target，用于编译模型使用。

```
auto target = Target("iluvatar")
```

创建 Device，用于指定 op 和 tensor 的运行设备。

```
DLDevice dev{kDLILUVATAR, 0};
```

Target 接收一个字符串来创建 Target，这里使用“iluvatar”来表示创建 Iluvatar gpu 后端，使用“llvm”来表示创建 LLVM CPU 后端。

DLDevice 接收两个参数，第一个参数是设备标志符，kDLILUVATAR 代表 Iluvatar GPU 设备，第 2 个参数是设备 ID，这里设为 GPU 设备 0。

下面是 DLDevice 的结构定义：

```

/!!
 * \brief A Device for Tensor and operator.
 */
typedef struct {
    /!! \brief The device type used in the device. */
    DLDeviceType device_type;
    /!! \brief The device index */
    int device_id;
} DLDevice;

```

4.6.3 步骤 3：创建 Tensor

Tensor 是一个 N 维的数组，主要用于存放模型的输入和输出数据。

1. 使用 `tvm::runtime::NDArray::Empty(ShapeTuple shape, DLDataType dtype, Device dev)` 创建一个空 Tensor。

例如：创建一个 `shape=[4]`，`dtype` 类型为 `float32`，在 Iluvatar GPU 上的 Tensor。

```

// 设置 Tensor 位于的设备
DLDevice dev{kDLILUVATAR, 0};

// 设置 Tensor 的类型
DLDataType dtype{kDLFloat, 32, 1};
tvm::runtime::NDArray input = tvm::runtime::NDArray::Empty({4}, dtype, dev);

// copy data from cpu to gpu
float data[4] = {1.0, 2.0, 3.0, 4.0};
input.CopyFromBytes(data, 4 * sizeof(float));

```

2. 使用 `tvm::runtime::NDArray::FromDLPack` 创建 Tensor。IGIE 支持以零拷贝的方式创建 Tensor：

该接口支持以零拷贝的方式构造 IGIE Tensor，void *data 可以是 CUDA Memory 数据指针。

```

tvm::runtime::NDArray CreateNDArray(void *data,
    ShapeTuple &shape,
    const DLDataType &dtype,
    const DLDevice &dev) {

    // 创建 DLTensor
    DLTensor tensor;
    tensor.data = data;
    tensor.device = dev;
    tensor.ndim = static_cast<int>(shape.size());
}

```

```

tensor.shape = const_cast<int64_t *>(shape.data());
tensor.dtype = dtype;
tensor.strides = nullptr;
tensor.byte_offset = 0;

// 创建 DLManagerTensor (DLPack 结构体)
DLManagedTensor managed_tensor;
managed_tensor.dl_tensor = tensor;
managed_tensor.deleter = nullptr;

tvm::runtime::NDArray x_array =
↪ tvm::runtime::NDArray::FromDLPack(&managed_tensor);
return x_array;
}

// GPU 内存的 data
float *d_data;
cudaMalloc((void**)&d_data, size * sizeof(float));
cudaMemcpy(d_data, h_data, size * sizeof(float), cudaMemcpyHostToDevice);

// 调用 CreateNDArray 函数，零拷贝方式创建 IGIE Tensor,
ShapeTuple in_shape{1, 224, 224, 3};
tvm::runtime::NDArray in_tensor;
in_tensor = CreateNDArray(mag_tensor, d_data, in_shape, DLDataType{kDLFloat, 32, 1},
↪ dev);

```

4.6.4 步骤 4：加载引擎文件

使用 `tvm::runtime::Module::LoadFromFile(file_name, format)` 加载引擎文件。

```

// 加载引擎文件到 IGIE 中
tvm::runtime::Module mod_factory = tvm::runtime::Module::LoadFromFile("resnet50.so");

// 创建 runtime context
tvm::runtime::Module gmod = mod_factory.GetFunction("default")(dev);

```

4.6.5 步骤 5：获取模型推理函数

使用 `gmod.GetFunction(function_name)` 获取推理函数。

```

// 设置模型输入函数
tvm::runtime::PackedFunc set_input = gmod.GetFunction("set_input");
// 获取模型输出函数
tvm::runtime::PackedFunc get_output = gmod.GetFunction("get_output");
// 模型运行函数
tvm::runtime::PackedFunc run = gmod.GetFunction("run");

```

4.6.6 步骤 6：设置模型输入

```
set_input("input_0", in_tensor);
```

4.6.7 步骤 7：创建计时器

```
tvm::runtime::Timer timer = tvm::runtime::Timer::GetTimer(dev);
```

4.6.8 步骤 8：运行模型

```
timer->Start();
run();
timer->Stop();
```

4.6.9 步骤 9：获取模型输出并统计耗时

```
float latency = timer->SyncAndGetElapsedNanos() / 1e6; // ns->ms
float FPS = 1000 / time;

// 创建 Output Tensor
ShapeTuple out_shape{1, 1000};
out_tensor = tvm::runtime::NDArray::Empty(out_shape, DLDataType{kDLFloat, 32, 1}, dev);
get_output(0, out_tensor);
```

4.6.10 IGIE C++ Inference 完整代码示例

Note

IGIE C++ 示例代码不在本发布包中。如有需要，请向您的应用工程师获取示例代码。

下面是完整的 C++ 代码示例：

```
#include <dlpack/dlpack.h>
#include <tvm/runtime/module.h>
#include <tvm/runtime/packed_func.h>
#include <tvm/runtime/registry.h>
#include <tvm/runtime/profiling.h>
#include <cstdio>
#include <stdlib.h>
```

```
#include <iostream>
#include <iomanip>
#include <string>
#include <fstream>
#include <errno.h>
#include <exception>

// 读取图片数据到 data
bool GetImageData(std::string file_path, float *data, uint32_t size) {
    try {
        std::ifstream file;
        file.exceptions(file.failbit | file.badbit);
        file.open(file_path);
        if (file.is_open()) {
            float tmp;
            uint32_t count = 0;
            while (file >> tmp) {
                *data++ = tmp;
                count++;
                if (count >= size) {
                    break;
                }
            }
            file.close();
            return true;
        }
    } catch (std::ifstream::failure &e) {
        std::cout << "open file failed: " << e.what() << std::endl;
        return false;
    }
    return false;
}

// 保存推理结果
bool SaveOutput(const std::string &file_path, float *data, uint32_t size) {
    try {
        std::ofstream out_file;
        out_file.exceptions(out_file.failbit | out_file.badbit);
        out_file.open(file_path);
        if (out_file.is_open()) {
            float tmp;
            for (uint32_t i = 0; i < size; ++i) {
                tmp = *data++;
                out_file << tmp;
                out_file << '\n';
            }
            out_file.close();
            return true;
        }
    } catch (std::ifstream::failure &e) {
```

```

    std::cout << "open file failed: " << e.what() << std::endl;
    return false;
}
return false;
}

// 使用引擎文件部署模型
void DeployResnet50(std::string in_file, std::string out_file, std::string dev_type =
    "gpu") {
    LOG(INFO) << "Running C++ resnet50 graph executor...";

    // 读取测试图片数据
    float *data = new float[1 * 3 * 224 * 224];
    if (!GetImageData(in_file, data, 1*3*224*224)) {
        std::cout << "Get data failed!" << std::endl;
        return;
    }

    // 设置 device
    DLDevice dev{kDLILUVATAR, 0};
    tvm::runtime::Module mod_factory;

    // 加载引擎文件
    if (dev_type == "cpu") {
        dev.device_type = kDLCPU;
        mod_factory = tvm::runtime::Module::LoadFromFile("model/resnet50_b1_llvm.so");
    } else if (dev_type == "gpu") {
        dev.device_type = kDLILUVATAR;
        mod_factory = tvm::runtime::Module::LoadFromFile("model/resnet50_b1_iluvatar.so");
    } else {
        std::cout << "ERROR: unrecognized device type!" << std::endl;
        return;
    }

    // 获取模型推理函数
    tvm::runtime::Module gmod = mod_factory.GetFunction("default")(dev);
    tvm::runtime::PackedFunc set_input = gmod.GetFunction("set_input");
    tvm::runtime::PackedFunc get_output = gmod.GetFunction("get_output");
    tvm::runtime::PackedFunc run = gmod.GetFunction("run");

    // 创建空的 Tensor
    tvm::runtime::NDArray input = tvm::runtime::NDArray::Empty({1, 3, 224, 224},
    DLDataType{kDLFloat, 32, 1}, dev);
    tvm::runtime::NDArray output_gpu = tvm::runtime::NDArray::Empty({1, 1000},
    DLDataType{kDLFloat, 32, 1}, dev);

    // 创建 Timer
    tvm::runtime::Timer timer = tvm::runtime::Timer::GetTimer(dev);

    // Warm Up

```

```
std::cout << "Start Warmup ..." << std::endl;
for(size_t i = 0; i < 3; i++)
{
    run();
}

// 拷贝 Host data 到 GPU input tensor 中 (HToD)
input.CopyFromBytes(data, 1 * 3 * 224 * 224 * sizeof(float));

// 设置模型输入
set_input("data", input);

// 推理模型
timer->Start();
run();
timer->Stop();

// 计算耗时
float latency = timer->SyncAndGetElapsedNanos() / 1e6; // ns->ms
float FPS = 1000 / time;
std::cout << "Consume time: " << latency << " ms" << std::endl;
std::cout << "FPS: " << FPS << std::endl;

// 获取模型输出
get_output(0, output_gpu);

// 拷贝 GPU Tensor 数据到 Host output 中 (DtoH)
float output[1000];
output_gpu.CopyToBytes((void*)output, 1000 * sizeof(float));

for (int32_t i = 0; i < 10; ++i) {
    std::cout << ((float*)output)[i] << '\t';
}
std::cout << std::endl;

// save inference output to postprocess
SaveOutput(out_file, output, 1000);

if (data != nullptr) {
    LOG(INFO) << "free data when device type is cuda!!!";
    delete[] data;
}
LOG(INFO) << "End running resnet50 graph executor...";
}

int main(int argc, char *argv[]) {
    if (argc != 4) {
        std::cout << "ERROR: two arguments for main, but get " << argc - 1 << std::endl;
        return 1;
    }
}
```

```

}
std::string in_file(argv[1]);
std::string out_file(argv[2]);
std::string dev_type(argv[3]);

std::cout << "in_file: " << in_file << ", out_file: " << out_file << std::endl;
DeployResnet50(in_file, out_file, dev_type);
return 0;
}

```

执行:

```

$ cd igie/tests/test_iluvatar/test_cpp/
$ bash run.sh

```

4.7 教程 6：如何使用 igie-exec 生成引擎并推理

igie-exec 是 IGIE 提供的一套基于命令行调用的模型快速部署和验证工具，方便用户使用一行命令运行 FP32/FP16/INT8 模型推理和生成相应的引擎文件，并进行性能验证。

当前 igie-exec 内置了 imagenet 和 coco2017 数据集，方便对使用这两种标准数据集的模型进行快速精度测试，而其它模型则使用随机数据集进行性能测试。以下是 igie-exec 的部分使用示例，具体说明请参考“igie-exec 模型快速部署和验证工具使用指南”。

4.7.1 示例 1：运行 PyTorch ResNet50 模型

```

$ igie-exec --model_path data/models/resnet/resnet50-fp32.pt --input input:32,3,224,224
↪ --precision int8 --use_imagenet True

```

执行结果

```

{'acc@1': 0.7604633482714469,
 'acc@5': 0.9275568181818182,
 'acc_result': 0.7604633482714469,
 'fps_result': 7623.5377158913625}

```

4.7.2 示例 2：运行 YOLOv5 ONNX 模型

```

$ igie-exec --model_path data/models/yolov5/yolov5m.onnx --input images:32,3,640,640
↪ --precision int8 --use_coco2017 True --automatic_yolo_quantization True

```

执行结果

```

{'acc_result': 0.6236765361287956, 'fps_result': 1094.5590025580452}

```

```
# YOLOv5m + GPU NMS 处理
$ igie-exec --model_path data/models/yolov5/yolov5m.onnx --input images:32,3,640,640
↪ --precision int8 --use_coco2017 True --automatic_yolo_quantization True
↪ --custom_option with_nms:True iou_thres:0.65 conf_thres:0.001 topk:100
```

4.7.3 示例 3：运行 ONNX BERT-Base Squad 模型

BERT-Base Squad 是一个多输入 NLP 模型，下面展示生成其 FP16 的引擎文件，并使用随机数据进行性能测试的过程。

向您的应用工程师获取数据集。源模型来源于 <https://huggingface.co/csarron/bert-base-uncased-squad-v1>。

```
# 运行模型
$ igie-exec --model_path data/models/bert/bert-base-uncased-squad-v1.onnx --input
↪ input_ids:8,256 attention_mask:8,256 token_type_ids:8,256 --precision fp16

# 执行结果
[2023-05-18 03:36:24,299 igie-exec line:67] INFO: Evaluate inference igie time cost...
[2023-05-18 03:36:24,638 igie-exec line:70] INFO: Mean inference time (std dev): 9.773 ms
↪ (0.016 ms)
[2023-05-18 03:36:24,638 igie-exec line:72] INFO: Mean fps: 818.612
```

4.8 教程 7：IGIE 完整推理说明及示例

4.8.1 1. IGIE Fallback 机制说明

目前 IGIE 已经增加了后端选择的自动 Fallback 机制，支持特性如下：

- 当前 IxRT 兼容 TensorRT API，IGIE 也增加了 IxRT 后端的支持，导入模型时可以指定 IxRT 后端，来使用 IxRT 推理引擎编译和运行模型。
- 当使用 ONNX 模型进行推理时，默认使用 IxRT Backend。如果 IxRT 不支持该模型，则自动 Fallback 到 IGIE Backend 执行，该过程将自动化完成，无需您操作。

Note

当前 IxRT Backend 只支持 ONNX 模型的导入推理，因此上述的 IGIE Fallback 机制只支持 ONNX 模型自动 Fallback，其他类型格式的模型 (如 TensorFlow, PyTorch, PaddlePaddle) 会直接使用 IGIE Backend 执行。

IGIE 增加 Fallback 机制后的最终效果示例如下：

```
# 测试 ResNet50 FP16 推理，默认使用 IxRT Backend 运行
$ python3 inference.py --model_path=resnet50.onnx --input input:32,3,224,224 --precision
↪ fp16 --use_imagenet True
```

```
# 测试 ResNet50 INT8 推理，默认使用 IxRT Backend 运行，如果检测到 IxRT 编译该模型失败，自动
↪ Fallback 到 IGIE Backend 执行
$ python3 inference.py --model_path=resnet50.onnx --input input:32,3,224,224 --precision
↪ int8 --use_imagenet True
```

Note

参考2.4 完整推理代码示例 获取上述测试的详细代码。

4.8.2 2. 如何进行推理

IGIE 推理流程分为 导入模型到 IGIE -> 优化和编译模型引擎文件 -> 推理模型等步骤。下面我们一一进行使用介绍。

4.8.2.1 2.1 导入模型到 IGIE

当前 IGIE 支持各大主流深度学习框架模型 (如 TensorFlow、PyTorch、ONNX 模型) 的直接导入，在“教程 1: 如何导入一个模型到 IGIE”中，我们介绍了各个框架模型的导入方法。

为了方便地导入模型，我们整合了所有框架地导入接口，您仅需使用一个 `import_model_to_igie` 函数即可导入 ONNX、PyTorch、TensorFlow、PaddlePaddle 等模型到 IGIE。

import_model_to_igie 的使用方法

```
# 首先 import 函数
from tvm.relay.import_model import import_model_to_igie

### 设置模型信息
batch_size = 32
model_path_or_name = "./resnet50.onnx"
input_dict = {"input": ([32, 3, 224, 224], "float32")}
precision = "fp16"

# 使用 IGIE Backend 推理
mod, params = import_model_to_igie(model_path_or_name, inputs_info=input_dict,
↪ outputs_info=None, precision=precision, backend="igie")

# 使用 IxRT Backend 推理
output_dict = {"output": "float32"}
mod, params = import_model_to_igie(model_path_or_name, inputs_info=input_dict,
↪ outputs_info=output_dict, precision=precision, backend="ixrt")
```

import_model_to_igie 的函数说明

使用该接口，可直接导入 ONNX、TensorFlow、PyTorch、TorchVision、PaddlePaddle 模型到 IGIE 中。

```
def import_model_to_igie(model_path_or_name, inputs_info, outputs_info=None,
    ↪ precision=None, backend="default", dynamic_info=None)
```

参数说明

- `model_path_or_name`: 根据自己的模型类型设置 模型路径或 模型目录或 模型名称等字符串。详细示例如下:
 - 模型文件路径:
 - * ONNX 模型: 例如 `/path/to/resnet50.onnx`
 - * PyTorch 模型: 例如 `/path/to/resnet50.pt`
 - * TensorFlow 模型: 例如 `/path/to/resnet50.pb`
 - 模型文件目录: 以 目录/模型名称的形式指定 `model_path_or_name` 参数。例如:
 - * PaddlePaddle 模型文件: `resnet50/model`。目录下需要包含 `resnet50/model.pdiparams`、`model.pdiparams.info` 和 `model.pdmodel` 文件。
 - 模型名称: 如果传入参数为模型名称, 如 `resnet50`、`vgg16`, 该接口会自动从 TorchVision 中寻找并加载 torch pre-train 模型。

- `inputs_info`: 模型的输入信息。

期望以 `inputs_info={'name': (shape, dtype)}` 的形式设置 `inputs_info`。例如:

```
inputs_info = {
    'input0': ([32, 3, 224, 224], "float32"),
    'input1': ([32, 256], "int32")
}
```

- `outputs_info`: 仅在使用 IxRT Backend 时需要设置, 设置模型的输出信息, 默认值为 `None`。

期望以 `outputs_info={'name': dtype}` 的形式设置 `outputs_info`。例如:

```
outputs_info={'output0': "float32", 'output1': "int32"}
```

- `precision`: 仅在使用 IxRT Backend 时需要设置, 设置模型的推理精度, 默认值为 `None`。
例如: `precision="fp16"`。
- `dynamic_info`: 仅在使用 IxRT Backend 时需要设置, 设置模型的输入为动态 shape, 默认值为 `None`。
例如:

```
dynamic_info={
    'min_shape': {'input0': ([1, 3, 224, 224], "float32"), 'input1': ([1,
    ↪ 256], "int32")},
    'opt_shape': {'input0': ([32, 3, 224, 224], "float32"), 'input1':
    ↪ ([32, 256], "int32")},
    'max_shape': {'input0': ([128, 3, 224, 224], "float32"), 'input1':
    ↪ ([128, 256], "int32")}
}
```

- `backend`: 直接指定模型推理后端, 默认值为 `default`。
 - `backend="default"`, 表示默认使用 IxRT Backend。IxRT 运行失败后, Fallback 到 IGIE Backend 执行。

- backend="ixrt", 表示使用 IxRT Backend 执行模型推理。。
- backend="igie", 表示使用原有的 IGIE Backend 执行模型推理。

返回值

返回 mod 和 params 两个对象，用来下一步进行优化和编译引擎文件。

- mod: 模型结构对象。
- params: 模型参数对象。

4.8.2.2 2.2 优化和编译模型引擎文件

当导入的模型完成后，开始第 2 步，对导入模型进行优化和编译，生成引擎文件。

```
# 1) 导入模型
from tvm.relay.import_model import import_model_to_igie
model_path_or_name = "./resnet50.onnx"
input_dict = {"input": ([32, 3, 224, 224], "float32")}
precision = "fp16"
mod, params = import_model_to_igie(model_path_or_name, inputs_info=input_dict,
    ↪ outputs_info=None, precision=precision)

# 2) 设置运行模型的设备信息
target = tvm.target.iluvatar(options="-libs=cudnn,cublas,ixinfer")
device = tvm.device(target.kind.name, 0)

# 3) 编译引擎文件
engine = relay.build(mod, params=params, target=target, precision=precision,
    ↪ device=device)
```

首先，我们通过 `tvm.target.iluvatar` 设置 IGIE 运行的天数智芯目标设备信息。使用 `device = tvm.device(target.kind.name, 0)` 创建 device，指定了 IGIE 运行在第几个 GPU 设备上，这里设置为设备 0。

Note

创建 Target 时 options 参数设置：

当使用 IGIE Backend 时，可以通过 `options="-libs=cudnn,cublas,ixinfer"` 参数来选择使用哪些算子库实现 (关于 IGIE 使用算子库的详细说明，请查阅“附录：IGIE 框架算子支持列表”)。

而当使用 IxRT Backend 时，该 options 参数不会生效。

接下来，将以上参数传递给 `relay.build` 函数，开始优化和编译模型的引擎文件。

relay.build 函数说明

编译和优化模型为引擎文件。

```
def relay.build(mod, params=None, target=None, precision="", device=None)
```

参数说明

- mod: import_model_to_igie 接口返回的模型 module 对象。
- params: import_model_to_igie 接口返回的模型参数对象。
- target: IGIE 运行的天数智芯目标设备信息。
一般固定设置为 target = tvm.target.iluvatar(options="-libs=cudnn,cublas,ixinfer")。
- precision: 模型运行精度, 可设置 fp16 和 int8。
- device: IGIE 运行的 GPU 设备信息, 可以指定运行在第几个设备上。
一般固定设置为 device = tvm.device(target.kind.name, 0)。

返回值

返回优化和编译好的引擎文件对象, 用于下一步推理。

engine: 模型引擎文件对象。

支持导入或导入引擎文件

```
# 导出引擎文件
engine.export_library("./resnet50.engine")
# 导入引擎文件
engine = tvm.runtime.load_module("./resnet50.engine")
```

4.8.2.3 2.3 推理模型

在编译完引擎文件后, 可以使用该引擎文件推理模型, 推理过程分为 设置模型输入 -> 运行模型 -> 获取模型输出等步骤。使用方法如下:

```
engine = relay.build(mod, params=params, target=target, precision=precision,
    ↪ device=device)
# 加载引擎文件到 GPU 设备
module = graph_executor.GraphModule(engine["default"](device))

print("\n预热模型推理...")
for i in range(3):
    module.run()

print("\n开始推理模型...")
for i in range(10):
    # 设置模型输入
    np_data = np.random.randint(low=0, high=255, size=[batch_size, 224, 224,
    ↪ 3]).astype("float32")
    module.set_input("input", np_data)

    # 推理模型
    module.run()

    # 输出推理结果, 以 numpy 数组的形式返回模型输出结果 output
    output = module.get_output(0)
```

```
device.sync()
print(" 输出推理结果: ", output)
```

首先, 通过 `module = graph_executor.GraphModule(engine["default"](device))` 将引擎文件加载到 GPU 设备中, 并创建 `GraphModule` 用于执行模型。

接下来需要使用 `set_input`、`run()` 和 `get_output` 接口来运行模型。

1. set_input() 函数

函数功能

设置模型输入。

当模型有多个输入时, 可以多次 `set_input(key, value)`, 也可以使用 `dict` 的形式一次性传入所有输入。

```
def set_input(key=None, value=None, **input_dict)
```

参数说明

- `key`: 模型输入的 `name` 字符串。
- `value`: 模型输入 `name` 所对应的 `data numpy` 数据。
- `input_dict`: 一次性传入所有输入, 方式为 `key` 和 `value` 组成的字典, 例如:

```
input_dict = {'input0': np_data1, 'input1': np_data2}
module.set_input(input_dict)
```

返回值

无返回值。

2. run() 函数

函数功能

运行模型单次推理。

```
def run()
```

参数说明

无参数。

返回值

无返回值。

3. get_output() 函数

函数功能

获取模型单次推理的输出结果。

```
def get_output(index==1)
```

参数说明

index: 返回模型第几个输出。默认为-1，返回所有输出的 list。

返回值

返回模型的输出结果，以 numpy 数组的形式返回。

4.8.2.4 2.4 完整推理代码示例

Note

IGIE C++ 代码示例不在本发布包中。如有需要，请向您的应用工程师获取示例代码。

完成代码参考示例如下 (IGIE 使用 IxRT Backend 测试 ResNet50 模型):

```
import numpy as np
import os
import sys
from pprint import pprint
from tqdm import tqdm

import tvn
from tvn import relay
from tvn.contrib import graph_executor
from tvn.relay.import_model import import_model_to_igie

# set util_dir python path
util_dir = os.path.dirname(__file__)
sys.path.insert(1, util_dir + "..")

import util
from util import Timer

def main():
    args = util.get_args()
    target, device = util.get_target(args.target)

    print("Run Model args: ")
    pprint(vars(args), indent=2)

    batch_size = args.batch_size
    precision = args.precision

    input_dict = args.input_dict
    model_path_or_name = args.model_path

    if not os.path.isfile(args.engine_path):
        if precision == "int8" and args.model_framework == "onnx":
```

```

        model_path_or_name = util.onnx_quantize_model_from_args(args)

        mod, params = import_model_to_igie(model_path_or_name, input_dict,
        ↪ outputs_info=None, precision=precision)
        lib = relay.build(mod, target=target, params=params, precision=precision,
        ↪ device=device, verbose=True)
        lib.export_library(args.engine_path)

    lib = tvml.runtime.load_module(args.engine_path)
    module = graph_executor.GraphModule(lib["default"](device))

    print("\nWarm-up...")
    for _ in range(args.warmup):
        module.run()

    print("\Strat inference...")
    batch_size = args.batch_size
    dataloader = util.get_dataloader_from_args(args)

    total_num = 0
    timer = Timer(device)

    if args.use_imagenet:
        top1_acc = 0
        top5_acc = 0

        for image, label in tqdm(dataloader):
            image = np.array(image)

            module.set_input(args.input_name_list[0], image)

            timer.start()
            module.run()
            timer.stop()

            pred = module.get_output(0)

            if not hasattr(module, "runtime"):
                pred = pred.numpy()

            ## batch accuracy
            batch_top1_acc, batch_top5_acc = util.get_topk_accuracy(pred, label)
            top1_acc += batch_top1_acc
            top5_acc += batch_top5_acc
            total_num += batch_size

            ## batch performance
            batch_infer_time = timer.last_duration # ms
            batch_infer_fps = (1000 * batch_size / batch_infer_time)

```

```

        logging.info(f"* Inference Latency: {batch_inferenc_time:.3f} ms, Inference
↪ FPS: {batch_inferenc_fps:.3f}")
        logging.info(f"* Acc @1 {top1_acc/total_num:.5f} Acc @5 {top5_acc/total_num:.
↪ 5f}")

if __name__ == "__main__":
    main()

```

执行代码：

```

#!/bin/bash
current_dir=`pwd`

if [[ ! -d ../data ]]; then
    cd ..
    bash prepare_data.sh
    cd ${current_dir}
fi

# 测试 ResNet50 FP16 推理，默认使用 IxRT Backend 运行
$ python3 inference.py --model_path=../data/models/resnet50/resnet50.onnx --input input:
↪ 32,3,224,224 --precision fp16 --use_imagenet True

# 测试 ResNet50 INT8 推理，默认使用 IxRT Backend 运行，如果检测到 IxRT 编译该模型失败，自动
↪ Fallback 到 IGIE Backend 执行
$ python3 inference.py --model_path=../data/models/resnet50/resnet50.onnx --input input:
↪ 32,3,224,224 --precision int8 --use_imagenet True

```

4.9 教程 8：快速上手 Relax 进行模型推理

当前 IGIE v0.18.0 版本已经支持了一部分动态模型 (如 CV, NLP) 的推理问题，后续将会支持更多动态模型。

下面代码演示一个通过使用 Relax ONNX frontend 前端导入 ONNX 模型，编译并加载到 Relax 虚拟机中完成推理的完整的 ResNet50 动态模型推理过程，同一个虚拟机可以处理不同的 batch 类型。

```

import onnx
import tvm
from tvm import relax, tir
from tvm.relax.frontend.onnx import from_onnx
from tvm.relax.transform.iluvatar import optimize_with_target
import numpy as np
import torch
import torchvision

target = tvm.target.iluvatar(model="MR", options="-libs=cublas,ixinfer")
device = tvm.device("iluvatar", 0)

```

```
# wget https://download.pytorch.org/models/resnet50-0676ba61.pth
model = torchvision.models.resnet50()
model.load_state_dict(torch.load(r'resnet50-0676ba61.pth'))
model.eval()

input_names = ['input']
output_names = ['output']
dynamic_axes = {'input': {0: 'batch'}, 'output': {0: 'batch'}}
dummy_input = torch.randn(1, 3, 224, 224)

torch.onnx.export(
    model,
    dummy_input,
    r'resnet50_dyn.onnx',
    input_names = input_names,
    dynamic_axes = dynamic_axes,
    output_names = output_names,
    opset_version=13
)

onnx_mod = onnx.load(r'./resnet50_dyn.onnx')
shape_dict = {"input": [tir.Var("batch", "int64"), 3, 224, 224]}

relax_mod = from_onnx(onnx_mod, shape_dict=shape_dict)
relax_mod = optimize_with_target(relax_mod, target=target, precision="float16",
    ↪ policy="dgs")

ex = relax.build(relax_mod, target=target)
vm = relax.VirtualMachine(ex, device=device)

inputs = tvm.nd.array(np.zeros([2, 3, 224, 224], dtype="float32"), device)
pred = vm["main"](inputs)
print(pred.numpy().shape)

inputs = tvm.nd.array(np.zeros([16, 3, 224, 224], dtype="float32"), device)
pred = vm["main"](inputs)
print(pred.numpy().shape)

# you can test other input shape here.
# ...
```

输出结果：

```
(2, 1000) (16, 1000)
```

更多关于 Relax 的使用详情，请参考“高级教程：IGIE 推理框架高级功能使用教程”>“教程 1：如何使用 Relax”。

4.10 教程 9：如何使用 IGIE 多进程推理

获取模型引擎文件后，在多推理进程应用场景下，可以参考如下示例。

Note

需要注意的是，目前单卡设备最多支持开启 15 个推理进程。

4.10.0.1 实现推理进程

```
def igie_inference_process(device_id, engine_path, input_data)
    try:
        # 设置进程执行 device
        device = tvm.device("iluvatar", device_id)

        # 加载 Engine 文件到 device
        lib = tvm.runtime.load_module(engine_path)
        module = graph_executor.GraphModule(lib["default"](device))

        # 设置模型输入
        # 需要根据模型真实 input_name 做对应修改
        module.set_input("input", tvm.nd.array(input_data, device))

        # 推理模型
        module.run()

        # 获取输出结果
        output = module.get_output(0)

        print(
            f"Process {multiprocessing.current_process().name}: "
            f"Inference completed on GPU {device_id}"
        )
    except Exception as e:
        print(
            f"Process {multiprocessing.current_process().name}: "
            f"Error during inference on GPU {device_id}: {e}"
        )
```

4.10.0.2 开启多进程推理

使用 multiprocessing 库创建多个推理进程。

```
# 设置 Device 数目
num_gpus = 1
```

```
# 设置单个 Device 执行的推理进程数目, 最大为 15
num_processes_per_gpu = 15

processes = []
for device_id in range(num_gpus):
    for i in range(num_processes_per_gpu):

        # 创建推理进程并传入所需参数
        p = multiprocessing.Process(
            target=igie_inference_process,
            args=(device_id, engine_path, input_data)
        )

        processes.append(p)
        p.start()

for p in processes:
    p.join()
```

完整推理代码如下：

```
import os
import multiprocessing

import tvn
from tvn.contrib import graph_executor
import numpy as np

test_num = 100

def igie_inference_process(device_id, engine_path, input_data):
    try:
        # 设置进程执行 device
        device = tvn.device("iluvatar", device_id)

        # 加载 Engine 文件到 device
        lib = tvn.runtime.load_module(engine_path)
        module = graph_executor.GraphModule(lib["default"](device))

        # 设置模型输入
        # 需要根据模型真实 input_name 做对应修改
        module.set_input("input", tvn.nd.array(input_data, device))

        # 推理模型
        module.run()

        # 获取输出结果
        output = module.get_output(0)
```

```

        print(
            f"Process {multiprocessing.current_process().name}: "
            f"Inference completed on GPU {device_id}"
        )

    except Exception as e:

        print(
            f"Process {multiprocessing.current_process().name}: "
            f"Error during inference on GPU {device_id}: {e}"
        )

def main():
    engine_path = "resnet50_batchsize_32_fp16.so"
    if not os.path.exists(engine_path):
        print(
            f"Model file '{engine_path}' not found!"
            f"Please provide a valid path."
        )

    return

    input_data = np.random.rand(32, 3, 224, 224).astype("float32")

    # 设置 Device 数目
    num_gpus = 1

    # 设置单个 Device 执行的推理进程数目, 最大为 15
    num_processes_per_gpu = 15

    processes = []
    for device_id in range(num_gpus):
        for i in range(num_processes_per_gpu):
            # 创建推理进程并传入所需参数
            p = multiprocessing.Process(
                target=igie_inference_process,
                args=(device_id, engine_path, input_data)
            )

            processes.append(p)
            p.start()

    for p in processes:
        p.join()

    print("All inference processes completed.")

if __name__ == "__main__":
    main()

```

参考输出如下：

```
Process Process-14: Inference completed on GPU 0
Process Process-12: Inference completed on GPU 0
Process Process-8: Inference completed on GPU 0
Process Process-10: Inference completed on GPU 0
Process Process-9: Inference completed on GPU 0
Process Process-11: Inference completed on GPU 0
Process Process-13: Inference completed on GPU 0
Process Process-15: Inference completed on GPU 0
Process Process-1: Inference completed on GPU 0
Process Process-7: Inference completed on GPU 0
Process Process-6: Inference completed on GPU 0
Process Process-4: Inference completed on GPU 0
Process Process-3: Inference completed on GPU 0
Process Process-5: Inference completed on GPU 0
Process Process-2: Inference completed on GPU 0
All inference processes completed.
```

5 igie-exec 模型快速部署和验证工具使用指南

igie-exec 项目是 IGIE 提供的一套基于命令行调用的模型快速部署和验证工具，方便用户一键运行 FP32/FP16/INT8 模型推理和生成相应的引擎文件。

5.1 igie-exec 功能说明

特性	说明
多框架支持	支持 ONNX、PyTorch、TensorFlow 多框架模型导入
多 batch 支持	支持设置不同 batch 推理测试
多精度支持	支持 FP32、FP16、INT8 精度推理测试
多个量化框架支持	目前支持两种 INT8 量化方式：IGIE 自身的 INT8 8 量化；使用 ONNXRuntime 进行 INT8 量化（仅支持 ONNX 模型）
引擎文件导出和导入	支持引擎文件导出和导入，方便测试部署
CV 类模型支持	对大多数 CV 类模型 FP32/ FP16/INT8 推理支持较好，部分模型使用 IGIE INT8 量化可能会出错，建议尝试 ONNXRuntime 量化
NLP 模型支持	支持 FP32、FP16 的 NLP 模型，目前适配部分 INT8 NLP 模型，正在增加更多模型适配
动态 shape 支持	目前 igie-exec 对于有动态 shape 的模型需要填充成固定输入进行测试

5.2 igie-exec 的使用指南

5.2.1 开始之前：准备环境

通过安装天数智算软件栈安装 igie-exec：

```
# 使用 .run 安装包或 Docker 镜像安装包安装天数智算软件栈
# (仅 .run 安装包形式需要) 通过 .whl 包安装 IGIE 框架，igie-exec 将随 .whl 包同时安装
# 确认是否已成功安装 igie-exec，在任意目录下执行
$ igie-exec -h # igie-exec 代码通常被放在 IGIE 的安装目录下，可以通过 pip3 show igie 查看

# 安装项目 Python 包依赖
$ pip3 install -r requirements.txt
# requirements.txt 内容如下
```

```

pytest
opencv-python==4.6.0.66
onnx==1.17.0
pycocotools
onnxoptimizer
onnxsim
loguru
tabulate
transformers
datasets
h5py
inflect
tqdm
argparse
bert4torch
pillow

# 向您的应用工程师获取数据集 data.tar.gz 并解压, 内含 imagenet/coco2017 标准数据集以及 ResNet, YOLO
# 等常见模型
$ tar xzf data.tar.gz
$ bash prepare_data.sh

```

5.2.2 测试模型推理性能

测试模型推理性能, 无需编程, 您只需要在任意目录下运行一行命令即可。下面是模型推理示例 (注意: 下面的性能数据仅供参考):

5.2.2.1 示例 1: 运行 PyTorch ResNet50

```

$ igie-exec --model_path data/models/resnet/resnet50-fp32.pt --input input:32,3,224,224
  --precision int8 --use_imagenet True

```

执行结果:

执行结果:

```

{ 'acc@1': 0.7604633482714469,
  'acc@5': 0.9275568181818182,
  'acc_result': 0.7604633482714469,
  'fps_result': 7623.5377158913625}

```

5.2.2.2 示例 2: 运行 PyTorch TorchVision

对于 PyTorch 模型, igie-exec 也内置了 torchvision.model 来下载预训练模型, 只要使用 --model_path efficientnet_b0 指定模型名称, 即可下载模型进行推理。TorchVision 支持预训练的模型列表可参考 [torchVision Models](#)。

- 运行 TorchVision ResNet18 模型

```
$ igie-exec --model_path resnet18 --input input:32,3,224,224 --precision fp16
↳ --use_imagenet True
```

执行结果:

```
{'acc@1': 0.6070342509603073,
 'acc@5': 0.8355873879641486,
 'acc_result': 0.6070342509603073,
 'fps_result': 19526.346486512382}
```

- 运行 TorchVision EfficientNet 模型:

```
$ igie-exec --model_path efficientnet_b0 --input input:32,3,224,224 --precision fp16
↳ --use_imagenet True
```

执行结果:

```
执行结果:
{'acc@1': 0.7768285851472471,
 'acc@5': 0.9357394366197183,
 'acc_result': 0.7768285851472471,
 'fps_result': 2800.5860386306513}
```

5.2.2.3 示例 3: 运行 TensorFlow VGG16

```
$ igie-exec --model_path data/models/vgg/vgg16.pb --input input:32,224,224,3 --precision
↳ int8 --use_imagenet True --input_layout NHWC
```

执行结果:

```
{'acc@1': 0.7006442061459667,
 'acc@5': 0.8942661651728553,
 'acc_result': 0.7006442061459667,
 'fps_result': 4093.361278897936}
```

5.2.2.4 示例 4: 运行 YOLOv3 ONNX

```
$ igie-exec --model_path data/models/yolov3/yolov3_nhwc.onnx --input "input/input_data:
↳ 0": "32,416,416,3" --precision int8 --use_coco2017 True --automatic_yolo_quantization
↳ True --input_layout NHWC
```

执行结果:

```
{'acc_result': 0.5967312564573963, 'fps_result': 1479.6737448863978}
```

5.2.2.5 示例 5: 运行 YOLOv5 ONNX

```
$ igie-exec --model_path data/models/yolov5/yolov5m.onnx --input images:32,3,640,640
↪ --precision int8 --use_coco2017 True --automatic_yolo_quantization True
↪ --custom_option required_pass:MutatePadMod --verbose True
```

执行结果:

```
{'acc_result': 0.6236765361287956, 'fps_result': 1094.5590025580452}
```

YOLOv5m + GPU NMS 处理:

```
$ igie-exec --model_path data/models/yolov5/yolov5m.onnx --input images:32,3,640,640
↪ --precision fp16 --use_coco2017 True --automatic_yolo_quantization True
↪ --custom_option with_nms:True iou_thres:0.65 conf_thres:0.001 topk:100
```

5.2.2.6 示例 6: 运行 YOLOv7 ONNX

```
$ igie-exec --model_path data/models/yolov7/yolov7_dynamic.onnx --input images:32,3,640,
↪ 640 --precision int8 --use_coco2017 True --automatic_yolo_quantization True
```

执行结果:

```
{'acc_result': 0.6851086809169135, 'fps_result': 643.7612413146849}
```

5.2.2.7 示例 7: 运行 ONNX BERT-Base Squad

BERT-Base Squad 是一个多输入 NLP 模型, 下面展示生成其 FP16 的引擎文件, 并使用随机数据进行性能测试的过程。

源模型来源于 <https://huggingface.co/csarron/bert-base-uncased-squad-v1>。

运行模型

```
$ igie-exec --model_path data/models/bert/bert-base-uncased-squad-v1.onnx --input
↪ input_ids:8,256 attention_mask:8,256 token_type_ids:8,256 --precision fp16
```

执行结果

```
[2023-05-18 03:36:24,299 igie-exec line:67] INFO: Evaluate inference igie time cost...
[2023-05-18 03:36:24,638 igie-exec line:70] INFO: Mean inference time (std dev): 9.773 ms
↪ (0.016 ms)
[2023-05-18 03:36:24,638 igie-exec line:72] INFO: Mean fps: 818.612
```

5.2.2.8 示例 8: 运行 ONNX BERT-Large Squad

BERT-Large Squad 是一个多输入 NLP 模型, 下面展示生成其 FP16 的引擎文件, 并使用随机数据进行性能测试的过程。

源模型来源于 <https://huggingface.co/neuralmagic/bert-large-uncased-finetuned-squadv1>。

运行模型

```
$ igie-exec --model_path data/models/bert/bert-large-uncased-finetuned-squadv1.onnx
  ↳ --input input_ids:8,384 attention_mask:8,384 token_type_ids:8,384 --precision fp16
```

执行结果:

```
[2023-05-18 03:38:44,399 igie-exec line:67] INFO: Evaluate inference igie time cost...
[2023-05-18 03:38:45,098 igie-exec line:70] INFO: Mean inference time (std dev): 44.010 ms
  ↳ (0.021 ms)
[2023-05-18 03:38:45,099 igie-exec line:72] INFO: Mean fps: 181.778
```

5.2.2.9 示例 9: 运行 ONNX Electra

Electra 是一个多输入 NLP 模型，下面展示生成其 FP16 的引擎文件，并使用随机数据进行性能测试的过程。

源模型来源于 <https://huggingface.co/bhadresh-savani/electra-base-emotion>。

运行模型

```
$ igie-exec --model_path data/models/electra/electra-base-emotion.onnx --input input_ids:
  ↳ 32,64 attention_mask:32,64 token_type_ids:32,64 --precision fp16
```

执行结果:

```
[2023-05-18 03:39:49,471 igie-exec line:67] INFO: Evaluate inference igie time cost...
[2023-05-18 03:39:49,743 igie-exec line:70] INFO: Mean inference time (std dev): 2.751 ms
  ↳ (0.003 ms)
[2023-05-18 03:39:49,743 igie-exec line:72] INFO: Mean fps: 11630.930
```

5.2.2.10 示例 10: 运行 ONNX ViT

ViT 是一个 Transformer 架构的 CV 模型，下面展示生成其 FP16 的引擎文件，并使用随机数据进行性能测试的过程。

源模型来源于 <https://huggingface.co/google/vit-base-patch16-224>。

运行模型

```
$ igie-exec --model_path data/models/vit/vit.onnx --input pixel_values:32,3,224,224
  ↳ --precision fp16
```

执行结果:

```
[2023-05-18 03:41:11,573 igie-exec line:67] INFO: Evaluate inference igie time cost...
[2023-05-18 03:41:12,519 igie-exec line:70] INFO: Mean inference time (std dev): 56.862 ms
  ↳ (0.037 ms)
[2023-05-18 03:41:12,519 igie-exec line:72] INFO: Mean fps: 562.767
```

5.2.2.11 示例 11: 运行 ONNX Swin Transformer

Swin Transformer 是一个 Transformer 架构的 CV 模型，下面展示生成其 FP16 的引擎文件，并使用随机数据进行性能测试的过程。

源模型来源于 <https://huggingface.co/microsoft/swin-small-patch4-window7-224>。

运行模型

```
$ igie-exec --model_path data/models/swin_transformer/swin_transformer.onnx --input
↪ pixel_values:32,3,224,224 --precision fp16
```

执行结果:

```
[2023-05-18 03:46:26,360 igie-exec line:67] INFO: Evaluate inference igie time cost...
[2023-05-18 03:46:27,028 igie-exec line:70] INFO: Mean inference time (std dev): 32.053 ms
↪ (0.031 ms)
[2023-05-18 03:46:27,028 igie-exec line:72] INFO: Mean fps: 998.360
```

5.2.2.12 示例 12: 运行 ONNX Conformer

Conformer 是一个 Audio 类的语音识别模型，下面展示生成其 FP16 的引擎文件，并使用随机数据进行性能测试的过程：

```
$ igie-exec --model_path data/models/conformer/encoder_bs24_seq384_static_opt_matmul.onnx
↪ --input speech:24,384,80 speech_lengths:24 --precision fp16
```

执行结果:

```
[2023-05-18 03:48:20,999 igie-exec line:67] INFO: Evaluate inference igie time cost...
[2023-05-18 03:48:21,720 igie-exec line:70] INFO: Mean inference time (std dev): 39.498 ms
↪ (0.026 ms)
[2023-05-18 03:48:21,720 igie-exec line:72] INFO: Mean fps: 607.627
```

Tip

如果把最后一个 topk 算子 offload 到模型外，性能将提升到 1200FPS 左右。

5.2.2.13 示例 13: 使用自定义数据集进行 INT8 量化

igie-exec 支持用户提供自定义的 pkl 数据用于 INT8 量化校准，自定义数据要求后缀名为 .pkl。对于 .pkl 格式的输入，igie-exec 内部会通过 pickle 读取，格式要求如下：

- 单输入，比如 resnet@imagenet。如果我们准备 100 个数据，首先创建一个以模型输入名为 key，校准数据集为 value 的字典，然后将其保存为 pkl 格式的文件：

```
a = np.random.randn(100, 3, 224, 224).astype(np.float32)
data = {'input': a}

with open('calibration.pkl', 'wb') as pkl_file:
    pickle.dump(data, pkl_file)
```

- 多输入，假设有两个输入，比如也是 100 个数据，按照模型不同输入，保存对应的校准数据集即可：

```
a = np.random.randn(100, 3, 224, 224).astype(np.float32)
b = np.random.randn(100, 3, 100).astype(np.float32)

data = {'input_1': a, 'input_2': b}

with open('calibration.pkl', 'wb') as pkl_file:
    pickle.dump(data, pkl_file)
```

下面是 PyTorch ResNet50 使用 .pkl 量化的例子：

```
# 获取.pkl 格式的校准数据集

# 使用自定义数据进行 INT8 量化并生成引擎文件
# 对于除 ONNX 外的模型，这里可以指定 batchsize 为 1，用来加速量化系数表的生成
# 后续生成其他 batch 的引擎文件时，可以重复利用该量化系数表，以节省时间和内存
igie-exec --model_path data/models/resnet/resnet50-fp32.pt --input input:1,3,224,224
  --precision int8 --calibration_file_path calibration.pkl

# 使用已有的量化系数表，生成大 batch 的引擎文件，并在完整的数据集上测试
igie-exec --model_path data/models/resnet/resnet50-fp32.pt --input input:32,3,224,224
  --precision int8 --use_imagenet True

# 执行结果：
{'acc@1': 0.744538252240717,
 'acc@5': 0.9224951984635084,
 'acc_result': 0.744538252240717,
 'fps_result': 5753.188203730447}
```

5.3 igie-exec 测试工具的参数定义

您可通过 `./igie-exec -h` 命令获取全部参数信息，具体参数介绍如下：

必填参数

- `--model-path`：模型的路径，目前 igie-exec 支持导入 TensorFlow/PyTorch/ONNX/PaddlePaddle 四种框架的模型以及从 TorchVision 直接下载模型。
 - `--model_path resnet50.pb`：导入 TensorFlow 模型，以 *.pb 为后缀名。
 - `--model_path resnet50.pt`：导入 PyTorch 模型，以 *.pt 为后缀名。
 - `--model_path resnet50.onnx`：导入 ONNX 模型，以 *.onnx 为后缀名。
 - `--model_path resnet50/inference`：导入 PaddlePaddle 模型，注意 PaddlePaddle 以目录的方式给出，如 resnet50，内含名为 inference.pdiparams, inference.pdiparams.info 和 inference.pdmodel 的文件。
 - `--model_path resnet18`：导入 TorchVision 预训练模型，igie-exec 内置了 TorchVision 的模型，可通过指定 TorchVision 中的模型名字自动下载模型并生成引擎。

- `--input`: 模型的输入信息, 支持多输入, 包含 name、shape 和 dtype 三种信息, 格式为 name: shape/dtype。其中, name 和 shape 为必填, dtype 不填写默认为 float32, 常见的 dtype 包含 float32/int32/int64。例如:
 - 单输入: `--input input1:32,3,224,224`, 此时默认 dtype 为 float32。
 - 单输入指定 dtype: `--input input1:32,3,224,224/int32`
 - 多输入: `--input input1:32,3,224,224 input2:32,100`
 - 多输入指定 dtype: `--input input1:32,3,224,224/float32 input2:32,100/int64`
- `--precision`: 模型的推理精度, 根据该精度会选择对应的量化策略以及图优化选项。
 - 使用 `--precision fp16` 参数会自动将大部分的计算转换为 FP16。
 - 使用 `--precision int8` 参数会根据模型的原生框架以及是否提供校准数据集来进行对应的量化策略。
 - * 模型框架:
 - 如果是 ONNX 模型, 将自动采用 ONNXRuntime 量化。
 - 如果是 TensorFlow/PyTorch/PaddlePaddle 模型, 将采用 IGIE 进行量化。
 - * 校准数据:
 - 如果提供校准数据集 (参考后续 `--calibration_file_path` 的说明), 或是使用内置的 imagenet/coco2017 数据 (参考后续的 `--use_imagenet` 和 `--use_coco2017` 的说明), 将采用内置的校准数据集进行量化。
 - 如果没有指定数据集, 将根据输入的 input 信息生成随机数据进行量化和推理。

可选参数

- `--target`: 模型推理的目标设备, 默认值为 `--target iluvatar_with_all_libs`, 可选项以及对说明如下:
 - `--target llvm`: CPU 推理。
 - `--target iluvatar`: GPU 推理。
 - `--target iluvatar_with_cudnn_cublas`: GPU 推理, 且启用 cuBLAS/cudnn 加速库。
 - `--target iluvatar_with_all_libs`: GPU 推理, 且启用 cuBLAS/cudnn/ixInfer 加速库。ixInfer 是天数智芯加速卡底层的高性能算子库, 能大幅度地提高推理速度。
- `--engine_path`: 生成的引擎文件的保存路径, 后缀名必须以 .so 结尾, 默认值为 None, 此时则保存在当前工作路径。用例如下:


```
--engine_path resent50.so
```
- `--warmup`: 性能测试前的 warmup 次数, 默认值为 `--warmup 3`, 一般不需要特殊指定。
- `--verbose`: 是否保存编译期间的模型结构对应的 mod 信息, 默认为 `--verbose False`, 使用 `--verbose True` 可将编译期间的 mod 保存在当前目录的 module 目录中, 方便 debug。
- `--num_workers`: 加载数据集使用的线程数, 默认值为 `--num_workers 16`, 在 CPU 核数较少的情况下建议设置成 `--num_workers 1` 或 `--num_workers 4`。
- `--batch_size`: 一个 batch 的大小, 默认值为 None, 会自动从 `--input` 中传入的 shape 信息自动解析。但是对于一些特殊的多输入的情况, 如 `--input input_ids:1000,22 pixel_values:32,3,224,224 attention_mask:1000,22`, 实际一个 batch 的数据应该为 32, 此时可通过该 `--batch_size 32` 参数进行显式指定。

- `--use_imagenet`: 使用标准的 imagenet 的 val 数据集进行量化校准以及精度验证, 通过 `--use_imagenet True` 启用, 适用于 ResNet/VGG/ShuffleNet/MobileNet 等经典模型, 输入格式默认为 NCHW, 模型预处理按照 PyTorch 格式进行。
- `--use_coco2017`: 使用标准的 coco2018 的 val 数据集进行量化校准以及精度验证, 通过 `--use_coco2017 True` 启用, 适用于 YOLO 系列经典模型, 输入格式默认为 NCHW, 模型预处理按照 PyTorch 格式进行。
- `--input_layout`: 在使用 imagenet/coco2017 数据集时的输入数据格式, 默认值为 `--input_layoyut NCHW`, 可改变为 `--input_layout NHWC`。此时 imagnet 的输入预处理会采用 TensorFlow 格式。
- `--calibration_file_path`: 用户定义的量化校准数据集, 需要以 numpy 的格式提供, 具体可参考[示例 13: 使用自定义数据集进行 INT8 量化](#)。
- `--automatic_yolo_quantization`: 是否自动跳过 YOLO 系列 ONNX 模型的 detect 部分的节点的量化, 通过开启该选项可大幅提高 YOLO 系列模型的量化精度, 默认值为 `--automatic_yolo_quantization False`。
- `--quantization_config_path`: 精细配置 ONNXRuntime/IGIE 量化参数的 json 文件, 具体可参考[示例 13: 使用自定义数据集进行 INT8 量化](#)。
- `--acc_target`: 使用 imagenet/coco2017 数据集进行精度评价的目标值, 其中 imagenet 使用 `acc@1`, coco2017 使用 `map@.5` 作为目标。
- `--fps_target`: 使用 imagenet/coco2017 数据集进行性能评价的目标值, fps 的计算公式为 $(batch_size / batch_infer_time)$ 的平均值。
- `--perf_only`: 是否仅使用随机数据做性能测试, 默认值为 `--perf_only False`, 但是在未使用 imagenet/coco2017 的情况下, 该选项默认会被自动打开。
- `--just_export`: 是否仅生成引擎文件, 默认值为 `--just_export False`。
- `--custom_option`: 拓展选项, 使用类似 `--input`, 支持复数拓展选项, 格式为 `--custom_option key1:value1 key2:value2 key3:value3`。目前开放的拓展参数如下:
 - `with_nms`: YOLO 系列模型是否将 nms 也加入到生成的引擎中, 可通过 `with_nms:True` 的方式指定。
 - `iou_thres`: nms 中的 iou 阈值, 可通过 `iou_thres:0.65` 的方式指定。
 - `conf_thres`: nms 中的 conf 阈值, 可通过 `conf_thres:0.001` 的方式指定。
 - `topk`: nms 生成的最大的 box 数量, 可通过 `topk:100` 的方式指定。

6 从 TensorRT 迁移到 IGIE

IGIE 与 TensorRT 拥有一模一样的推理流程，推理代码都遵循 导入模型 -> 量化（可选）-> 自动图优化（编译）-> 导出或导入引擎文件 -> 设置模型输入 -> 推理模型等步骤。从使用 API 角度看，IGIE 的调用接口更加简洁。

下面以 ResNet50 模型为例，比较 TensorRT 和 IGIE 推理过程：

6.1 使用 TensorRT 推理 ResNet50 模型

TensorRT 推理模型的流程分为 导入模型 -> 量化（量化）-> 自动图优化（编译）-> 导出或导入引擎文件 -> 设置模型输入 -> 推理模型等步骤。

```
# 1) 导入 ONNX 模型到 TRT
network = trt.builder.create_network(batch_size)
parser = trt.OnnxParser(network)
with open(model_file, "rb") as model:
    parser.parse(model.read())

# 2) 模型量化，以 FP16 为例
config = trt.builder.create_builder_config()
config.set_flag(trt.BuilderFlag.FP16)

# 3) 自动优化模型，编译模型为引擎文件
config.max_workspace_size = 1 << 30 # 1GB
engine = builder.build_engine(network, config)

# 4) 导出或导入引擎文件
plan = trt.builder.build_serialized_network(network, config) # 导出引擎文件
with open("ResNet50.engine", "wb") as f:
    f.write(plan)
engine = trt.runtime.deserialize_cuda_engine(plan) # 导入引擎文件

context = engine.create_execution_context() # 设置 module context，加载引擎文件到 GPU device

# 5) 设置模型输入
inputs = []
outputs = []
bindings = []
stream = cuda.Stream()
for binding in engine:
    size = trt.volume(engine.get_binding_shape(binding)) * engine.max_batch_size
    dtype = trt.nptype(engine.get_binding_dtype(binding))
    # Allocate host and device buffers
    host_mem = cuda.pagelocked_empty(size, dtype)
```

```

device_mem = cuda.mem_alloc(host_mem.nbytes)
# Append the device buffer to device bindings.
bindings.append(int(device_mem))
# Append to the appropriate list.
if engine.binding_is_input(binding):
    inputs.append(HostDeviceMem(host_mem, device_mem))
else:
    outputs.append(HostDeviceMem(host_mem, device_mem))

cuda.memcpy_htod_async(inputs.device, inputs.host, stream)

# 6) 执行推理
context.execute_async_v2(bindings=bindings, stream_handle=stream.handle)

# 7) 获取 TRT 推理输出
cuda.memcpy_dtoh_async(outputs.host, outputs.device, stream)
stream.synchronize()
print(outputs.host)

```

6.2 使用 IGIE 推理 ResNet50 模型

使用 IGIE 推理模型依然遵循此流程：导入模型 -> 量化 -> 自动图优化（编译） -> 导出或导入引擎文件 -> 设置模型输入 -> 推理模型。

```

# 1) 导入 ONNX 模型到 IGIE
input_name = "input0"
shape_list = [(input_name, input_shape)]
mod, params = relay.frontend.from_pytorch(scripted_model, shape_list)

# 2) 设置模型使用 FP16 推理
precision = "fp16"

# 3) 自动优化模型，编译模型为 engine
print("Strat build engine...")
engine = relay.build(mod, target=target, params=params, precision=precision)

# 4) 导出或导入引擎文件
engine.export_library("resnet50.so") # 导出引擎文件
engine = tvm.runtime.load_module("resnet50.so") # 导入引擎文件

m = graph_executor.GraphModule(engine["default"](dev)) # 设置 model context, 加载引擎文件到
    GPU device

```

5) 设置模型输入

```
image = np.random.uniform(size=input_shape).astype("float32")  
image = tvm.nd.array(image, device)  
m.set_input(input_name, image)
```

6) 推理模型

```
m.run()
```

7) 输出推理结果

```
igie_output = m.get_output(0)  
print(igie_output)
```

6.3 总结

IGIE 与 TensorRT 推理对比总结：

IGIE 与 TensorRT 拥有一模一样的推理流程，推理代码都遵循导入模型 -> 量化 -> 自动图优化 (编译) -> 导出或导入引擎文件 -> 设置模型输入 -> 推理模型等步骤。从使用 API 角度看，IGIE 的调用接口更加简洁。

当然，在部署模型时，更常用的方法是直接拿编好的引擎文件进行部署推理，即下图的 4, 5, 6, 7 步骤，使得适配迁移 TensorRT 更加方便。

IGIE与TensorRT模型推理对比

	TensorRT模型推理	IGIE模型推理	
1	<pre># 1) 导入ONNX模型到TRT network = trt.builder.create_network(batch_size) parser = trt.OnnxParser(network) with open(model_file, "rb") as model: parser.parse(model.read())</pre>	<pre># 1) 导入ONNX模型到IGIE input_name = "input0" shape_list = [(input_name, input_shape)] mod, params = relay.frontend.from_pytorch(model, shape_list)</pre>	导入模型
2	<pre># 2) 模型量化, 以FP16为例 config = trt.builder.create_builder_config() config.set_flag(trt.BuilderFlag.FP16)</pre>	<pre># 2) 模型量化, 以FP16为例 from tvn.relay.transform import ToMixedPrecision mod = ToMixedPrecision(mixed_precision_type="float16")(mod)</pre>	量化模型
3	<pre># 3) 自动优化模型, 编译模型为engine config.max_workspace_size = 1 << 30 # 1GB engine = builder.build_engine(network, config)</pre>	<pre># 3) 自动优化模型, 编译模型为engine engine = relay.build(mod, target=target, params=params)</pre>	编译engine
4	<pre># 4) 导出 or 导入 engine文件 plan = trt.builder.build_serialized_network(network, config) with open("ResNet50.engine", "wb") as f: f.write(plan) engine = trt.runtime.deserialize_cuda_engine(plan) context = engine.create_execution_context()</pre>	<pre># 4) 导出 or 导入engine文件 engine.export_library("resnet50.so") engine = tvn.Runtime.load_module("resnet50.so") m = graph_executor.GraphModule(engine["default"])(dev)</pre>	导入&导出engine
5	<pre># 5) 设置模型输入 inputs = [] outputs = [] bindings = [] stream = cuda.Stream() for binding in engine: size = trt.volume(engine.get_binding_shape(binding)) * engine.max_batch_size dtype = trt.nptype(engine.get_binding_dtype(binding)) # Allocate host and device buffers host_mem = cuda.pagelocked_empty(size, dtype) device_mem = cuda.mem_alloc(host_mem.nbytes) # Append the device buffer to device bindings. bindings.append(int(device_mem)) # Append to the appropriate list. if engine.binding_is_input(binding): inputs.append(HostDeviceMem(host_mem, device_mem)) else: outputs.append(HostDeviceMem(host_mem, device_mem)) inputs, outputs, bindings, stream = allocate_buffers(engine) cuda.memcpy_htod_async(inputs.device, inputs.host, stream)</pre>	<pre># 5) 设置模型输入 image = np.random.uniform(size=input_shape) image = tvn.nd.array(image, device) m.set_input(input_name, image)</pre>	设置模型输入
6	<pre># 6) 执行推理 context.execute_async_v2(bindings=bindings, stream_handle=stream.handle)</pre>	<pre># 6) 执行推理 m.run()</pre>	执行推理
7	<pre># 7) 获取TRT推理输出 cuda.memcpy_dtoh_async(outputs.host, outputs.device, stream) stream.synchronize() print(outputs.host)</pre>	<pre># 7) 输出推理结果 igie_output = m.get_output(0) print(igie_output)</pre>	输出结果

图 4: IGIE 与 TensorRT 推理对比

7 IGIE 自动图优化介绍

IGIE 图优化模块能够自动地对导入模型进行优化，主要包括自动算子融合，冗余算子消除，常量折叠，内存重用，layout 转换，自动 padding 等，最终转换为符合天数智芯硬件特性，性能更优的 IR Module，用于最终推理。

7.1 算子融合

算子融合是推理引擎加速重要的手段，将零散的算子融合成一个算子后，有两大好处：

- 减少 kernel launch 的个数。
- 目前大多数模型的算子都是 memory bound，融合算子后会节省很多 kernel 结果写回 global memory 的时间，这在拥有大量小算子的模型上尤为明显。

下面列举 IGIE 中几个常用的算子融合：

Conv2d+BatchNorm+Bias 算子融合：

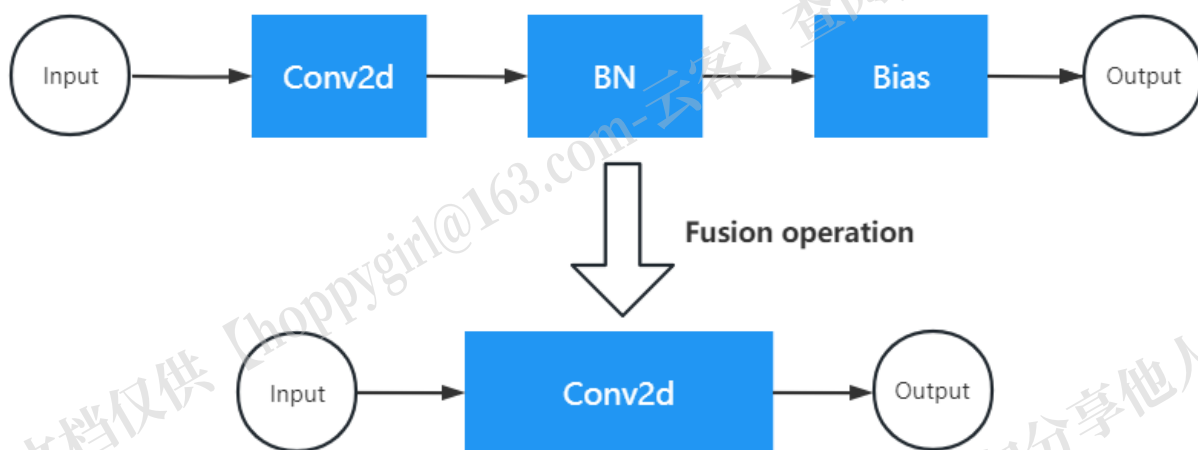


图 5: 算子融合

Conv2d+BatchNorm+Add+Relu 算子融合：

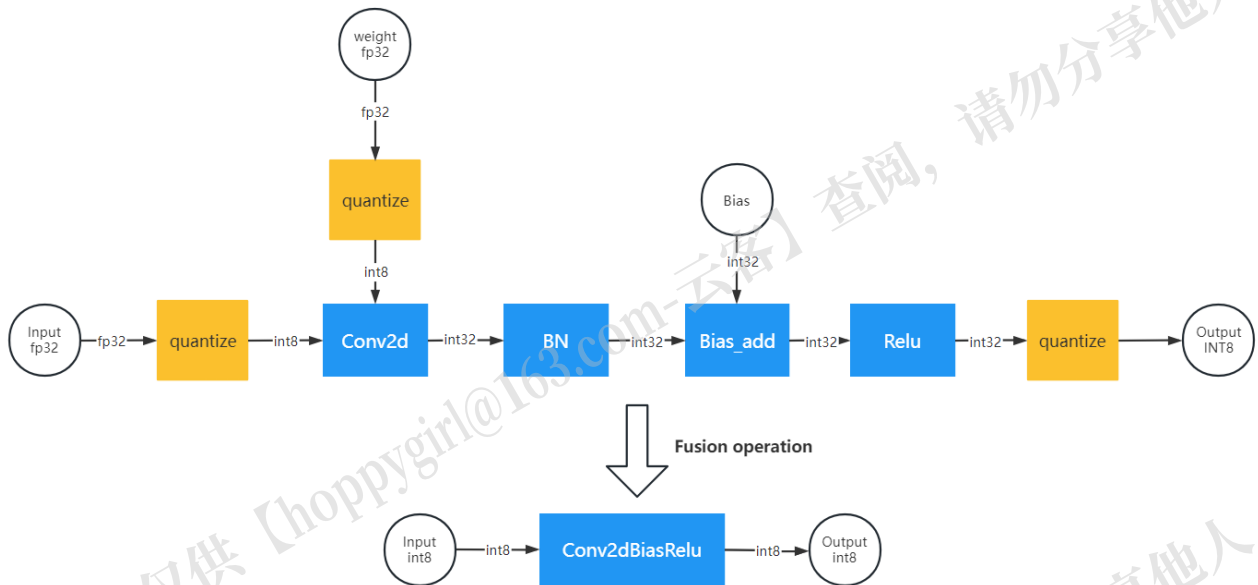


图 6: 算子融合

Conv2d+Sigmoid+multiply 算子融合:

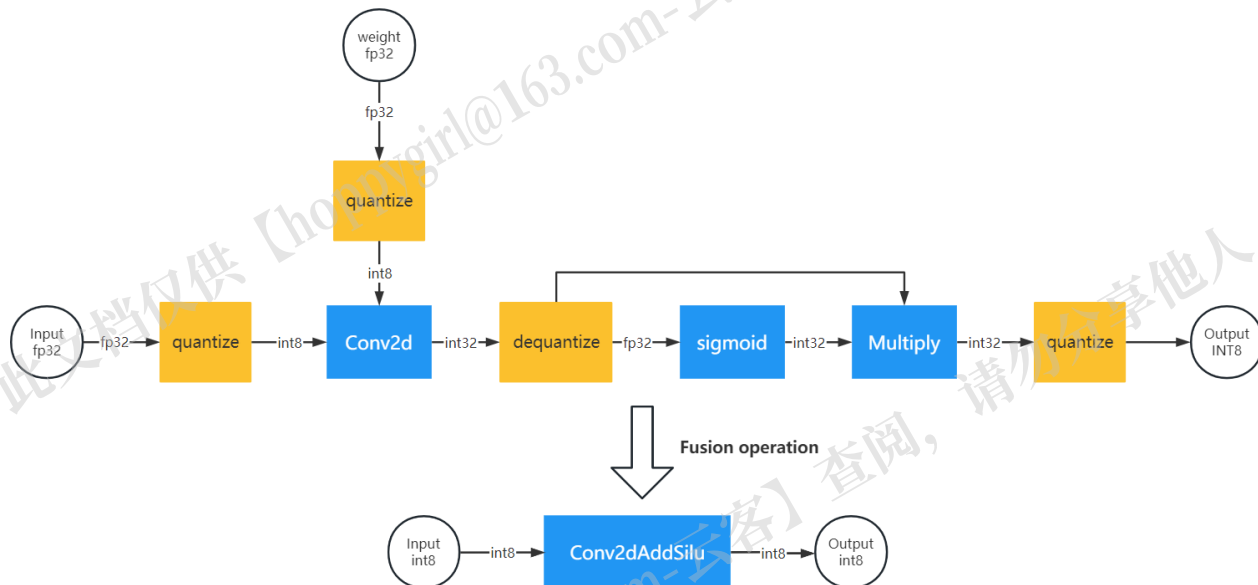


图 7: 算子融合

两个 Conv2d+Add+Relu 算子融合

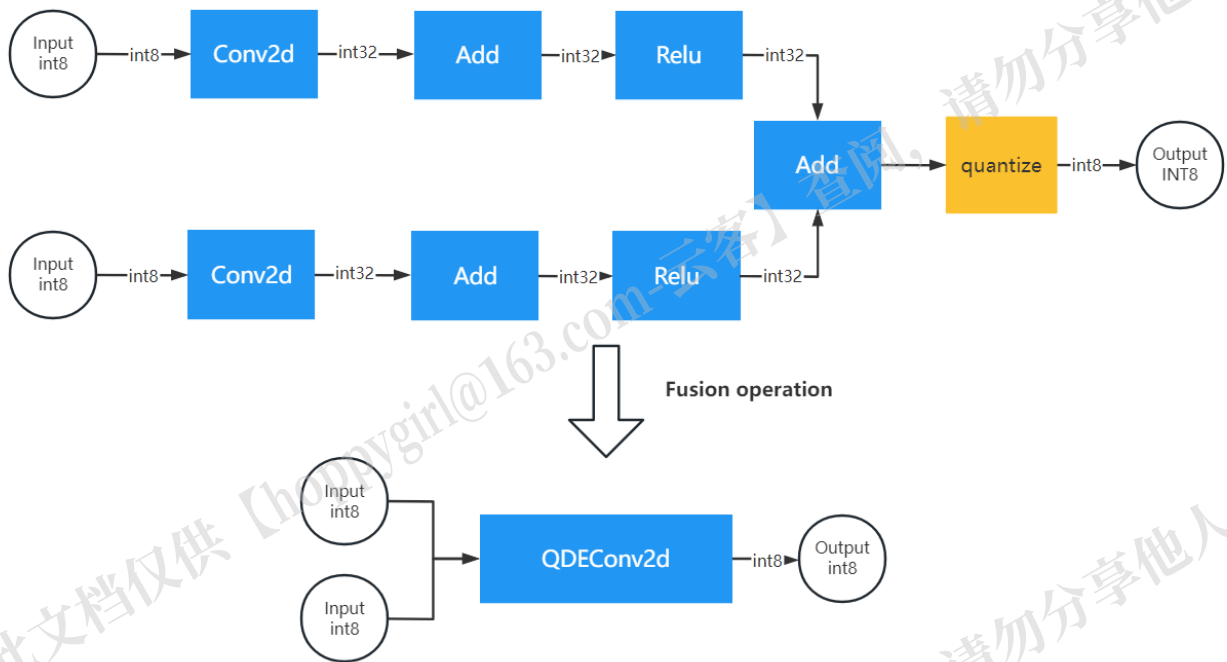


图 8: 算子融合

量化 Concat 算子融合

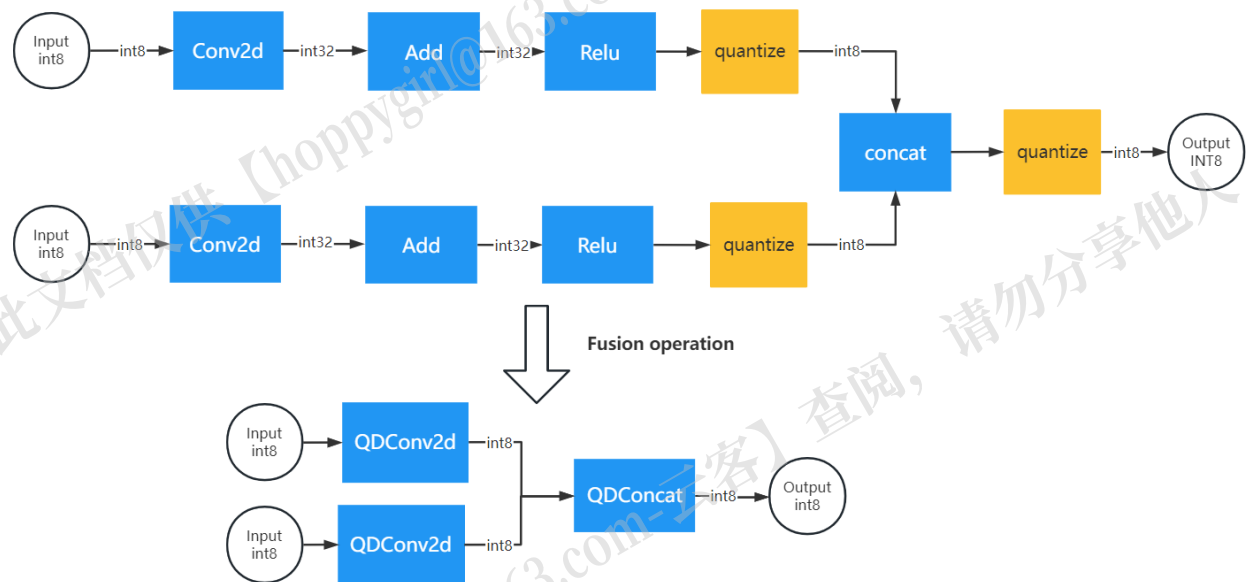


图 9: 算子融合

7.2 Layout 转换和自动 padding

IGIE 可以方便的对 Tensor 进行内存布局转换，以优化数据局部性，提升访存效率。也可以根据天数智芯加速卡中 TCU 的硬件特性进行自动 padding。

例如：

- 1) 模型 Layout 自动转换为算子库性能最佳方式 (NCHW <-> NHWC)。
- 2) 重新 Packing data，方便对接外部算子库的实现。(NCHW->NCHWC64)
- 3) 将整个模型的 conv2d channel 自动填充成适合天数 GPU 计算的大小。

7.3 内存复用

IGIE 通过对模型中部分算子的内存进行复用，以减少内存开销。

例如下图，颜色一样的算子内存会进行复用 (relu+relu+add)。

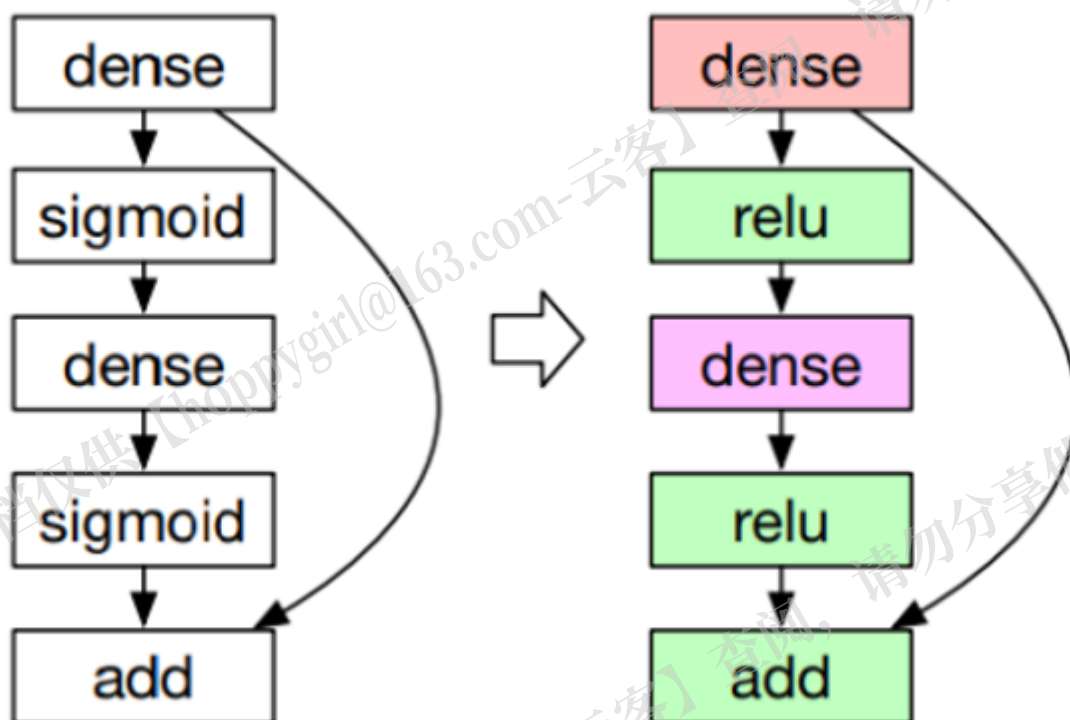


图 10: 内存复用

7.4 常量折叠和常量预计算

对于模型中的常量或者 weights 运算，IGIE 会在编译的时候，自动将常量提前计算好。

对于多个相关的常量，会进行折叠处理。

如下图，权重所有的运算操作，全部在模型运行前计算好。

Constant Parameter Path Pre-Computation (w is marked as a constant parameter)

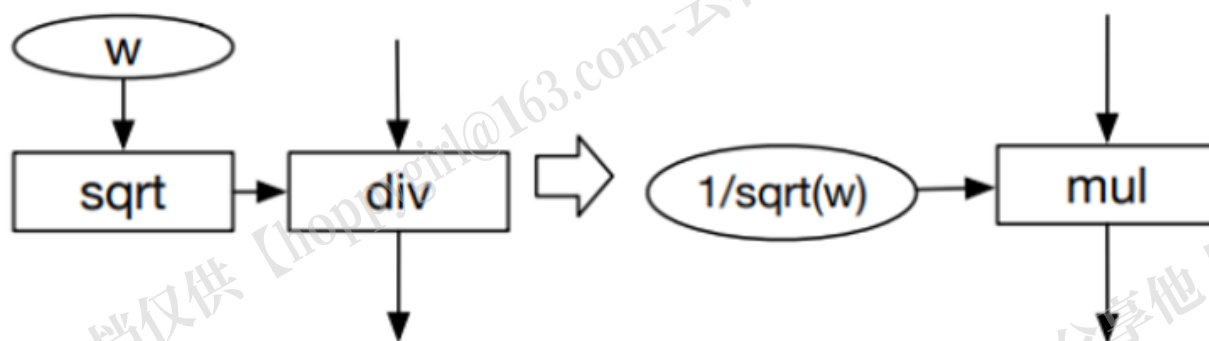


图 11: 常量计算

8 高级教程：IGIE 推理框架高级功能使用教程

本教程提供了一系列场景式的使用文档来指导用户使用 IGIE 的高级功能，帮助深度了解模型推理部署。

8.1 教程 0：如何使用 IGIE 给模型添加 NMS

NMS 是一种常用的后处理技术，用于消除目标检测等任务中冗余的检测结果，应用比较广泛。在本小节中，您将了解到如何使用 IGIE 为模型添加 NMS，并使用天数智芯加速卡完成 NMS 计算。

Torchvision 中有两种 NMS 的实现：

- NMS
- Batched_NMS

两者的主要区别在于对 bbox 进行过滤时，是否考虑类别信息。

NMS 不区分类别，对所有 bbox 进行过滤，如果有不同类别的 bbox 重叠，则会被过滤掉。定义如下：

Parameters:

- **boxes** (*Tensor*[*N*, 4]) – boxes to perform NMS on. They are expected to be in (x1, y1, x2, y2) format with $0 \leq x1 < x2$ and $0 \leq y1 < y2$.
- **scores** (*Tensor*[*N*]) – scores for each one of the boxes
- **iou_threshold** (*float*) – discards all overlapping boxes with $IoU > iou_threshold$

Returns:

int64 tensor with the indices of the elements that have been kept by NMS, sorted in decreasing order of scores

图 12: NMS 定义

Batched_NMS 需要区分类别，只针对同一种类别计算 IOU，对 bbox 进行过滤。定义如下：

Parameters:

- **boxes** (*Tensor*[*N*, 4]) – boxes where NMS will be performed. They are expected to be in (x1, y1, x2, y2) format with $0 \leq x1 < x2$ and $0 \leq y1 < y2$.
- **scores** (*Tensor*[*N*]) – scores for each one of the boxes
- **idxs** (*Tensor*[*N*]) – indices of the categories for each one of the boxes.
- **iou_threshold** (*float*) – discards all overlapping boxes with $IoU > iou_threshold$

Returns:

int64 tensor with the indices of the elements that have been kept by NMS, sorted in decreasing order of scores

图 13: Batched_NMS 定义

从应用场景来看，大部分检测类模型都使用 **Batched_NMS**，例如 YOLO 系列，但也有少部分模型需要用到 **NMS**，例如 RetinaFace。

IGIE 在 Relay 中注册了 vision.ixinfer_nms OP，通过调用 ixInfer API 来实现 **NMS** 和 **Batched_NMS** (下文无需区分实现处统称为 **NMS**) 功能。大多数情况下，模型的输出结果并不能直接作为该 OP 的输入，因此在将 **NMS** 添加到模型中时，首先需要对模型输出进行处理。

目前 igie-exec 已经针对适配过的模型添加了 AutoNMS 机制，可通过参数 **--custom_option with_nms:True** 开启。

开启时，igie-exec 会调用 ADD_NMS 来为模型添加 **NMS**：

```
from util.nms import ADD_NMS
mod = ADD_NMS(mod, iou_thres=iou_thres, conf_thres=conf_thres, topk=topk)
```

由于不同模型的输出结构不同，因此 AutoNMS 不能完全覆盖所有模型。当现有 AutoNMS 机制不满足需求时，需要参考 nms.py 手动添加 **NMS**。以下是 nms.py 的代码 (您也可以在 IGIE 安装目录的 tvn/igie-exec/util/ 目录下找到该文件)：

```
import tvn
from tvn import relay
from tvn.relay.expr_functor import ExprMutator
from tvn.relay.frontend.common import infer_shape

import numpy as np

visitor = ExprMutator()

def Batched_NMS(mod, iou_thres, conf_thres, topk, num_cls):
    # get org mod out
    mod_out = visitor.visit(mod['main'].body)
```

```
# get org mod shape
mod_shape = infer_shape(mod_out)

# If the model has multiple outputs
if isinstance(mod_shape, tvm.ir.type.TupleType):
    assert len(mod_out) == len(mod_shape.fields), "Output nums mustbe same as shape
    ↪ infer"

    out_infos = {}

    for idx, shape_info in enumerate(mod_shape.fields):
        if shape_info.shape[-1] == 4:
            key = "boxes"
        elif shape_info.shape[-1] == num_cls:
            key = "cls"
        else:
            key = "scores"

        out_infos[key] = {"data": mod_out[idx], "shape": np.array(shape_info.shape)}

    box_num = int(out_infos['boxes']['shape'][1])

    # get cls_id from cls output
    cls_ids = relay.op.argmax(out_infos['cls']['data'], axis=2, keepdims=True)
    cls_ids = relay.cast(cls_ids, dtype="float32")

    # get cls_scores from cls output
    cls_scores = relay.op.max(out_infos['cls']['data'], axis=2, keepdims=True)

    if "scores" in out_infos.keys():
        cls_scores = relay.multiply(out_infos['scores']['data'], cls_scores)

    if len(out_infos['boxes']['shape']) > 3:
        boxes = relay.squeeze(out_infos['boxes']['data'])
        out_infos['boxes']['shape'] = out_infos['boxes']['shape'].squeeze()
    else:
        boxes = out_infos['boxes']['data']

    assert len(out_infos['boxes']['shape'] == 3), "xywh2xyxy Only support 3 dimension."

    # box(center_x, center_y, w, h) to (x1, y1, x2, y2), not all models need to do
    boxes = relay.vision.xywh2xyxy(boxes)

    nms_input = relay.concatenate(relay.Tuple([boxes, cls_ids, cls_scores]), axis=2)

    # add NMS: [bs, num_boxes, 6] -> [N, topk, 6], [N, 1]
    nms_output = relay.vision.ixinfer_nms(nms_input,
                                          input_box_num=box_num,
                                          max_ouput_box_num=topk,
```

```

        iou_thres=iou_thres,
        conf_thres=conf_thres,
        class_agnostic=False)

nms_out = relay.Tuple([nms_output[0], nms_output[1]])

else:
    """
    The model output shape must follow:
    [bs, num_boxs, [x, y, w, h, score, num_cls]]

    eg:
    For COCO2017 the output shape might be: [bs, num_boxes, 85]
    """
    assert len(mod_shape) == 3, "AutoNMS add just support output shape len == 3"

    bs = mod_shape[0]
    box_num = mod_shape[1]

    # get boxes, scores, cls_scores
    boxes = relay.strided_slice(mod_out, begin=[0, 0, 0], end=[bs, box_num, 4])

    scores = relay.strided_slice(mod_out, begin=[0, 0, 4], end=[bs, box_num, 5])

    cls_scores = relay.strided_slice(mod_out, begin=[0, 0, 5], end=[bs, box_num,
↵ num_cls + 5])

    # get cls_ids and cls_scores
    cls_ids = relay.op.argmax(cls_scores, axis=2, keepdims=True)
    cls_ids = relay.cast(cls_ids, dtype="float32")

    cls_scores = relay.op.max(cls_scores, axis=2, keepdims=True)
    cls_scores = relay.multiply(scores, cls_scores)

    # box(center_x, center_y, w, h) to (x1, y1, x2, y2), not all models need to do
    boxes = relay.vision.xywh2xyxy(boxes)

    nms_input = relay.concatenate(relay.Tuple([boxes, cls_ids, cls_scores]), axis=2)

    # add NMS: [bs, num_boxes, 6] -> [N, topk, 6], [N, 1], [N,topk]
    nms_output = relay.vision.ixinfer_nms(nms_input,
        input_box_num=box_num,
        max_output_box_num=topk,
        iou_thres=iou_thres,
        conf_thres=conf_thres,
        class_agnostic=False)

    nms_out = relay.Tuple([nms_output[0], nms_output[1]])

nms_func = relay.Function(relay.analysis.free_vars(nms_out), nms_out)

```

```

nms_mod = tvm.ir.IRModule.from_expr(nms_func)

return nms_mod

def NMS(mod, iou_thres, conf_thres, topk):
    # get org mod out
    mod_out = visitor.visit(mod['main'].body)

    # get org mod shape
    mod_shape = infer_shape(mod_out)

    # If the model has multiple outputs
    if isinstance(mod_shape, tvm.ir.type.TupleType):
        assert len(mod_out) == len(mod_shape.fields), "Output nums mustbe same as shape
        ↪ infer"

        out_infos = {}

        for idx, shape_info in enumerate(mod_shape.fields):
            if shape_info.shape[-1] == 4:
                key = "boxes"
                # just for retinaface
            elif shape_info.shape[-1] == 2:
                key = "scores"
            else:
                key = "saved_input"

            out_infos[key] = {"data": mod_out[idx], "shape": np.array(shape_info.shape)}

            n = int(out_infos['boxes']['shape'][0])
            box_num = int(out_infos['boxes']['shape'][1])

            scales = relay.strided_slice(out_infos['scores']['data'], begin=[0, 0, 1], end=[n,
            ↪ box_num, 2])

            nms_input = relay.concatenate(relay.Tuple([out_infos['boxes']['data'], scales]),
            ↪ axis=2)

            # add NMS: [bs, num_boxes, 5] -> [N, topk, 5], [N, 1], [N,topk]
            nms_output = relay.vision.ixinfer_nms(nms_input,
                input_box_num=box_num,
                max_ouput_box_num=topk,
                iou_thres=iou_thres,
                conf_thres=conf_thres,
                class_agnostic=True)

            nms_out = relay.Tuple([nms_output[0], nms_output[1], nms_output[2],
            ↪ out_infos['saved_input']['data']])

            nms_func = relay.Function(relay.analysis.free_vars(nms_out), nms_out)

```

```

nms_mod = tvm.ir.IRModule.from_expr(nms_func)

return nms_mod

def ADD_NMS(mod, iou_thres=0.65, conf_thres=0.01, topk=100, num_cls=80,
    ↪ class_agnostic=False):
    if class_agnostic:
        mod = NMS(mod, iou_thres, conf_thres, topk)
    else:
        mod = Batched_NMS(mod, iou_thres, conf_thres, topk, num_cls)

    return mod

```

NMS 的添加方法可以归纳为：利用 Relay 对模型输出进行处理，使其符合 vision.ixinfer_nms OP 定义的输入要求，然后添加 vision.ixinfer_nms OP，实现 **NMS** 计算。

下面以 RetinaFace 和 YOLOv5 为例，分别介绍 **NMS** 和 **Batched_NMS** 的添加方法。

8.1.1 Vision ixinfer_nms OP 定义

首先，需要了解 relay.vision.ixinfer_nms OP 的定义：

```

def ixinfer_nms(bboxes, input_box_num, max_output_box_num, iou_thres, conf_thres,
    ↪ class_agnostic):
    """
    NMS
    bboxes : relay.Expr
        3-D tensor with shape (batch_size, input_box_num, 6), channel 6 is
    ↪ [x1,y1,x2,y2,class_id,score] or
        3-D tensor with shape (batch_size, input_box_num, 5), channel 5 is
    ↪ [x1,y1,x2,y2,score]
    input_box_num : int
        input box numbers == bboxes.shape[1]
    max_output_box_num : int
        output box numbers == output.shape[1]
    iou_thres : float
        iou_thres
    conf_thres : float
        conf_thres
    class_agnostic: bool
        if class_agnostic set NMS else set Batched_NMS

    return
    output : relay.Expr
        boxes: 3-D tensor with shape (batch_size, max_output_box_num, 6)
        boxes_num: 1-D tensor with shape (batch_size)
        boxes_index: 2-D tensor with shape (batch_size, box_idx)
    """

```

```

out = _make.ixinfer_nms(boxes, input_box_num, max_output_box_num, iou_thres,
    ↪ conf_thres, class_agnostic)

return expr.TupleWrapper(out, 3)

```

参数说明：

- class_agnostic: 控制 OP 行为
 - 设置为 True, 表示执行 **NMS**
 - 设置为 False, 表示执行 **Batched_NMS**
- boxes: OP 输入, **NMS** 和 **Batched_NMS** 所需输入不同, 区别在于是否包含类别信息
 - 执行 **NMS**, 输入为 (batch_size, input_box_num, 5)
 - 执行 **Batched_NMS**, 输入为 (batch_size, input_box_num, 6)
- input_box_num: 输入 box 的数目
- max_output_box_num: 输出 box 的数目
- iou_thres: 交并比阈值
- conf_thres: 置信度阈值

8.1.2 为 RetinaFace 添加 NMS

从 RetainFace 模型输出来看, 包括三部分:

- loc[bs, 16800, 4]
- conf[bs, 16800, 2]
- landms[bs, 16800, 10]

后处理对 [loc, conf[:, :, 1]] 进行 concat 后, 执行 **NMS**, 返回 bbox 和 bbod index, 根据 index 信息从 landms 中获取人脸关键点, 参考 [detect.py](#)。

上述逻辑可以使用 Relay 进行实现 (完整代码, 请参考 IGIE 安装目录下的 tvn/igie-exec/util/nms.py: 115 line):

1. 获取模型输出及输出 shape:

```

visitor = ExprMutator()

# get org mod out
mod_out = visitor.visit(mod['main'].body)

# get org mod shape
mod_shape = infer_shape(mod_out)

```

2. 多输出情况下, 获取模型输出信息:

```

# If the model has multiple outputs
if isinstance(mod_shape, tvn.ir.type.TupleType):
    assert len(mod_out) == len(mod_shape.fields), "Output nums mustbe same as shape
    ↪ infer"

```

```

out_infos = {}

for idx, shape_info in enumerate(mod_shape.fields):
    if shape_info.shape[-1] == 4:
        key = "boxes"
        # just for retinaface
    elif shape_info.shape[-1] == 2:
        key = "scores"
    else:
        key = "saved_input"

    out_infos[key] = {"data": mod_out[idx], "shape": np.array(shape_info.shape)}

```

3. 使用 Relay 对 conf 进行切片 (conf[:, :, 1]), 在此之前需要获取 shape 信息:

```

n = int(out_infos['boxes']['shape'][0])
box_num = int(out_infos['boxes']['shape'][1])

scales = relay.strided_slice(out_infos['scores']['data'], begin=[0, 0, 1], end=[n,
↪ box_num, 2])

```

4. 使用 Relay 对 loc 和 conf 进行 concat:

```

nms_input = relay.concatenate(relay.Tuple([out_infos['boxes']['data'], scales]),
↪ axis=2)

```

5. 添加 NMS (class_agnostic=True):

```

# add NMS: [bs, num_boxes, 5] -> [N, topk, 5], [N, 1], [N, topk]
nms_output = relay.vision.ixinfer_nms(nms_input,
    input_box_num=box_num,
    max_output_box_num=topk,
    iou_thres=iou_thres,
    conf_thres=conf_thres,
    class_agnostic=True)

```

6. 重写模型的输出。

这里保存了 NMS 的三个输出和原始模型 landms 的输出, 用于精度验证:

```

nms_out = relay.Tuple([nms_output[0], nms_output[1], nms_output[2],
↪ out_infos['saved_input']['data']])

```

7. 获取修改后的模型:

```

nms_func = relay.Function(relay.analysis.free_vars(nms_out), nms_out)
nms_mod = tvn.ir.IRModule.from_expr(nms_func)

```

8.1.3 为 YOLOv5 添加 Batched_NMS

目前 IGIE 适配过的 YOLO 系列模型的输出分以下两种情况，以 coco 数据集 80 类为例：

- concat 输出情况下，模型的输出 shape 一般为 [bs, num_boxes, 85]
- 不 concat 输出的情况下，模型一般有三个输出，分别为：
 - boxes: [bs, num_boxes, 4]
 - scores: [bs, num_boxes, 1]
 - class: [bs, num_boxes, 80]

igie-exec 已经考虑上述可能出现的情况，并进行了分类处理。

以 concat 输出的情况为例，介绍如何使用 Relay 为模型添加 **Batched_NMS**。整体的添加流程与 **NMS** 区别不大，多出了对输出的 slice 操作和对 class 的 max 及 argmax 操作，详情如下 (完整代码，请参考 IGIE 安装目录下的 `tvm/igie-exec/util/nms.py:10 line`)：

1. 获取模型输出及输出 shape：

```
visitor = ExprMutator()

# get org mod out
mod_out = visitor.visit(mod['main'].body)

# get org mod shape
mod_shape = infer_shape(mod_out)
```

2. 对输出做 stride_slice，分别获取 boxes, scores, class：

```
# get boxes, scores, cls_scores
boxes = relay.strided_slice(mod_out, begin=[0, 0, 0], end=[bs, box_num, 4])

scores = relay.strided_slice(mod_out, begin=[0, 0, 4], end=[bs, box_num, 5])

cls_scores = relay.strided_slice(mod_out, begin=[0, 0, 5], end=[bs, box_num, num_cls
↪ + 5])
```

3. 获取 cls_id 和 cls_scores。

cls_scores 为每个 box class 最大值与 scores 输出的乘积。

注：部分模型不存在 scores 输出，此时无需做 multiply。

```
# get cls_ids and cls_scores
cls_ids = relay.op.argmax(cls_scores, axis=2, keepdims=True)
cls_ids = relay.cast(cls_ids, dtype="float32")

cls_scores = relay.op.max(cls_scores, axis=2, keepdims=True)
cls_scores = relay.multiply(scores, cls_scores)
```

4. 对 bbox 进行转换。

将 bbox 格式从 [x, y, w, h] 转换为 [x1, y1, x2, y2]。该步骤并非所有模型都需要执行，后处理有此需求时可以将其添加到模型中。

```
boxes = relay.vision.xywh2xyxy(boxes)
```

5. 使用 Relay 执行 concat, 获取 vision.ixinfer_nms OP 输入:

```
nms_input = relay.concatenate(relay.Tuple([boxes, cls_ids, cls_scores]), axis=2)
```

6. 添加 Batched_NMS(class_agnostic=False):

```
# add NMS: [bs, num_boxes, 6] -> [N, topk, 6], [N, 1], [N,topk]
nms_output = relay.vision.ixinfer_nms(nms_input,
                                     input_box_num=box_num,
                                     max_output_box_num=topk,
                                     iou_thres=iou_thres,
                                     conf_thres=conf_thres,
                                     class_agnostic=False)
```

7. 重写模型的输出。

NMS 返回结果有三个, 这里只需取其中两个即可满足后续的精度验证需求:

```
nms_out = relay.Tuple([nms_output[0], nms_output[1]])
```

8. 获取修改后的模型:

```
nms_func = relay.Function(relay.analysis.free_vars(nms_out), nms_out)
nms_mod = tvm.ir.IRModule.from_expr(nms_func)
```

8.2 教程 1: 如何使用 Relax

本小节在“快速上手: IGIE 推理框架使用教程”>“教程 8: 快速上手 Relax 进行模型推理”的基础上, 进一步细化 Relax 的相关使用。

8.2.1 如何使用 TVMScript

TVMScript 是一种应用于 TVM 软件栈的领域专用语言, 可用于构造 IRModule。在 IGIE v0.18.0 中, Unity 对 TVMScript 模块进行代码重构, 支持 Python 语法形式的 round-trippable TVM IR, 即您可以将 TVMScript 源码转化为 IRModule, 也可以将 IRModule 以 TVMScript 的形式显示出来, 而且都是 Python 语法完全兼容的形式。以下内容来自 TVM 官方网站:

IRModule can be created by writing TVMScript, which is a round-trippable syntax for TVM IR.

Different than creating a computational expression by Tensor Expression ([Working with Operators Using Tensor Expression](#)), TensorIR allow users to program through TVMScript, a language embedded in python AST. The new method makes it possible to write complex programs and further schedule and optimize it.

首先, 理解 TVMScript 模块的设计初衷, 它为使用者/开发者提供了一种可以快速构建、查看和修改 IRModule 的 DSL, 一般应用于 IRModule 的调试和快速验证等。前端模型的导入转化, 或实际模型的搭建, 很少使用 TVMScript (后文将提到 Relax Frontend 的实现, 其本质上是通过 `relax.BlockBuilder` 来完成的)。

在 .../IGIE/tests/python/tvmscript 路径中有大量 tvmscript 的使用示例，这里将重点提供一个 end2end 的 tvmscript 形式的 IRModule 创建、修改和推理的完整过程。

A Simple end2end TVMScript-Style IRModule

下面代码使用 TVMScript 搭建了一个由 2 个 Tensor 相加的 IRModule，并在使用 Relax 编译后添加到 Relax 虚拟机中进行推理。

代码中，定义了一组优化 Pass，用于调度 OP 在天数智芯加速卡上的计算实现，其中用到了 DefaultGPUSchedule() 接口，这将在下文中讨论。

```
import tvm
from tvm.script import ir as I, relax as R, tir as T
from tvm.relax.transform import LegalizeOps
from tvm.tir.transform import DefaultGPUSchedule, ForceNarrowIndexToInt32
from tvm import relax
import tvm.testing
import numpy as np

target = tvm.target.Target("iluvatar -mcpu=ivcore11")
device = tvm.device("iluvatar", 0)

seq = tvm.transform.Sequential([
    LegalizeOps(),
    DefaultGPUSchedule(),
    ForceNarrowIndexToInt32(),
])

@I.ir_module
class Module:
    @R.function
    def main(x: R.Tensor((3, 4), "float32"), y: R.Tensor((3, 4), "float32")):
        z = R.add(x, y)
        # z = R.subtract(x, y)
        # z = R.multiply(x, y)
        # z = R.divide(x, y)
        return z

mod = Module

with target:
    mod = seq(mod)

mod.show()

ex = relax.build(mod, target=target)
vm = relax.VirtualMachine(ex, device)

x = tvm.nd.array(np.random.random([3,4]).astype(np.float32), device)
y = tvm.nd.array(np.random.random([3,4]).astype(np.float32), device)
res = vm["main"](x, y)
print("\nx:\n", x)
```

```
print("\ny:\n", y)
print("\nx?y:\n", res)
```

上面代码中，`mod.show()` 会打印 TVMScript 形式的 IRModule 形式，即：

```
# from tvm.script import ir as I
# from tvm.script import tir as T
# from tvm.script import relax as R

@I.ir_module
class Module:
    @T.prim_func(private=True)
    def add(x: T.Buffer((3, 4), "float32"), y: T.Buffer((3, 4), "float32"), T_add:
        ↪ T.Buffer((3, 4), "float32")):
        T.func_attr({"tir.is_scheduled": T.bool(True), "tir.noalias": T.bool(True)})
        # with T.block("root"):
        for ax0_ax1_fused_0 in T.thread_binding(1, thread="blockIdx.x"):
            for ax0_ax1_fused_1 in T.thread_binding(12, thread="threadIdx.x"):
                with T.block("T_add"):
                    v_ax0 = T.axis.spatial(3, (ax0_ax1_fused_0 * 12 + ax0_ax1_fused_1) // 4)
                    v_ax1 = T.axis.spatial(4, (ax0_ax1_fused_0 * 12 + ax0_ax1_fused_1) % 4)
                    T.reads(x[v_ax0, v_ax1], y[v_ax0, v_ax1])
                    T.writes(T_add[v_ax0, v_ax1])
                    T_add[v_ax0, v_ax1] = x[v_ax0, v_ax1] + y[v_ax0, v_ax1]

    @R.function
    def main(x: R.Tensor((3, 4), dtype="float32"), y: R.Tensor((3, 4), dtype="float32"))
        ↪ -> R.Tensor((3, 4), dtype="float32"):
        cls = Module
        z = R.call_tir(cls.add, (x, y), out_sinfo=R.Tensor((3, 4), dtype="float32"))
        return z
```

这是在天数智芯加速卡上调度后的结果，可以看到存在 thread binding 的操作。

Parse a TVMScript-Style IRModule

从前文可知，TVMScript 提供了 Python 语法形式的 round-trippable TVM IR，对于上面的示例，您也可以使用 TVMScript 输出形式来构造 IRModule。

下面代码同样实现了与 2 个 shape 为 (3, 4) @float32 的 Tensor 相加的 IRModule。

```
import tvm
from tvm.script import ir as I, relax as R, tir as T
from tvm import relax
import tvm.testing
import numpy as np

target = tvm.target.Target("iluvatar -mcpu=ivcore11")
device = tvm.device("iluvatar", 0)
```

```
@I.ir_module
class Module:
    @T.prim_func(private=True)
    def add(x: T.Buffer((3, 4), "float32"), y: T.Buffer((3, 4), "float32"), T_add:
        ↪ T.Buffer((3, 4), "float32")):
        T.func_attr({"tir.is_scheduled": T.bool(True), "tir.noalias": T.bool(True)})
        # with T.block("root"):
        for ax0_ax1_fused_0 in T.thread_binding(1, thread="blockIdx.x"):
            for ax0_ax1_fused_1 in T.thread_binding(12, thread="threadIdx.x"):
                with T.block("T_add"):
                    v_ax0 = T.axis.spatial(3, (ax0_ax1_fused_0 * 12 + ax0_ax1_fused_1) // 4)
                    v_ax1 = T.axis.spatial(4, (ax0_ax1_fused_0 * 12 + ax0_ax1_fused_1) % 4)
                    T.reads(x[v_ax0, v_ax1], y[v_ax0, v_ax1])
                    T.writes(T_add[v_ax0, v_ax1])
                    T_add[v_ax0, v_ax1] = x[v_ax0, v_ax1] + y[v_ax0, v_ax1]

    @R.function
    def main(x: R.Tensor((3, 4), dtype="float32"), y: R.Tensor((3, 4), dtype="float32"))
        ↪ -> R.Tensor((3, 4), dtype="float32"):
        cls = Module
        z = R.call_tir(cls.add, (x, y), out_sinfo=R.Tensor((3, 4), dtype="float32"))
        return z

mod = Module

mod.show()

ex = relax.build(mod, target=target)
vm = relax.VirtualMachine(ex, device)

x = tvn.nd.array(np.random.random([3,4]).astype(np.float32), device)
y = tvn.nd.array(np.random.random([3,4]).astype(np.float32), device)
res = vm["main"](x, y)
print("\nx:\n", x)
print("\ny:\n", y)
print("\nx?y:\n", res)
```

可以看到，代码中使用 Python 关键字 class 定义了一个 Module 类，然后使用 mod = Module 来创建一个 IIRModule，然后进行构建和推理。

上面 mod.show() 打印出的 IIRModule，和代码中 IIRModule 是一致的：

```
# from tvn.script import ir as I
# from tvn.script import tir as T
# from tvn.script import relax as R

@I.ir_module
class Module:
    @T.prim_func(private=True)
    def add(x: T.Buffer((3, 4), "float32"), y: T.Buffer((3, 4), "float32"), T_add:
        ↪ T.Buffer((3, 4), "float32")):
```

```
T.func_attr({"tir.is_scheduled": T.bool(True), "tir.noalias": T.bool(True)})
# with T.block("root"):
for ax0_ax1_fused_0 in T.thread_binding(1, thread="blockIdx.x"):
    for ax0_ax1_fused_1 in T.thread_binding(12, thread="threadIdx.x"):
        with T.block("T_add"):
            v_ax0 = T.axis.spatial(3, (ax0_ax1_fused_0 * 12 + ax0_ax1_fused_1) // 4)
            v_ax1 = T.axis.spatial(4, (ax0_ax1_fused_0 * 12 + ax0_ax1_fused_1) % 4)
            T.reads(x[v_ax0, v_ax1], y[v_ax0, v_ax1])
            T.writes(T_add[v_ax0, v_ax1])
            T_add[v_ax0, v_ax1] = x[v_ax0, v_ax1] + y[v_ax0, v_ax1]

@R.function
def main(x: R.Tensor((3, 4), dtype="float32"), y: R.Tensor((3, 4), dtype="float32"))
    ↪ -> R.Tensor((3, 4), dtype="float32"):
    cls = Module
    z = R.call_tir(cls.add, (x, y), out_sinfo=R.Tensor((3, 4), dtype="float32"))
    return z
```

上面的 2 个示例只是简单介绍 TVMScript 语法的使用方法，更多供参考的示例可在 .../IGIE/tests/python/tvmscript 路径下找到。

8.2.2 如何构建 Relax 模型

TVM 社区为了提高 Relay 的表达能力、性能及移植性，提出了另一套 IR，即 Relax，解释为 Relay Next。您可以访问 [Relax: Co-Designing High-Level Abstraction Towards TVM Unity - Development / pre-RFC - Apache TVM Discuss](#) 了解更多信息。

目前有如下 4 种构建 Relax IRModule (暂不算 Relax 前端导入的方式，该方式使用的是 blockbuilder 方式)：

- Relax.BlockBuilder
- TVMScript
- Relax.Function
- Relax.nn

在不同的场景下推荐您使用不同的方式。

8.2.2.1 Relax.BlockBuilder

Relax.BlockBuilder 提供了一套接口，可以方便地构造 Relax 函数且条理清楚。一个 Relax Function 中要有 block、形参、body、返回值等，通过 with 关键字可灵活地配置 Relax 函数，所见即所得，是一种高层次上创建 Relax Function 的语法。通过 relax.emit() 接口可以调用 Relax 层的 ops，您可暂且不用关心 relax.op 的实现，有助于在看懂 Relax 函数的整体逻辑，虽然最终 relax.emit() 都会 lower() 到 Relax.BlockBuilder；Relax 还提供了 call_te() / emit_te() 接口，可以调用 TE 层的 op。

要知道，Relax 的前端都是用 relax.BlockBuilder() 搭建的，可见其地位比 tvmscript 高。

Note

注意，R.call_tir() 是 tvmscript() 中的接口，并不是 blockbuilder() 中的接口。

下面提供一些示例加以说明。

Build Relax IR with Single Block

准确来说，block 是控制流图 (Control-flow graph, CFG) 中的概念，CFG 在计算科学和 LLVM 中是很重要的概念，用于实现循环及分支跳转等。在 TVM 和 LLVM 中，一个 Function 内部由一个个 CFG 构成，在每个 CFG 中，才是具体的计算逻辑。

您可以访问 [Control-flow graph - Wikipedia](https://en.cppreference.com/w/cpp/algorithm/control_flow_graph) 获取更多关于 CFG 的信息。

下面代码使用 relax.BlockBuilder 构造了 2 个 Tensor 相加的 Relax IRModule。

```
import tvm
from tvm import te, tir, topi, relax

n = tir.Var("n", "int64")
x = relax.Var("x", relax.TensorStructInfo([n], "float16"))
y = relax.Var("y", relax.TensorStructInfo([n], "float16"))
bb = relax.BlockBuilder()

with bb.function("main", [x, y]):
    with bb.dataflow():
        lv0 = bb.emit(relax.op.add(x, y))
        gv0 = bb.emit_output(lv0)
    bb.emit_func_output(gv0)

mod = bb.get()
func = mod["main"]
print(func)
```

其 func 的 TVMScript 形式为：

```
@R.function
def main(x: R.Tensor(("n",), dtype="float16"), y: R.Tensor(("n",), dtype="float16")) ->
    R.Tensor(("n",), dtype="float16"):
    n = T.int64()
    with R.dataflow():
        lv: R.Tensor((n,), dtype="float16") = R.add(x, y)
        gv: R.Tensor((n,), dtype="float16") = lv
        R.output(gv)
    return gv
```

Build Relax IR with Multi Block

实际 IR 中，您可能会使用多个 block 去隔离不同的程序分支，比如 if-else，for-loop 分支的 CFG 实现中，就会存在多个 block。

下面代码仅为了说明多个 block 的构造过程：

```
import tvm
from tvm import te, tir, topi, relax
```

```
n = tir.Var("n", "int64")
x = relax.Var("x", relax.TensorStructInfo([n], "float16"))
y = relax.Var("y", relax.TensorStructInfo([n], "float16"))
bb = relax.BlockBuilder()

with bb.function("main", [x, y]):
    with bb.dataflow():
        lv0 = bb.emit(relax.op.add(x, y))
        assert lv0.name_hint == "lv"
        gv0 = bb.emit_output(lv0)
        gv1 = bb.emit(relax.op.add(x, gv0))
    with bb.dataflow():
        lv1 = bb.emit(relax.op.add(gv1, y))
        gv2 = bb.emit_output(lv1)
    bb.emit_func_output(gv2)

mod = bb.get()
func = mod["main"]
print(func)
```

其 func 的 TVMScript 形式为:

```
@R.function
def main(x: R.Tensor(("n",), dtype="float16"), y: R.Tensor(("n",), dtype="float16")) ->
    R.Tensor(("n",), dtype="float16"):
    n = T.int64()
    with R.dataflow():
        lv: R.Tensor((n,), dtype="float16") = R.add(x, y)
        gv: R.Tensor((n,), dtype="float16") = lv
        R.output(gv)
    gv1: R.Tensor((n,), dtype="float16") = R.add(x, gv)
    with R.dataflow():
        lv1: R.Tensor((n,), dtype="float16") = R.add(gv1, y)
        gv2: R.Tensor((n,), dtype="float16") = lv1
        R.output(gv2)
    return gv2
```

Build Relax IR with Multi Function

在实际使用中会存在一个 Relax IRModule 中有多个 Relax Function 的情形，也是可以通过 relax.BlockBuilder 来构建的，举例如下：

```
import tvm
from tvm import te, tir, topi, relax

n = tir.Var("n", "int64")
x = relax.Var("x", relax.TensorStructInfo([n], "float16"))
y = relax.Var("y", relax.TensorStructInfo([n], "float16"))
bb = relax.BlockBuilder()
```

```
with bb.function("func1", [x, y]):
    with bb.dataflow():
        lv0 = bb.emit(relax.op.add(x, y))
        assert lv0.name_hint == "lv"
        gv0 = bb.emit_output(lv0)
        bb.emit_func_output(gv0)

with bb.function("func2", [x, y]):
    with bb.dataflow():
        lv0 = bb.emit(relax.op.add(x, y))
        assert lv0.name_hint == "lv1"
        gv0 = bb.emit_output(lv0)
        bb.emit_func_output(gv0)

mod = bb.get()
func1 = mod["func1"]
assert func1.params[0] == x
assert func1.params[1] == y
assert len(func1.body.blocks) == 1

mod.show()
```

mod 的 TVMScript 形式为：

```
@I.ir_module
class Module:
    @R.function
    def func1(x: R.Tensor(("n",), dtype="float16"), y: R.Tensor(("n",), dtype="float16"))
        ↪ -> R.Tensor(("n",), dtype="float16"):
            n = T.int64()
            with R.dataflow():
                lv: R.Tensor((n,), dtype="float16") = R.add(x, y)
                gv: R.Tensor((n,), dtype="float16") = lv
                R.output(gv)
            return gv

    @R.function
    def func2(x: R.Tensor(("n",), dtype="float16"), y: R.Tensor(("n",), dtype="float16"))
        ↪ -> R.Tensor(("n",), dtype="float16"):
            n = T.int64()
            with R.dataflow():
                lv1: R.Tensor((n,), dtype="float16") = R.add(x, y)
                gv1: R.Tensor((n,), dtype="float16") = lv1
                R.output(gv1)
            return gv1
```

8.2.2.2 TVMScript

在上文中，已经讨论过 TVMScript 的使用和特点，使用方法在此将不再举例。

这里仅讨论其使用局限性，TVMScript 虽然可以详细呈现 Relax Function 和 TIR Function 的计算细节，但却容易丢失上层的接口抽象 (因为都是 PrimFunc 形式的具体展开)。以 NMS 算子为例，topi 层的 `non_max_suppression()` 算子是通过调用多个 Internal 算子实现的：

```
def non_max_suppression(...):
    data_buf = tvn.tir.decl_buffer(data.shape, data.dtype, "data_buf", data_alignment=8)

    sort_tensor = _get_sorted_indices(data, data_buf, score_index, (data.shape[0],
    ↪ data.shape[1]))

    out_bboxes, out_scores, out_class_ids, out_features, box_indices, num_valid_boxes =
    ↪ _run_nms(
    ...)

    if return_indices:
        return [box_indices, num_valid_boxes]

    return _concatenate_outputs(...)
```

但它们转化成 TIR Function 后将非常复杂，丢失了可读性。以下代码展示部分展开后的形式：

```
@T.prim_func(private=True)
def all_class_non_max_suppression(var_boxes: T.handle, scores: T.Buffer((T.int64(1),
    ↪ T.int64(2), T.int64(6000)), "float32"), C: T.Buffer((), "int64"), D: T.Buffer((),
    ↪ "float32"), E: T.Buffer((), "float32"), var_collect_indices: T.handle, compute_red:
    ↪ T.Buffer((T.int64(1),), "int64")):
    T.func_attr({"tir.noalias": T.bool(True)})
    boxes = T.match_buffer(var_boxes, (T.int64(1), T.int64(6000), T.int64(4)),
    ↪ offset_factor=1)
    collect_indices = T.match_buffer(var_collect_indices, (T.int64(12000), T.int64(3)),
    ↪ "int64", offset_factor=1)
    # with T.block("root"):
    T_reshape = T.alloc_buffer((T.int64(2), T.int64(6000)))
    out_buf = T.alloc_buffer((T.int64(2), T.int64(6000)), "int32", align=8)
    T_gather = T.alloc_buffer((T.int64(2), T.int64(6000)))
    searchsorted = T.alloc_buffer((T.int64(2),), "int32", offset_factor=1)
    all_class_nms = T.alloc_buffer((T.int64(2), T.int64(6000)), "int32", offset_factor=1)
    all_class_nms_1 = T.alloc_buffer((T.int64(1), T.int64(2)), "int32", offset_factor=1)
    out_buf_1 = T.alloc_buffer((T.int64(2),), "int64")
    compute = T.alloc_buffer((T.int64(1), T.int64(2)), "int64")
    for ax0, ax1 in T.grid(T.int64(2), T.int64(6000)):
        with T.block("T_reshape"):
            v_ax0, v_ax1 = T.axis.remap("SS", [ax0, ax1])
            T.reads(scores[T.int64(0), (v_ax1 // T.int64(6000) + v_ax0) % T.int64(2),
            ↪ v_ax1 % T.int64(6000)])
            T.writes(T_reshape[v_ax0, v_ax1])
```

```

        T_reshape[v_ax0, v_ax1] = scores[T.int64(0), (v_ax1 // T.int64(6000) + v_ax0)
↪ % T.int64(2), v_ax1 % T.int64(6000)]
        with T.block("argsort_cpu"):
            T.reads()
            T.writes()
            T.call_packed("tvm.contrib.sort.argsort", T.tvm_stack_make_array(T_reshape.data,
↪ T.tvm_stack_make_shape(T.int64(2), T.int64(6000)), 0, 2, T.float32(0), T.int64(0)),
↪ T.tvm_stack_make_array(out_buf.data, T.tvm_stack_make_shape(T.int64(2),
↪ T.int64(6000)), 0, 2, 0, T.int64(0)), 1, T.bool(False))
        for ax0, ax1 in T.grid(T.int64(2), T.int64(6000)):
            with T.block("T_gather"):
                v_ax0, v_ax1 = T.axis.remap("SS", [ax0, ax1])
                T.reads(T_reshape[v_ax0, out_buf[v_ax0, v_ax1]], out_buf[v_ax0, v_ax1])
                T.writes(T_gather[v_ax0, v_ax1])
                T_gather[v_ax0, v_ax1] = T_reshape[v_ax0, out_buf[v_ax0, v_ax1]]
        ...

```

8.2.2.3 Relax.Function

Relax.Function 接口也可以用于构建 Relax IR, 下面示例代码参考了 `.../IGIE/tests/python/relax/test_expr.py`:

```

import tvm
from tvm import relax as rx
from tvm import tir
from tvm.script import relax as R

x = rx.Var("foo", R.Tensor(dtype="float32", ndim=2))
bindings = [rx.VarBinding(x, rx.const(1))]
blocks = [rx.BindingBlock(bindings)]

seque = rx.SeqExpr(blocks, x)
ret_struct_info = R.Tensor(dtype="float32", ndim=-1)
func = rx.Function([x], seque, ret_struct_info)
func = func.with_attr("global_symbol", "func")
assert func.params[0] == x
assert func.body == seque
assert func.ret_struct_info == ret_struct_info
assert func.attrs["global_symbol"] == "func"
func.show()

```

其中 func 的 TVMScript 形式为:

```

@R.function
def func(foo: R.Tensor(dtype="float32", ndim=2)) -> R.Tensor(dtype="float32"):
    foo: R.Tensor(dtype="float32", ndim=2) = R.const(1, "int32")
    return foo

```

8.2.2.4 Relax.nn

Relax 提供了一套 PyTorch-like API,方便您使用 PyTorch 风格来搭建网络模型,实现于 `.../relax/testing/nn.py` 中。有几个重要的概念/类,在 `nn.py` 中实现了 `Placeholder`、`Parameter` 以及 `Module` 类,与 PyTorch 中的概念对等。其中 `Placeholder` 和 `Parameter` 类都继承 `relax.Var` 类, `Module` 类提供 `forward()` 方法。

`relax.nn` 的所有使用方法和测试代码详见 `.../IGIE/tests/python/relaxtest_testing_nn.py`。

下面这个示例准确来说是 3 种构图方式的杂糅,以 `relax.blockbuilder` 为主线,使用 `nn.Module` 封装了 `Layer`,同时与 TVMScript 风格的 IR 进行对比。

```
import tvm
from tvm import te, tir, topi, relax
from tvm.relax.testing import nn
from tvm.script import ir as I, relax as R, tir as T

class Relu(nn.Module):
    def forward(self, input: relax.Expr) -> relax.Var:
        return nn.emit(relax.op.nn.relu(input))

@I.ir_module
class Expected:
    @R.function
    def main(x: R.Tensor((32, 32), dtype="float32")) -> R.Tensor((32, 32),
        dtype="float32"):
        gv: R.Tensor((32, 32), dtype="float32") = R.nn.relu(x)
        return gv

bb = relax.BlockBuilder()
with bb.function("main"):
    model = Relu()
    x = nn.Placeholder((32, 32), dtype="float32", name="x")
    output = model(x)
    params = [x] + model.parameters()
    bb.emit_func_output(output, params)

tvm.ir.assert_structural_equal(bb.get(), Expected)
mod = bb.get()
mod.show()
```

mod 的 TVMScript 形式为:

```
@I.ir_module
class Module:
    @R.function
    def main(x: R.Tensor((32, 32), dtype="float32")) -> R.Tensor((32, 32),
        dtype="float32"):
        gv: R.Tensor((32, 32), dtype="float32") = R.nn.relu(x)
        return gv
```

8.2.3 如何优化 Relax 模型

给定一个 Relax 模型，在部署前用户往往会使用各种优化方法，目的是为了获得更好的模型性能。模型优化至少分两个层面，一个是图级别的优化，常见的优化方法为图融合 (graph-fusion)；另一个是算子层面的优化，其核心是调整算子内部 AST 的实现，从而获得在目标硬件上更好的性能。

下面从这两个层面讨论具体的优化方法 (由于下文所讨论的优化方法在 IGIE 中存在大量使用示例，因此这里不再单独提供具体代码，仅给出相关路径)。

Graph Optimization

计算图优化的内容，建议参考 MLC 第七章课程 [7. 计算图优化 --- 机器学习编译 0.0.1 documentation \(mlc.ai\)](#)。这一章展示了如何把 linear+relu 的子图融合成 linear_relu 节点。

Kernel Optimization

虽然加速算子主要是借助 GPU (这部分可以命名为 GPU Acceleration)，但并不是所有算子都适合或存在 GPU 实现。特定情况下，CPU 实现也是存在的。因此本小节的内容指的是通常意义上的算子优化实现。

对 Relax 架构来说，可用的算子调优方法大致有三种，即 DefaultGPUSchedule、dlight 和 MetaSchedule。这三种方法各有优劣，可以单独使用，也可以“三合一”混合使用，视具体使用场景而定。从要不要搜索 (是否基于模板) 来划分，前两个属于基于模板的调优方法，而后者属于基于搜索的调优方法。

DefaultGPUSchedule

DefaultGPUSchedule 是针对 GPU 设备的模板级调优方法，根据预设的硬件参数，自动给出 TIR 函数的 GPU 加速版本。虽然该方法不需要搜索，直接给出 GPU 默认调度，但对于计算密集型算子 (如 GEMM)，其性能一般。

相关社区讨论可参考 [\[Unity\] Schedule Needed while building relax model with GPU Target - Questions - Apache TVM Discuss](#)

DefaultGPUSchedule 的使用示例可参考 [tvm/tests/python/tir-transform/test_transform_default_gpu_schedule.py at main · apache/tvm \(github.com\)](#)

dlight

dlight 库同样也是针对 GPU 硬件的调优模块，目前已经实现了 gemv、matmul、reduction 等重要算子的调优模板，还在不断扩展中。dlight 库将会取代 DefaultGPUSchedule，不需要搜索的同时，获得不错的性能。

相关社区讨论可参考 [\[Dlight\] Enabling Fast and Efficient Kernel Generation by Hardware information - Development - Apache TVM Discuss](#)

dlight 的使用示例可参考 [tvm/tests/python/dlight at main · apache/tvm \(github.com\)](#)

MetaSchedule

区别于基于 manually-set schedule template 的搜索的 dlight，MetaSchedule 是基于搜索空间的调优，所以理论上性能会好于前者，只是需要搜索时间的代价。好在 Relax 可以把搜索结果以 TVMScript 的形式保存下来，且从上文得知，可以利用 TVMScript 的 round-trippable 特性，加载上次调优后的 TIR 实现，即只要搜索一次即可。

MLC 课程上同样给出了对 MetaSchedule 非常详细的介绍，内容包括从一般意义上的随即变换搜索，到整个 MetaSchedule 设计。相关内容可参考 [4. 自动程序优化 --- 机器学习编译 0.0.1 documentation \(mlc.ai\)](#)

MetaSchedule 的使用示例可参考 [tvm/tests/python/relax/test_transform_meta_schedule_tuning.py at main · apache/tvm \(github.com\)](#)

8.2.4 如何使用 Relax 前端

目前 Relax 前端所支持的框架类型比 Relay 少一些，但常见的 ONNX 和 PyTorch 都是支持的。由于 Relax 前端在设计时极大地考虑了动态 Shape 的问题，因此可以导入更多动态输入的模型，体现了 Relax 前端的灵活性。如果您存在 Relax 前端类型无法满足其业务需求的问题，仍可以使用 Relay 前端，而且 Unity 提供了 `from_relay()` 接口，可以将导入的 Relay 模型转化为 Relax 模型。

下面就 Relax 前端常见的使用方法进行举例说明。

ONNX Frontend

如下示例说明如何使用 Relax ONNX 前端导入一个 ONNX 模型：

```
import onnx
from onnx import helper, numpy_helper
import numpy as np
import tvn
from tvn import relax, tir
from tvn.relax.frontend.onnx import from_onnx
from tvn.relax.transform import LegalizeOps

add_node_0 = helper.make_node("Add", ["input", "add_0_constant_tensor"],
    ↪ outputs=["add_node_0_output"], name="Add_0")
mul_node_0 = helper.make_node("Mul", ["add_node_0_output", "mul_0_constant_tensor"],
    ↪ outputs=["mul_node_0_output"], name="Mul_0")
add_node_1 = helper.make_node("Add", ["mul_node_0_output", "add_1_constant_tensor"],
    ↪ outputs=["output"], name="Add_1")

shape = [3, 224, 224]
input_value_info = helper.make_tensor_value_info("input", onnx.TensorProto.FLOAT,
    ↪ shape=shape)
output_value_info = helper.make_tensor_value_info("output", onnx.TensorProto.FLOAT,
    ↪ shape=shape)

add_0_constant_tensor = numpy_helper.from_array(np.random.randint(low=1, high=10, size=
    ↪ shape).astype(np.float32),
    ↪ name="add_0_constant_tensor")
mul_0_constant_tensor = numpy_helper.from_array(np.random.randint(low=1, high=10, size=
    ↪ shape).astype(np.float32),
    ↪ name="mul_0_constant_tensor")
add_1_constant_tensor = numpy_helper.from_array(np.random.randint(low=1, high=10, size=
    ↪ shape).astype(np.float32),
    ↪ name="add_1_constant_tensor")

graph = helper.make_graph(nodes=[add_node_0, mul_node_0, add_node_1], name="test_model",
    inputs=[input_value_info], outputs=[output_value_info],
    initializer=[add_0_constant_tensor, mul_0_constant_tensor,
    ↪ add_1_constant_tensor])

onnx_mod = helper.make_model(graph, opset_imports = [onnx.helper.make_opsetid("ai.onnx",
    ↪ 13)])
```

```
shape_dict = {"input": [3, 224, 224]}
relax_mod = from_onnx(onnx_mod, shape_dict)
# relax_mod = LegalizeOps()(relax_mod)
relax_mod.show()
```

导入后的 Relax 模型为：

```
@I.ir_module
class Module:
    @R.function
    def main(input: R.Tensor((3, 224, 224), dtype="float32")) -> R.Tensor((3, 224, 224),
        dtype="float32"):
        R.func_attr({"num_input": 1})
        with R.dataflow():
            lv: R.Tensor((3, 224, 224), dtype="float32") = R.add(input,
            metadata["relax.expr.Constant"][0])
            lv1: R.Tensor((3, 224, 224), dtype="float32") = R.multiply(lv,
            metadata["relax.expr.Constant"][1])
            gv: R.Tensor((3, 224, 224), dtype="float32") = R.add(lv1,
            metadata["relax.expr.Constant"][2])
            R.output(gv)
        return gv
```

From Relay to Relax

在 `relax.testing.relay_translator.py` 中提供了 `from_relay()` 接口，可以将 Relay 模型转化为 Relax 模型。这个接口实际上构成了 Relay 和 Relax 模块间的纽带，您可以借助 Relay 前端导入其它框架的模型，并沿用 Relay 的优化 Pass，在必要的时候再转化为 Relax 模型，然后在 Relax 层面继续进行模型的优化。

还是在上面代码的基础上，演示先使用 Relay ONNX 前端导入模型，再转化为 Relax 模型：

```
import onnx
from onnx import helper, numpy_helper
import numpy as np
import tvn
from tvn import relax, relay
from tvn.relay import testing
from tvn.relay.testing import relay_translator
from tvn.relay.transform import LegalizeOps
from tvn.tir.transform import DefaultGPUSchedule
from tvn.runtime import vm as vm_rt

add_node_0 = helper.make_node("Add", ["input", "add_0_constant_tensor"],
    outputs=["add_node_0_output"], name="Add_0")
mul_node_0 = helper.make_node("Mul", ["add_node_0_output", "mul_0_constant_tensor"],
    outputs=["mul_node_0_output"], name="Mul_0")
add_node_1 = helper.make_node("Add", ["mul_node_0_output", "add_1_constant_tensor"],
    outputs=["output"], name="Add_1")
```

```

shape = [3, 224, 224]
input_value_info = helper.make_tensor_value_info("input", onnx.TensorProto.FLOAT,
    ↪ shape=shape)
output_value_info = helper.make_tensor_value_info("output", onnx.TensorProto.FLOAT,
    ↪ shape=shape)

add_0_constant_tensor = numpy_helper.from_array(np.random.randint(low=1,high=10,size=
    ↪ shape).astype(np.float32),
    ↪ name="add_0_constant_tensor")
mul_0_constant_tensor = numpy_helper.from_array(np.random.randint(low=1,high=10,size=
    ↪ shape).astype(np.float32),
    ↪ name="mul_0_constant_tensor")
add_1_constant_tensor = numpy_helper.from_array(np.random.randint(low=1,high=10,size=
    ↪ shape).astype(np.float32),
    ↪ name="add_1_constant_tensor")

graph = helper.make_graph(nodes=[add_node_0, mul_node_0, add_node_1], name="test_model",
    inputs=[input_value_info], outputs=[output_value_info],
    initializer=[add_0_constant_tensor, mul_0_constant_tensor,
    ↪ add_1_constant_tensor])

onnx_mod = helper.make_model(graph, opset_imports = [onnx.helper.make_opsetid("ai.onnx",
    ↪ 13)])

target = tvm.target.iluvatar(model="MR")
device = tvm.device("iluvatar", 0)

shape_dict = {"input": [3, 224, 224]}
relay_mod, params = relay.frontend.from_onnx(onnx_mod, freeze_params=True)
relax_mod = relay_translator.from_relay(relay_mod["main"], target)

exe = relay.vm.compile(relay_mod, target)
relay_vm = vm_rt.VirtualMachine(exe, device)

with target:
    relax_mod = LegalizeOps()(relax_mod)
    relax_mod = DefaultGPUSchedule()(relax_mod)

ex = relax.build(relax_mod, target)
vm = relax.VirtualMachine(ex, device)

input_nd = tvm.nd.array(np.random.uniform(0, 1, size=[3, 224,224]).astype("float32"),
    ↪ device)
expected_output = relay_vm.run(input_nd)

res = vm["main"](input_nd)

tvm.testing.assert_allclose(res.numpy(), expected_output.numpy(), rtol=1e-4, atol=1e-4)
print("Passed!")

```

8.2.5 关于动态 Shape

关于动态 Shape 问题，至少可以分为以下两大类：

类型 1 模型的输出 Shape 完全可以由输入 Shape 推导出来

即使模型的输出不是确定的数值，而是以符号来表示的，输出 Shape 总可以表示为包含输入 Shape 的表达式。受限于 TVM 软件栈的版本，Infer Shape 可能做得不够彻底，即有些算子的 Shape 理论上是可以表示出来的，但实际上去推导会出现报错。这类问题可以随着软件栈的不断完善得到缓解或修复。

类型 2 数据驱动的动态 Shape 问题

有些算子 (如 where, nozero, squeeze, argmax, nms 等) 的输出 Shape，只有在实际推理计算时才能确定下来，从而导致动态 Shape 问题。包含此类算子的模型将会引入动态 Shape 问题。

下面给出一个 BERT-Base 的 NLP 示例，用于演示类型 1 的动态 Shape 模型推理。

```
import onnx
import tvm
from tvm import relax, tir
from tvm.relax.frontend.onnx import from_onnx
from tvm.relax.transform.iluvatar import optimize_with_target
import numpy as np

target = tvm.target.iluvatar(model="MR", options="-libs=cublas,ixinfer")
device = tvm.device("iluvatar", 0)

# wget http://10.113.3.3/data/Model/bert-base-uncased-squad-v1.onnx
onnx_mod = onnx.load('bert-base-uncased-squad-v1.onnx')

relax_mod = from_onnx(onnx_mod)
relax_mod = optimize_with_target(relax_mod, target=target, precision="float16",
    ↪ policy="dgs")

ex = relax.build(relax_mod, target=target)
vm = relax.VirtualMachine(ex, device=device)

batch = 16
seq = 256

input_ids = tvm.nd.array(np.zeros((batch, seq), "int64"), device)
attention_mask = tvm.nd.array(np.zeros((batch, seq), "int64"), device)
token_type_ids = tvm.nd.array(np.zeros((batch, seq), "int64"), device)
vm.set_input("main",
    tvm.nd.array(input_ids, device),
    tvm.nd.array(attention_mask, device),
    tvm.nd.array(token_type_ids, device))

# run
vm.invoke_stateful("main")
```

```
outputs = vm.get_outputs("main")
start_logits = outputs[0].asnumpy()
end_logits = outputs[1].asnumpy()

print(start_logits.shape)
print(end_logits.shape)
```

输出结果为：

```
(16, 256) (16, 256)
```

9 IGIE C++ API 说明

主要介绍 IGIE 模型部署相关 C++ API，包括：

- Target 相关类和函数
- Device 类和获取设备信息相关函数
- 内存操作和 Stream 设置相关函数
- 构造 Tensor 相关类和函数
- 加载模型引擎相关类和函数
- 模型输入输出信息相关函数
- 模型运行相关函数
- 模型耗时统计
- 推理代码参考示例

9.1 引入必要的头文件

```
#include <dlpack/dlpack.h>
#include <tvm/runtime/device_api.h>
#include <tvm/runtime/module.h>
#include <tvm/runtime/packed_func.h>
#include <tvm/runtime/registry.h>
#include <tvm/runtime/ndarray.h>
#include <tvm/runtime/profiling.h>
```

9.2 Target 相关类

9.2.1 Target()

功能概述

构造函数，根据传入 string 名称创建 Target。

函数原型

```
tvm::Target::Target(const String &target_str)
```

参数说明

target_str: 传入 target name 字符串，用于构造 Target。

返回值

返回创建的 Target。

使用示例

```
auto target = Target("iluvatar")
```

9.2.2 Target::Current()

功能概述

获得当前上下文的 Target。

函数原型

```
static tvm::Target tvm::Target::Current(bool allow_not_defined=true)
```

参数说明

- allow_not_defined: 如果该参数设置为 true, 允许返回 undefined Target。否则, 当 context stack 为 empty 时, 会导致运行时错误。

返回值

返回当前上下文的 Target。

9.3 Device 相关类

设备 Device 相关的类和函数。主要包括:

- Device 类的构造
- DeviceAPI 获取设备信息, 该接口是 cudaDeviceGetAttribute 的包装, 可以获取以下设备信息:
 - kWarpSize: Warp size 的数量。
 - kComputeVersion: CUDA 计算版本。
 - kDeviceName: 设备名称。
 - kMaxClockRate: 最大时钟频率。
 - kMultiProcessorCount: CU 个数。
 - kMaxThreadsPerBlock: GPGPU 设备中每个 Block 可用的最大线程数。
 - kMaxSharedMemoryPerBlock: GPGPU 设备中每个 Block 可用的最大 shape memory 大小。
 - kMaxRegistersPerBlock: 每个 Block 可用最大寄存器的数量。
 - kMaxThreadDimensions: Block 能设置的最大 X、Y、Z 维度。

9.3.1 DLDevice 结构体

功能概述

创建 DLDevice, 用于指定 op 和 tensor 的运行设备。

函数原型

```
typedef struct {
    DLDeviceType device_type;
    int device_id;
} DLDevice;

DLDevice dev{kDLILUVATAR, 0};
```

参数说明

- DLDeviceType: 指定设备类型, 天数智芯加速卡设备的宏为: kDLILUVATAR; CPU 设备的宏为: kDLCPU。
- device_id: 指定第几个设备, 例如一台机器上有 8 卡 GPU, device_id 可以指定为 0~7。

返回值

返回 DLDevice 结构体。

9.3.2 DeviceAPI 类

功能概述

DeviceAPI 是一个获取 device 信息和对 device 进行操作的基础类, 通过使用 DeviceAPI 类的成员函数对 Device 进行操作。您需要使用 device 构建 DeviceAPI 对象。

函数原型

```
static DeviceAPI* DeviceAPI::Get(const DLDevice& dev)
```

参数说明

DLDevice& dev: 通过 DLDevice 创建的 device 结构体。

返回值

返回 DeviceAPI 类对象。

使用示例

```
DLDevice dev{kDLILUVATAR, 0};
DeviceAPI* api = DeviceAPI::Get(dev);
```

9.3.3 DeviceAPI::GetAttr()

功能概述

获取 Device 设备属性信息, 该接口是对 cudaDeviceGetAttribute 的包装, 可以通过 DeviceAttrKind 获取以下设备信息:

- kWarpSize: Warp size 的数量。
- kComputeVersion: CUDA 计算版本。

- kDeviceName: 设备名称。
- kMaxClockRate: 最大时钟频率。
- kMultiProcessorCount: CU 个数。
- kMaxThreadsPerBlock: GPGPU 设备中每个 Block 可用的最大线程数。
- kMaxSharedMemoryPerBlock: GPGPU 设备中每个 Block 可用的最大 shape memory 大小。
- kMaxRegistersPerBlock: 每个 Block 可用最大寄存器的数量。
- kMaxThreadDimensions: Block 能设置的最大 X、Y、Z 维度。

函数原型

```
virtual void GetAttr(Device dev, DeviceAttrKind kind, TVMRetValue* rv)
```

参数说明

- Device& dev: 通过 DLDevice 创建的 device 结构体。
- DeviceAttrKind kind: 指定获取的设备属性 Kind, DeviceAttrKind 结构体定义如下:

```
enum DeviceAttrKind : int {
    kExist = 0,
    kMaxThreadsPerBlock = 1,
    kWarpSize = 2,
    kMaxSharedMemoryPerBlock = 3,
    kComputeVersion = 4,
    kDeviceName = 5,
    kMaxClockRate = 6,
    kMultiProcessorCount = 7,
    kMaxThreadDimensions = 8,
    kMaxRegistersPerBlock = 9,
    kGcnArch = 10,
    kApiVersion = 11,
    kDriverVersion = 12
};
```

- TVMRetValue* rv: GetAttr 函数的返回值会存放在 rv 中。

返回值

无返回值。

使用示例

```
DLDevice dev{kDLILUVATAR, 0};
DeviceAPI* api = DeviceAPI::Get(dev);
kind = tvm::runtime::DeviceAttrKind::kWarpSize;
tvm::runtime::TVMRetValue ret;
api->GetAttr(dev, kind, &ret);
int warp_size = ret;
```

9.4 Device 内存、流相关操作和设置

9.4.1 SetDevice()

功能概述

设置模型的运行设备 Device id，底层调用为 cudaSetDevice 函数。

函数原型

```
virtual void DeviceAPI::SetDevice(Device dev)
```

参数说明

Device& dev: 通过 DLDevice 创建的 device 结构体。

返回值

无返回值。

9.4.2 CreateStream()

功能概述

创建一个新的执行 stream。

函数原型

```
virtual TVMStreamHandle DeviceAPI::CreateStream(Device dev)
```

参数说明

Device& dev: 通过 DLDevice 创建的 device 结构体。

返回值

返回一个 TVMStreamHandle 对象。

使用示例

```
DLDevice dev{kDLILUVATAR, 0};  
DeviceAPI* api = DeviceAPI::Get(dev);  
TVMStreamHandle stream = api->CreateStream(dev);
```

9.4.3 SetStream()

功能概述

设置模型推理到指定 stream。

函数原型

```
virtual void tvm::runtime::DeviceAPI::SetStream(Device dev, TVMStreamHandle stream)
```

参数说明

- Device& dev: 通过 DLDevice 创建的 device 结构体。
- TVMStreamHandle stream: 通过 CreateStream() 创建的 Stream 对象。

返回值

无返回值。

使用示例

```
DLDevice dev{kDLILUVATAR, 0};
DeviceAPI* api = DeviceAPI::Get(dev);
TVMStreamHandle stream = api->CreateStream(dev);
api->SetStream(dev, stream);
```

9.4.4 StreamSync()

功能概述

同步指定 stream。

函数原型

```
virtual void tvm::runtime::DeviceAPI::StreamSync(Device dev, TVMStreamHandle stream)
```

参数说明

- Device& dev: 通过 DLDevice 创建的 device 结构体。
- TVMStreamHandle stream: 通过 CreateStream() 创建的 Stream 对象。

返回值

无返回值。

使用示例

```
DLDevice dev{kDLILUVATAR, 0};
DeviceAPI* api = DeviceAPI::Get(dev);
TVMStreamHandle stream = api->CreateStream(dev);
...
api->StreamSync(dev, stream);
```

9.4.5 DeviceSync()

功能概述

同步当前 Device。

函数原型

```
virtual void tvm::runtime::DeviceAPI::DeviceSync(Device dev)
```

参数说明

- Device& dev: 通过 DLDevice 创建的 device 结构体。

返回值

无返回值。

使用示例

```
DLDevice dev{kDLILUVATAR, 0};  
DeviceAPI* api = DeviceAPI::Get(dev);  
...  
api->DeviceSync(dev);
```

9.4.6 FreeStream()

功能概述

释放 stream 对象。

函数原型

```
virtual void tvm::runtime::DeviceAPI::FreeStream(Device dev, TVMStreamHandle stream)
```

参数说明

- Device& dev: 通过 DLDevice 创建的 device 结构体。
- TVMStreamHandle stream: 通过 CreateStream() 创建的 Stream 对象。

返回值

无返回值。

使用示例

```
DLDevice dev{kDLILUVATAR, 0};  
DeviceAPI* api = DeviceAPI::Get(dev);  
TVMStreamHandle stream = api->CreateStream(dev);  
...  
api->FreeStream(dev, stream);
```

9.4.7 AllocDataSpace()

功能概述

申请一块设备内存。

函数原型

```
virtual void* tvm::runtime::DeviceAPI::AllocDataSpace(Device dev, size_t nbytes, size_t
↪ alignment, DLDataType type_hint)
```

参数说明

- Device& dev: 通过 DLDevice 创建的 device 结构体。
- size_t nbytes: 申请内存的 byte 大小。
- size_t alignment: 内存对齐大小。
- DLDataType type_hint: 申请单个 elements 的类型。对于 Float 类型 Element 为: DL-DataType{kDLFloat, 32, 1};

DLDataType 定义如下:

```
typedef struct {
    /*!
    * \brief Type code of base types.
    * We keep it uint8_t instead of DLDataTypeCode for minimal memory
    * footprint, but the value should be one of DLDataTypeCode enum values.
    */
    uint8_t code;
    /*!
    * \brief Number of bits, common choices are 8, 16, 32.
    */
    uint8_t bits;
    /*! \brief Number of lanes in the type, used for vector types. */
    uint16_t lanes;
} DLDataType;
```

返回值

void* ptr: 返回申请的内存空间指针。

使用示例

```
DLDevice dev{kDLILUVATAR, 0};
DeviceAPI* api = DeviceAPI::Get(dev);
DLDataType dtype = DLDataType{kDLFloat, 32, 1};
void* ptr;
ptr = api->AllocDataSpace(dev, 1024, 64, dtype);
```

9.4.8 CopyDataFromTo()

功能概述

拷贝内存，从一个设备内存 place 到另一个设备内存 place。例如拷贝 cpu 内存数据到 GPU 内存，(可以是 cpu->gpu，也可以是 gpu->gpu、gpu->cpu 的内存数据拷贝)。

函数原型

```
virtual void tvm::runtime::DeviceAPI::CopyDataFromTo(DLTensor* from, DLTensor* to,
    TVMStreamHandle stream)
```

参数说明

- DLTensor* from: 通过 DLDevice 创建的 device 结构体。
- DLTensor* to: 申请内存的 byte 大小。
- TVMStreamHandle stream: 指定内存操作的流，可以是默认流 nullptr。

DLTensor 定义如下：

```
typedef struct {
    /*
     * \brief The opaque data pointer points to the allocated data. This will be
     * CUDA device pointer or cl_mem handle in OpenCL. This pointer is always
     * aligned to 256 bytes as in CUDA.
     */
    void* data;
    /* \brief The device of the tensor */
    DLDevice device;
    /* \brief Number of dimensions */
    int ndim;
    /* \brief The data type of the pointer */
    DLDataType dtype;
    /* \brief The shape of the tensor */
    int64_t* shape;
    /*
     * \brief strides of the tensor (in number of elements, not bytes)
     * can be NULL, indicating tensor is compact and row-major.
     */
    int64_t* strides;
    /* \brief The offset in bytes to the beginning pointer to data */
    uint64_t byte_offset;
} DLTensor;
```

返回值

无返回值。

使用示例

```
DLDevice dev{kDLILUVATAR, 0};
DeviceAPI* api = DeviceAPI::Get(dev);

DLTensor from;
from.data = const_cast<void*>(data);
from.device = DLDevice{kDLILUVATAR, 0};
from.dtype = DLDataType{kDLFloat, 32, 1};
ShapeTuple shape{1, 224, 224, 3};
from.ndim = static_cast<int>(shape.size());
from.shape = const_cast<int64_t*>(shape.data());
from.strides = nullptr;
from.byte_offset = 0;

DLTensor to;
...
api->CopyDataFromTo(&from, &to, nullptr);
```

9.4.9 FreeDataSpace()

功能概述

释放一块设备内存。

函数原型

```
virtual void tvm::runtime::DeviceAPI::FreeDataSpace(Device dev, void* ptr)
```

参数说明

- Device& dev: 通过 DLDevice 创建的 device 结构体。
- void* ptr: 设备指针。

返回值

无返回值。

使用示例

```
DLDevice dev{kDLILUVATAR, 0};
DeviceAPI* api = DeviceAPI::Get(dev);
void* ptr;
api->FreeDataSpace(dev, ptr);
```

9.5 Tensor 相关类和函数

Tensor 是深度学习常用的数据结构，通常表示为一个 N 维的数组。

9.5.1 DLTensor 结构体

功能概述：

DLTensor 是 IGIE 表示 Tensor 的基础结构体，在 NDAarray 中被使用。该结构体储存了 Tensor 的数据指针、设备类型、维度、shape 信息、内存 stride 和内存 offset 等 Memory 层面的信息。

结构体原型

DLTensor 定义如下：

```
typedef struct {
    /*!
     * \brief The opaque data pointer points to the allocated data. This will be
     * CUDA device pointer or cl_mem handle in OpenCL. This pointer is always
     * aligned to 256 bytes as in CUDA.
     */
    void* data;
    /*! \brief The device of the tensor */
    DLDevice device;
    /*! \brief Number of dimensions */
    int ndim;
    /*! \brief The data type of the pointer*/
    DLDataType dtype;
    /*! \brief The shape of the tensor */
    int64_t* shape;
    /*!
     * \brief strides of the tensor (in number of elements, not bytes)
     * can be NULL, indicating tensor is compact and row-majored.
     */
    int64_t* strides;
    /*! \brief The offset in bytes to the beginning pointer to data */
    uint64_t byte_offset;
} DLTensor;
```

参数说明

- void* data: 数据指针，可以是 CPU Memory 指针，也可以是 CUDA Memory 指针。
- DLDevice device: 指定 tensor 所在设备。
- int ndim: Tensor 维度。
- DLDataType dtype: Tensor 的数据类型。
- int64_t* shape: Tensor 的 shape 信息。
- int64_t* strides: Tensor 的 stride, stride 的单位是 element 的个数，而非 nbyte。
- uint64_t byte_offset: 相当于 data 数据开始位置的偏移量。

使用示例

```
DLTensor from;
from.data = const_cast<void*>(data);
from.device = DLDevice{kDLILUVATAR, 0};
from.dtype = DLDataType{kDLFloat, 32, 1};
```

```
ShapeTuple shape{1, 224, 224, 3};
from.ndim = static_cast<int>(shape.size());
from.shape = const_cast<int64_t *>(shape.data());
from.strides = nullptr;
from.byte_offset = 0;
```

9.5.2 NDArrary 类

NDArrary 是表示 N 维的数组的类，主要用于创建模型的输入和输出。

9.5.3 NDArrary::Empty()

功能概述

创建一个空 NDArrary。

函数原型

```
static NDArrary tvm::runtime::NDArrary::Empty(ShapeTuple shape, DLDataType dtype, Device
↪ dev, Optional<String> mem_scope = NullOpt)
```

参数说明

- ShapeTuple shape: 新 array 的 shape 大小。
- DLDataType dtype: 新 array 的数据类型。
- Device& dev: 通过 DLDevice 创建的 device 结构体。
- mem_scope: 内存类型。

返回值

返回 NDArrary 对象。

使用示例

```
DLDevice dev{kDLILUVATAR, 0};
tvm::runtime::NDArrary x;
x = tvm::runtime::NDArrary::Empty({2, 2}, DLDataType{kDLFloat, 32, 1}, dev);
```

9.5.4 NDArrary::Shape()

功能概述

获取 NDArrary 的 Shape 信息。

函数原型

```
ShapeTuple tvm::runtime::NDArray::Shape()
```

参数说明

无参数

返回值

返回 ShapeTuple 对象。

使用示例

```
tvm::runtime::NDArray input;
tvm::runtime::ShapeTuple input_shape = input.Shape();

// 打印 shape 信息
std::cout << "Input Shape: [";
for (int i=0; i < input_shape.size(); i++){
std::cout << input_shape.data()[i] << ",";
}
std::cout << "]" << std::endl;
```

9.5.5 NDArray::DataType()

功能概述

获取 NDArray 的 Type 信息。

函数原型

```
DataType tvm::runtime::NDArray::DataType()
```

参数说明

无参数

返回值

返回 DataType 对象。

使用示例:

```
tvm::runtime::NDArray input;
tvm::runtime::DataType input_type = input.DataType();

// 打印 Dtype 信息
std::string dtype = tvm::runtime::DLDataType2String(input_type);
std::cout << "Input Type: " << dtype << std::endl;
```

9.5.6 NDArray::CopyFromTo()

功能概述

拷贝一个 DLTensor 数据到另一个 DLTensor，例如，拷贝 CPU 上的 DLTensor 数据到 GPU DLTensor。

函数原型

```
static void tvm::runtime::NDArray::CopyFromTo(const DLTensor* from, DLTensor* to,
    TVMStreamHandle stream = nullptr)
```

参数说明

- DLTensor* from: 源 DLTensor。
- DLTensor* to: 目标 DLTensor。
- TVMStreamHandle stream: 执行拷贝操作的 stream，默认是 nullptr stream。

返回值

无返回值。

使用示例

```
DLTensor from;
from.data = const_cast<void*>(data);
from.device = DLDevice{kDLCPU, 0};
from.dtype = DLDataType{kDLFloat, 32, 1};
ShapeTuple shape{1, 224, 224, 3};
from.ndim = static_cast<int>(shape.size());
from.shape = const_cast<int64_t*>(shape.data());
from.strides = nullptr;
from.byte_offset = 0;

DLTensor to;
to.device = DLDevice{kDLILUVATAR, 0};

tvm::runtime::NDArray::CopyFromTo(from, to)
```

9.5.7 NDArray::CopyToBytes()

功能概述

拷贝数据内容到另一个 NDArray，例如，拷贝 CPU 上的 data 数据到 GPU DLTensor。

函数原型

```
void tvm::runtime::NDArray::CopyToBytes(void* data, size_t nbytes)
```

参数说明

- void* data: 数据指针。

- `size_t nbytes`: 拷贝数据大小 bytes。

返回值

无返回值。

使用示例

```
// 例如: 创建一个 shape=[4], dtype 类型为 float32, 在 Iluvatar GPU 上的 Tensor

// 设置 Tensor 位于的设备
DLDevice dev{kDLILUVATAR, 0};
// 设置 Tensor 的类型
DLDataType dtype{kDLFloat, 32, 1};
tvm::runtime::NDArray input = tvm::runtime::NDArray::Empty({4}, dtype, dev);
// 拷贝 CPU 数据 data 到 GPU Tensor 中
float data[4] = {1.0, 2.0, 3.0, 4.0};
input.CopyFromBytes(data, 4 * sizeof(float));
```

9.5.8 NDArray::FromDLPack()

功能概述

从一个 `dlpack tensor` 创建 `NDArray`。

函数原型

```
static NDArray tvm::runtime::NDArray::FromDLPack(DLManagedTensor* tensor)
```

参数说明

`DLManagedTensor* tensor`: `dlpack tensor`。

返回值

返回 `NDArray`。

使用说明:

`DLPack` 是一种通用的 `Tensor` 数据结构, 它允许不同框架之间的 `Tensor` 进行相互转换。`DLPack` 相关介绍可以查看 [DLPack 文档](#)。

因此, `tvm::runtime::NDArray::FromDLPack` 接口可以将支持 `DLPack` 的其他框架 `Tensor` 转换为 `IGIE NDArray` 进行推理, 支持的框架包括: `Torch`、`TensorFlow`、`Paddle`、`ONNXRuntime`。

转换过程如下: **torch tensor -> dlpack tensor -> igie tensor(NDArray)**

使用 `tvm::runtime::NDArray::FromDLPack` 创建 `Tensor`, 支持零拷贝的方式创建 `Tensor`:

```
// 该接口支持以零拷贝的方式构造 IGIE Tensor, void *data 可以是 CUDA Memory 数据指针。
tvm::runtime::NDArray CreateNDArray(void *data,
                                     ShapeTuple &shape,
                                     const DLDataType &dtype,
```

```

const DLDevice &dev) {
    // 创建 DLTensor
    DLTensor tensor;
    tensor.data = data;
    tensor.device = dev;
    tensor.ndim = static_cast<int>(shape.size());
    tensor.shape = const_cast<int64_t *>(shape.data());
    tensor.dtype = dtype;
    tensor.strides = nullptr;
    tensor.byte_offset = 0;

    // 创建 DLManagerTensor (DLPack 结构体)
    DLManagedTensor managed_tensor;
    managed_tensor.dl_tensor = tensor;
    managed_tensor.deleter = nullptr;

    tvm::runtime::NDArray x_array = tvm::runtime::NDArray::FromDLPack(&managed_tensor);
    return x_array;
}

// GPU 内存的 data
float *d_data;
cudaMalloc((void**)&d_data, size * sizeof(float));
cudaMemcpy(d_data, h_data, size * sizeof(float), cudaMemcpyHostToDevice);

// 调用 CreateNDArray 函数，零拷贝方式创建 IGIE Tensor,
ShapeTuple in_shape{1, 224, 224, 3};
tvm::runtime::NDArray in_tensor;
in_tensor = CreateNDArray(mag_tensor, d_data, in_shape, DLDataType{kDLFloat, 32, 1}, dev);

```

9.6 加载模型引擎相关类和函数

9.6.1 Module 类

IGIE 中表示运行时模型的类。

9.6.1.1 Module::LoadFromFile()

功能概述

从引擎文件构造 Module 对象，用于推理。

函数原型

```

static Module tvm::runtime::Module::LoadFromFile(const std::string& file_name, const
    ↪ std::string& format = "")

```

参数说明

- `const std::string& file_name`: 引擎文件路径。
- `const std::string& format = ""`: 引擎文件类型，默认为空。

返回值

返回 `Module` 对象。

使用说明:

```
// 加载引擎文件到 IGIE 中
tvm::runtime::Module mod_factory = tvm::runtime::Module::LoadFromFile("./resnet50.so");
```

9.6.1.2 Module::GetFunction("default")()

功能概述

创建 `GraphExecutor` 对象，加载 `Module` 到 `Device GPU`。

函数原型

```
PackedFunc tvm::runtime::Module::GetFunction("default")(DLDevice device)
```

参数说明

`DLDevice device`: 设备对象。

返回值

`GraphExecutor` 对象。

使用说明:

因为 `GraphExecutor` 的父类是 `ModuleNode` (`class GraphExecutor : public ModuleNode()`)，因此，`mod_factory.GetFunction("default")(dev)` 返回的 `GraphExecutor` 对象可以是 `Module` 类型。

```
DLDevice dev{kDLILUVATAR, 0};
// 加载引擎文件到 IGIE 中
tvm::runtime::Module mod_factory = tvm::runtime::Module::LoadFromFile("./resnet50.so");
// 创建 GraphExecutor 对象，加载 Module 到 Device GPU。
tvm::runtime::Module graph_executor = mod_factory.GetFunction("default")(dev);
```

9.7 模型输入输出信息相关函数

9.7.1 GraphExecutor::GetFunction()

功能概述

通过统一的 `GetFunction` 接口，获取 `GraphExecutor` 的所有成员函数。比如：

- `get_input()`: 获取模型输入信息。
- `set_input()`: 设置模型输入。
- `set_input_zero_copy()`: 以零拷贝的形式设置模型输入。
- `get_output()`: 获取模型输出。
- `get_num_outputs()`: 获取模型的输出个数。
- `get_num_inputs()`: 获取模型的输入个数。
- `get_input_names()`: 获取模型的输入 name。
- `get_output_names()`: 获取模型的输出 name。

函数原型

```
PackedFunc GraphExecutor::GetFunction(const std::string& name)
```

参数说明

`const std::string& name`: 函数名字符串。

返回值

返回函数对象，可以直接调用。

使用说明:

```
DLDevice dev{kDLILUVATAR, 0};
// 加载引擎文件到 IGIE 中
tvm::runtime::Module mod_factory = tvm::runtime::Module::LoadFromFile("./resnet50.so");
// 创建 GraphExecutor 对象, 加载 Module 到 Device GPU.
tvm::runtime::Module graph_executor = mod_factory.GetFunction("default")(dev);

// 获取模型输入个数
tvm::runtime::PackedFunc get_num_inputs = graph_executor.GetFunction("get_num_inputs");

// 获取模型输出个数
tvm::runtime::PackedFunc get_num_outputs = graph_executor.GetFunction("get_num_outputs");

// 获取模型全部输入 names
tvm::runtime::PackedFunc get_input_names = graph_executor.GetFunction("get_input_names");

// 获取模型全部输出 names
tvm::runtime::PackedFunc get_output_names =
    graph_executor.GetFunction("get_output_names");

// 获取模型输入信息
tvm::runtime::PackedFunc set_input = graph_executor.GetFunction("set_input");

// 设置模型输入函数
tvm::runtime::PackedFunc set_input = graph_executor.GetFunction("set_input");

// 设置模型输入函数
tvm::runtime::PackedFunc set_input_zero_copy =
    graph_executor.GetFunction("set_input_zero_copy");
```

```
// 获取模型输出函数  
tvm::runtime::PackedFunc get_output = graph_executor.GetFunction("get_output");
```

9.7.2 get_num_inputs()

功能概述

获取模型输入的个数。

函数原型

```
GraphExecutor::GetFunction("get_num_inputs")()
```

参数说明

无

返回值

返回模型输入的个数。

使用说明:

```
DLDevice dev{kDLILUVATAR, 0};  
// 加载引擎文件到 IGIE 中  
tvm::runtime::Module mod_factory = tvm::runtime::Module::LoadFromFile("./resnet50.so");  
// 创建 GraphExecutor 对象, 加载 Module 到 Device GPU。  
tvm::runtime::Module graph_executor = mod_factory.GetFunction("default")(dev);  
  
// 获取模型输入个数  
tvm::runtime::PackedFunc get_num_inputs = graph_executor.GetFunction("get_num_inputs");  
int input_num = get_num_inputs();  
std::cout << "get_num_inputs: " << input_num << std::endl;
```

9.7.3 get_num_outputs()

功能概述

获取模型输出的个数。

函数原型

```
int GraphExecutor::GetFunction("get_num_outputs")()
```

参数说明

无

返回值

返回模型输出的个数。

使用说明：

```
DLDevice dev{kDLILUVATAR, 0};
// 加载引擎文件到 IGIE 中
tvm::runtime::Module mod_factory = tvm::runtime::Module::LoadFromFile("./resnet50.so");
// 创建 GraphExecutor 对象, 加载 Module 到 Device GPU。
tvm::runtime::Module graph_executor = mod_factory.GetFunction("default")(dev);

// 获取模型输出个数
tvm::runtime::PackedFunc get_num_outputs = graph_executor.GetFunction("get_num_outputs");
int output_num = graph_executor.NumOutputs();
std::cout << "get_num_outputs: " << output_num << std::endl;
```

9.7.4 get_input_names()

功能概述

获取模型所有输入 Name。

函数原型

```
std::vector<std::string> GraphExecutor::GetFunction("get_input_names")()
```

参数说明

无

返回值

返回模型输入名数组。

使用说明：

```
DLDevice dev{kDLILUVATAR, 0};
// 加载引擎文件到 IGIE 中
tvm::runtime::Module mod_factory = tvm::runtime::Module::LoadFromFile("./resnet50.so");
// 创建 GraphExecutor 对象, 加载 Module 到 Device GPU。
tvm::runtime::Module graph_executor = mod_factory.GetFunction("default")(dev);

// 获取模型全部输入 names
tvm::runtime::PackedFunc get_input_names = graph_executor.GetFunction("get_input_names");
void* input_names = get_input_names();
std::vector<std::string>* vector_names =
    ↪ static_cast<std::vector<std::string>*>(input_names);
for (const auto& str : *vector_names) {
    std::cout << "Input Name: " << str << std::endl;
}
```

9.7.5 get_output_names()

功能概述

获取模型所有输出 Name。

函数原型

```
std::vector<std::string> GraphExecutor::GetFunction("get_output_names")()
```

参数说明

无

返回值

返回模型输出名数组。

使用说明:

```
DLDevice dev{kDLILUVATAR, 0};
// 加载引擎文件到 IGIE 中
tvm::runtime::Module mod_factory = tvm::runtime::Module::LoadFromFile("./resnet50.so");
// 创建 GraphExecutor 对象, 加载 Module 到 Device GPU。
tvm::runtime::Module graph_executor = mod_factory.GetFunction("default")(dev);

// 获取模型全部输出 names
tvm::runtime::PackedFunc get_output_names =
    graph_executor.GetFunction("get_output_names");
void* output_names = get_output_names();
vector_names = static_cast<std::vector<std::string*>>(output_names);
for (const auto& str : *vector_names) {
    std::cout << str << std::endl;
}
```

9.7.6 get_input_index()

功能概述

根据输入名称获取输入的 index。

函数原型

```
int GraphExecutor::GetFunction("get_input_index")(const std::string& name)
```

参数说明

const std::string& name: 模型输入名。

返回值

输入的 index。

使用说明:

```
DLDevice dev{kDLILUVATAR, 0};
// 加载引擎文件到 IGIE 中
tvm::runtime::Module mod_factory = tvm::runtime::Module::LoadFromFile("./resnet50.so");
// 创建 GraphExecutor 对象, 加载 Module 到 Device GPU。
tvm::runtime::Module graph_executor = mod_factory.GetFunction("default")(dev);
tvm::runtime::PackedFunc get_output_names =
    graph_executor.GetFunction("get_input_index");

int index = get_output_names("input_0");
```

9.7.7 get_output_index()

功能概述

根据输出名称获取输出的 index。

函数原型

```
int GraphExecutor::GetFunction("get_output_index")(const std::string& name)
```

参数说明

const std::string& name: 模型输出名。

返回值

输出的 index。

使用说明:

```
DLDevice dev{kDLILUVATAR, 0};
// 加载引擎文件到 IGIE 中
tvm::runtime::Module mod_factory = tvm::runtime::Module::LoadFromFile("./resnet50.so");
// 创建 GraphExecutor 对象, 加载 Module 到 Device GPU。
tvm::runtime::Module graph_executor = mod_factory.GetFunction("default")(dev);
tvm::runtime::PackedFunc get_output_names =
    graph_executor.GetFunction("get_output_index");

int index = get_output_index("softmax_0");
```

9.7.8 get_input()

功能概述

设置模型输入, 获取输入的 Shape、Dtype 和 data 信息。

函数原型

```
NDArray GraphExecutor::GetFunction("get_input")(int index)
```

参数说明

int index: 第几个输入。

返回值

返回 NDArray。

使用说明:

```
DLDevice dev{kDLILUVATAR, 0};
// 加载引擎文件到 IGIE 中
tvm::runtime::Module mod_factory = tvm::runtime::Module::LoadFromFile("./resnet50.so");
// 创建 GraphExecutor 对象, 加载 Module 到 Device GPU。
tvm::runtime::Module graph_executor = mod_factory.GetFunction("default")(dev);
// 获取模型输入信息
tvm::runtime::PackedFunc get_input = gmod.GetFunction("get_input");
tvm::runtime::NDArray input_ = get_input(0);
tvm::runtime::ShapeTuplet input_shape = input_.Shape();
tvm::runtime::DataType input_type = input_.DataType();

// 打印 shape 信息
std::cout << "Input Shape: [";
for (int i=0; i < input_shape.size(); i++){
    std::cout << input_shape.data()[i] << ", ";
}
std::cout << "]" << std::endl;
// 打印 Dtype 信息
std::string dtype = tvm::runtime::DLDataType2String(input_type);
std::cout << "Input Type: " << dtype << std::endl;
```

9.7.9 set_input()

功能概述

设置模型输入。

函数原型

```
void GraphExecutor::GetFunction("set_input")(int index, DLTensor* data_in)
```

参数说明

- int index: 第几个输入。
- DLTensor* data_in: 输入的 DLTensor。

返回值

无返回值。

使用说明:

```
DLDevice dev{kDLILUVATAR, 0};
// 加载引擎文件到 IGIE 中
tvm::runtime::Module mod_factory = tvm::runtime::Module::LoadFromFile("./resnet50.so");
// 创建 GraphExecutor 对象, 加载 Module 到 Device GPU。
tvm::runtime::Module graph_executor = mod_factory.GetFunction("default")(dev);
// 获取模型输入信息
tvm::runtime::PackedFunc set_input = graph_executor.GetFunction("get_input");

// 设置模型输入函数
tvm::runtime::PackedFunc set_input = graph_executor.GetFunction("set_input");

DLTensor* input;
// 通过指定 input name 设置输入
set_input("data", input);
// 或者 通过指定 index 设置输入
set_input(0, input);
```

9.7.10 set_input_zero_copy()

功能概述

以零拷贝的方式设置模型输入。如果是在 GPU 运行模型, 则必须保证 data_ref 中的 data 指针指向 GPU 内存, 该接口本质是只改变 DLTensor 的 data 指针。

函数原型

```
void GraphExecutor::GetFunction("set_input_zero_copy")(int index, DLTensor* data_ref)
```

参数说明

- int index: 第几个输入。
- DLTensor* data_ref: 输入的 DLTensor。如果是在 GPU 运行模型, 则必须保证 data_ref 中的 data 指针指向 GPU 内存。

返回值

无返回值。

使用说明:

```
DLDevice dev{kDLILUVATAR, 0};
// 加载引擎文件到 IGIE 中
tvm::runtime::Module mod_factory = tvm::runtime::Module::LoadFromFile("./resnet50.so");
// 创建 GraphExecutor 对象, 加载 Module 到 Device GPU。
tvm::runtime::Module graph_executor = mod_factory.GetFunction("default")(dev);
// 设置模型输入函数
tvm::runtime::PackedFunc set_input_zero_copy =
    graph_executor.GetFunction("set_input_zero_copy");
```

```
DLTensor* data_ref;
int index = set_input_zero_copy(data_ref);
```

9.7.11 get_output()

功能概述

获取模型的输出。

函数原型

```
NDArray GraphExecutor::GetFunction("get_output")(int index)
```

参数说明

int index: 第几个输出。

返回值

返回 NDArray。

使用说明:

```
DLDevice dev{kDLILUVATAR, 0};
// 加载引擎文件到 IGIE 中
tvm::runtime::Module mod_factory = tvm::runtime::Module::LoadFromFile("./resnet50.so");
// 创建 GraphExecutor 对象, 加载 Module 到 Device GPU。
tvm::runtime::Module graph_executor = mod_factory.GetFunction("default")(dev);
// 获取模型输出函数
tvm::runtime::PackedFunc get_output = graph_executor.GetFunction("get_output");

tvm::runtime::NDArray output = get_output(0);

tvm::runtime::ShapeTuple output_shape = output.Shape();
tvm::runtime::DataType output_type = output.DataType();

// 打印 shape 信息
std::cout << "Output Shape: [";
for (int i=0; i< output_shape.size(); i++){
    std::cout << output_shape.data()[i] << ", ";
}
std::cout << "]" << std::endl;

// 打印 Dtype 信息
std::string dtype = tvm::runtime::DLDataType2String(output_type);
std::cout << "Output Type: " << dtype << std::endl;
```

9.8 模型运行相关函数

9.8.1 run()

功能概述

模型运行函数。

函数原型

```
GraphExecutor::GetFunction("run")()
```

参数说明

无。

返回值

无返回。

使用说明:

```
DLDevice dev{kDLILUVATAR, 0};
// 加载引擎文件到 IGIE 中
tvm::runtime::Module mod_factory = tvm::runtime::Module::LoadFromFile("./resnet50.so");
// 创建 GraphExecutor 对象, 加载 Module 到 Device GPU。
tvm::runtime::Module graph_executor = mod_factory.GetFunction("default")(dev);
tvm::runtime::PackedFunc run = graph_executor.GetFunction("run");

run();
```

9.9 性能统计相关函数

9.9.1 GetTimer()

功能概述

创建 Timer 对象, 用于耗时统计。

函数原型

```
tvm::runtime::Timer timer = tvm::runtime::Timer::GetTimer(dev);
```

参数说明

Device& dev: 通过 DLDevice 创建的 device 结构体。

返回值

Timer 对象。

使用说明:

通过传入不同类型的 Device 创建 Timer 对象，控制开启 GPU/CPU 耗时统计。

```
// CPU 耗时统计
DLDevice dev{kDLCPU, 0};
tvm::runtime::Timer timer = tvm::runtime::Timer::GetTimer(dev);

// GPU 耗时统计
DLDevice dev{kDLILUVATAR, 0};
tvm::runtime::Timer timer = tvm::runtime::Timer::GetTimer(dev);
```

9.9.2 Start()

功能概述

开启耗时统计。

参数说明

无。

返回值

无返回值。

使用说明:

```
timer->Start();
```

9.9.3 Stop()

功能概述

关闭耗时统计。

参数说明

无。

返回值

无返回值。

使用说明:

```
timer->Stop();
```

9.9.4 SyncAndGetElapsedNanos()

功能概述

计算 Start 和 Stop 之间的耗时。

参数说明

无。

返回值

返回耗时统计，类型为 int64_t，单位为 ns。

使用说明：

```
int64_t time = timer->SyncAndGetElapsedNanos();
```

9.10 推理完整参考示例

Note

IGIE C++ 示例代码不在本发布包中。如有需要，请向您的应用工程师获取示例代码。

下面是完整的 C++ 代码：

```
#include <dlpack/dlpack.h>
#include <tvm/runtime/module.h>
#include <tvm/runtime/packed_func.h>
#include <tvm/runtime/registry.h>
#include <tvm/runtime/ndarray.h>
#include <tvm/runtime/profiling.h>
#include <cstdio>
#include <stdlib.h>
#include <iostream>
#include <iomanip>
#include <string>
#include <fstream>
#include <errno.h>
#include <exception>

// 读取图片数据到 data
bool GetImageData(std::string file_path, float *data, uint32_t size) {
try {
    std::ifstream file;
    file.exceptions(file.failbit | file.badbit);
    file.open(file_path);
    if (file.is_open()) {
        float tmp;
        uint32_t count = 0;
        while (file >> tmp) {
```

```

        *data++ = tmp;
        count++;
        if (count >= size) {
            break;
        }
    }
    file.close();
    return true;
}
} catch (std::ifstream::failure &e) {
    std::cout << "open file failed: " << e.what() << std::endl;
    return false;
}
return false;
}

// 保存推理结果
bool SaveOutput(const std::string &file_path, float *data, uint32_t size) {
    try {
        std::ofstream out_file;
        out_file.exceptions(out_file.failbit | out_file.badbit);
        out_file.open(file_path);
        if (out_file.is_open()) {
            float tmp;
            for (uint32_t i = 0; i < size; ++i) {
                tmp = *data++;
                out_file << tmp;
                out_file << '\n';
            }
            out_file.close();
            return true;
        }
    } catch (std::ifstream::failure &e) {
        std::cout << "open file failed: " << e.what() << std::endl;
        return false;
    }
    return false;
}

// 使用引擎文件部署模型
void DeployResnet50(std::string in_file, std::string out_file, std::string dev_type =
    ↪ "gpu") {
    LOG(INFO) << "Running C++ resnet50 graph executor...";

    // 读取测试图片数据
    float *data = new float[1 * 3 * 224 * 224];
    if (!GetImageData(in_file, data, 1*3*224*224)) {
        std::cout << "Get data failed!" << std::endl;
        return;
    }
}

```

```
// 设置 device
DLDevice dev{kDLILUVATAR, 0};
tvm::runtime::Module mod_factory;

// 加载引擎文件
if (dev_type == "cpu") {
    dev.device_type = kDLCPU;
    mod_factory = tvm::runtime::Module::LoadFromFile("model/resnet50_b1_llvm.so");
} else if (dev_type == "gpu") {
    dev.device_type = kDLILUVATAR;
    mod_factory = tvm::runtime::Module::LoadFromFile("model/resnet50_b1_iluvatar.so");
} else {
    std::cout << "ERROR: unrecognized device type!" << std::endl;
    return;
}

// create the graph executor module
tvm::runtime::Module gmod = mod_factory.GetFunction("default")(dev);

// 获取模型输入个数
tvm::runtime::PackedFunc get_num_inputs = gmod.GetFunction("get_num_inputs");
int input_num = get_num_inputs();
std::cout << "get_num_inputs: " << input_num << std::endl;

// 获取模型输出个数
tvm::runtime::PackedFunc get_num_outputs = gmod.GetFunction("get_num_outputs");
int output_num = get_num_outputs();
std::cout << "get_num_outputs: " << output_num << std::endl;

// 获取模型全部输入 names
tvm::runtime::PackedFunc get_input_names = gmod.GetFunction("get_input_names");
void* input_names = get_input_names();

std::vector<std::string>* vector_names =
    static_cast<std::vector<std::string>*>(input_names);
for (const auto& str : *vector_names) {
    std::cout << str << std::endl;
}

// 获取模型全部输出 names
tvm::runtime::PackedFunc get_output_names = gmod.GetFunction("get_output_names");
void* output_names = get_output_names();

vector_names = static_cast<std::vector<std::string>*>(output_names);
for (const auto& str : *vector_names) {
    std::cout << str << std::endl;
}

tvm::runtime::PackedFunc get_input = gmod.GetFunction("get_input");
```

```

tvm::runtime::NDArray input_ = get_input(0);
tvm::runtime::ShapeTuple input_shape = input_.Shape();
tvm::runtime::DataType input_type = input_.DataType();
std::cout << "Input Shape: [";
for (int i=0; i< input_shape.size(); i++){
    std::cout << input_shape.data()[i] << ",";
}
std::cout << "]" << std::endl;
std::string dtype = tvm::runtime::DLDataType2String(input_type);
std::cout << "Input Type: " << dtype << std::endl;

// 获取模型推理函数
tvm::runtime::PackedFunc set_input = gmod.GetFunction("set_input");
tvm::runtime::PackedFunc get_output = gmod.GetFunction("get_output");
tvm::runtime::PackedFunc run = gmod.GetFunction("run");

// 创建空的 Tensor
tvm::runtime::NDArray input = tvm::runtime::NDArray::Empty({1, 3, 224, 224},
    DLDataType{kDLFloat, 32, 1}, dev);

// 创建 Timer
tvm::runtime::Timer timer = tvm::runtime::Timer::GetTimer(dev);

// Warm Up
std::cout << "Start Warmup ..." << std::endl;
for(size_t i = 0; i < 3; i++)
{
    run();
}

// 拷贝 Host data 到 GPU input tensor 中 (HToD)
input.CopyFromBytes(data, 1 * 3 * 224 * 224 * sizeof(float));

// 设置模型输入
set_input("data", input);

// 推理模型
timer->Start();
run();
timer->Stop();

// 计算耗时
float latency = timer->SyncAndGetElapsedNanos() / 1e6;    // ns->ms
float FPS = 1000 / time;
std::cout << "Consume time: " << latency << " ms" << std::endl;
std::cout << "FPS: " << FPS << std::endl;

// get the output
tvm::runtime::NDArray output_gpu;
output_gpu = get_output(0);

```

```

tvm::runtime::ShapeTuple output_shape = output_gpu.Shape();
tvm::runtime::DataType output_type = output_gpu.DataType();

std::cout << "Output Shape: [";
for (int i=0; i< output_shape.size(); i++){
    std::cout << output_shape.data()[i] << ",";
}
std::cout << "]" << std::endl;

// 打印 Dtype 信息
dtype = tvm::runtime::DLDataType2String(output_type);
std::cout << "Output Type: " << dtype << std::endl;

// 拷贝 GPU Tensor 数据到 Host output 中 (DtoH)
float output[1000];
output_gpu.CopyToBytes((void*)output, 1000 * sizeof(float));

for (int32_t i = 0; i < 10; ++i) {
    std::cout << ((float*)output)[i] << '\t';
}
std::cout << std::endl;

// save inference output to postprocess
SaveOutput(out_file, output, 1000);

if (data != nullptr) {
    LOG(INFO) << "free data when device type is cuda!!!";
    delete[] data;
}
LOG(INFO) << "End running resnet50 graph executor...";
}

int main(int argc, char *argv[]) {
if (argc != 4) {
    std::cout << "ERROR: two arguments for main, but get " << argc - 1 << std::endl;
    return 1;
}
std::string in_file(argv[1]);
std::string out_file(argv[2]);
std::string dev_type(argv[3]);

std::cout << "in_file: " << in_file << ", out_file: " << out_file << std::endl;
DeployResnet50(in_file, out_file, dev_type);
return 0;
}

```

执行：

```
$ cd igie/tests/test_iluvatar/test_cpp/  
$ bash run.sh
```

10 常见问题

问题 1: IGIE 的 Relay OP 是什么?

Relay 是 IGIE Graph 和 OP 的集合, 所有算子的定义都在 Relay Package 下面。因此 IGIE 的算子有时候也会被称为 Relay OP。

您在使用 IGIE 时, 可以通过 Relay OP 构造网络, 例如:

```
data = relay.var("data", shape=input_shape, dtype=dtype)
conv = relay.nn.conv2d(data, weight)
add = relay.add(conv, bias)
relu = relay.nn.relu(add)
```

您也可以使用 relay.build 进行模型 Graph 编译:

```
engine = relay.build(mod, params=params, target=target)
```

问题 2: 推理模型时, 使用什么 Layout 性能最好?

目前天数智芯高性能算子库的开发, 针对 NHWC Layout 算子适配程度和性能优化都是最好的, 对于 NCHW Layout 的算子目前优化支持较少。

因此, 建议您导入模型后尽量使用 NHWC Layout 进行模型推理。

如果模型内部使用的是 HCHW Layout, 可以在导入模型后使用下面代码对 Layout 进行转换, 注意输入输出的 Layout 是不会改变的:

```
from tvm.relay.transform.iluvatar.optimize_graph import convert_graph_layout_simplify
# NCHW -> NHWC
convert_layout="NHWC"
mod = convert_graph_layout_simplify(mod, convert_layout=convert_layout)
```

问题 3: 运行模型时, 出现“现在不支持, 以后会支持”的屏幕输出, 是怎么回事?

目前 ixInfer 算子库有部分 Shape 还没有完全支持, 遇到不支持的 conv2d 实现会输出“现在不支持, 以后会支持”字样, 您需要将该 Shape 提交给您的应用工程师, 应用工程师将进一步提交给对应研发进行处理; 或者您可以将该 Shape 暂时回退使用 IGIE 的默认实现。

问题 4: igie-exec 工具量化表生成

问题描述

IGIE 对于不同 Batch 的 INT8 推理, 是否每个 Batch 都需要重新量化? 量化表是否生成一次即可?

问题解答

目前 igie-exec 提供了两种量化方案供您选择: IGIE 量化和 ONNXRuntime 量化。

- 对于 IGIE 量化 (--igie-quantize 参数):
 - 优点: 适用大部分模型, 支持 FP16 和 INT8 量化
 - 缺点: 无法控制每一个量化节点, 只能跳过一类节点

- 对于 ONNXRuntime 量化 (`--onnx-quantize` 参数):
 - 优点: 对于 ONNX 模型支持较好, 可以配置参数选择哪些节点不量化, 哪些节点量化, 控制粒度更细
 - 缺点: 只支持 ONNX 模型

使用 `--igie-quantize` 参数进行 INT8 量化时, 会自动将量化系数 (量化表) 保存在当前目录的 `quantize_scale` 目录中, 下次进行其他 Batch 推理时, 会自动加载量化表, 无需再次量化。当 Batch 较大时, 会导致模型量化很慢或者内存溢出 OOM, 推荐使用 `batch=1` 进行 INT8 量化, 然后再执行其他 Batch 推理, 因为不同的 Batch 使用的量化系数是一样的。

使用 `--onnx-quantize` 参数进行 INT8 量化时, `igie-exec` 也推荐先对 1 batch 进行量化, 再次进行 32/64/128 batch 推理时即不再需要量化。`igie-exec` 内部有针对 ONNX 模型的 Batch 自动修改工具, 可以在 1 batch 已量化模型的基础上修改 Batch。

问题 5: 在 ARM 设备上安装 `onnxoptimizer` 和 `onnxsim` 报错

问题描述

在 ARM 设备上安装 `igie-exec` 的 `requirement.txt` 后报错。

问题解答

如果是编译安装 `onnxoptimizer`, 可能是因为 CMake 版本过低导致的编译失败, 需要升级 CMake。一般错误信息如下:

```
CMake Error at CMakeLists.txt:1 (cmake_minimum_required):
  CMake 3.22 or higher is required. You are running version 3.16.9
```

请参考下列步骤解决问题:

1. 升级 CMake 版本:

```
# 向您的应用工程师获取 cmake.git 文件

$ ./build_cmake.sh
$ ./install_cmake.sh
```

2. 升级完成后, 查看 CMake 版本:

```
$ cmake --version
cmake version 3.25.2-corex.3.1.0.20230322.534
```

3. 重新安装 `onnxoptimizer`。

问题 6: 如何使用 IGIE DEBUG 模式?

您可以通过编译选项 `ILUVATAR_LOG_DEBUG` 来控制是否打印 DEBUG 信息, 在 `igie/cmake/config.cmake` 文件中将该选项设置为 ON, 重新编译 IGIE 后即可看到 DEBUG 信息。

问题 7: 如何开启 IGIE 多算子库支持?

IGIE 可以在以下 4 种模式下工作, 分别能够支持不同的算子库:

- **llvm**: 模型中的所有 OP 全部走 IGIE CPU 实现, 性能最差, 一般用于精度对比

- **iluvatar**: 模型中的所有 OP 全部走 IGIE GPU 实现，性能相对较差，通用性高
- **iluvatar_with_cudnn_cublas**: 从 ixDNN、ixBLAS 库中寻找性能相对较高的实现，通用性取决于 ixDNN、ixBLAS 的实现情况
- **iluvatar_with_all_libs**: 从 ixInfer、ixDNN、ixBLAS 库中寻找性能最好的实现，能够获取最佳性能，通用性取决于 ixInfer、ixDNN、ixBLAS 的实现情况

llvm 模式使用 CPU 计算，在实际模型适配中很少用到。其余三种模式：

- 通用性和精度：iluvatar > iluvatar_with_cudnn_cublas > iluvatar_with_all_libs
- 性能：iluvatar_with_all_libs > iluvatar > iluvatar_with_cudnn_cublas

在编译生成引擎文件时，您可以通过设置 **target** 参数来控制 IGIE 工作在何种模式下：

```
# llvm
target = tvm.target.Target("llvm")

# iluvatar
target = tvm.target.iluvatar()

# iluvatar_with_cudnn_cublas
target = tvm.target.iluvatar(options="-libs=cudnn,cublas")

# iluvatar_with_all_libs
target = tvm.target.iluvatar(options="-libs=cudnn,cublas,ixinfer")
```

模型适配中遇到无法运行或者是精度问题，您可以逐步回退到 iluvatar 工作模式来验证模型的支持情况。

问题 8：IGIE 推理程序 Hang，如何解决？

IGIE INT8 推理可能会出现程序 Hang 住，这有可能是因为调用了 ixInfer 算子库时遇到了未完全支持的实现，可以调整 **target** 参数中的选项，运行不使用 ixInfer 算子库的引擎文件来进一步确认问题：

```
target = tvm.target.iluvatar(options="-libs=cudnn,cublas")
```

确认为 ixInfer 问题后，在 IGIE 层面还可以进行如下尝试：

- 尝试切换 ixInfer 算法实现：ixInfer 提供了两种算法选择，在调用 ixInfer INT8 算子时，IGIE 默认选择算法 0 的实现。将 python/tvm/contrib/ixinfer.py 脚本下 qdconv_forward 和 qdeconv_forward 函数中的 algo 设置为 1，可快速切换到 ixInfer 的另一种算法实现。另外，切换算法实现后，需要重新编译生成引擎文件。
- 尝试对模型进行 Pad：当模型中的 OP 输入、输出 Channel 为 64 的倍数时，ixInfer 在精度和性能上往往会有较好的表现。您可以调用 IGIE PadNet 接口对整个模型进行 64 倍数的 Pad，接口调用非常简单，在构建引擎文件时额外传参即可：

```
engine = relay.build(mod, target=target, params=params,
    ↪ required_pass=['MutatePadMod'])
```

该功能也包含在 igie-exec 中，您只需额外输入 **--custom_option required_pass:MutatePadMod** 参数即可开启。下面以 YOLOv5 模型为例：

```
$ igie-exec --model_path yolov5m.onnx --input images:32,3,640,640 --precision
↪ int8 --use_coco2017 True --automatic_yolo_quantization True --custom_option
↪ required_pass:MutatePadMod
```

问题 9: IGIE 推理出现精度问题, 如何快速定位解决?

下面按照推理的精度类型分别讨论。

FP16 推理

FP16 推理不涉及到模型量化, 出现精度问题的可能性比较小。如果出现精度问题, 可以遵循以下步骤分析定位问题:

1. 使用模型原生框架, 确认模型预处理、后处理计算无误;
2. 逐步剥离算子库支持, 通过修改 **target** 依次回退, 确认是否为算子库计算错误。

INT8 推理

INT8 推理首先需要排除模型量化本身出现错误的可能:

1. 同样需要确定模型预处理、后处理计算无误, 尤其是预处理。如果模型预处理错误, 量化会带来极大的精度损失。
2. IGIE 目前推荐使用 ONNXRuntime 量化, 量化后的模型可以使用 ONNXRuntime 进行精度验证。

如果量化后的模型本身有精度损失, 这个在[问题 10](#)中单独讨论。确认量化后的模型精度无误, 导入 IGIE 后精度错误, 一般也可以通过逐步剥离算法库支持来分析定位问题。总之, IGIE 在 **iluvatar target** 会有最佳的精度表现, 可以优先尝试修改 **target**。

假如您已经通过上述步骤, 定位到了问题出现在 ixInfer 算法库上, 那么同样可以参考[问题 8](#)中提到的两种方法尝试解决:

1. 切换 ixInfer 的算法实现
2. 调用 IGIE PadNet 接口

问题 10: 如何解决 INT8 量化本身带来的精度损失?

关于 IGIE INT8 量化的介绍, 可以参考“快速上手: IGIE 推理框架使用教程”>“教程 2: 如何对模型进行量化”。目前 IGIE 更推荐使用 ONNXRuntime 进行量化, 并提供以下配置选项控制量化行为:

- **op_types_to_quantize**: 指定要量化的 OP 类型, 例如 ['Conv'] 仅量化 Conv OP
- **nodes_to_exclude**: 指定需要跳过量化的 Node List, 设置该参数后, List 中的 Node 会跳过量化, 例如 ['Conv_0', 'Conv_1']
- **per_channel**: 是否开启 per-channel 量化选项, 相比 per-tensor, per-channel 一般能获得更好的精度

下面列出一些可能会引起量化精度损失的情况和解决方法:

序号	问题描述	解决方法
1	模型中存在 GroupConv、DepthWiseConv 量化带来精度损失	设置 per_channel=True, 使用 per-channel 量化

2	YOLO 系列模型后处理部分量化后，带来精度损失	将后处理部分的相关 Node 添加到 nodes_to_exclude 中
3	不清楚哪些层可能会带来精度损失	极端情况下，可以只量化 Conv

另外，igie-exec 提供了 **--automatic_yolo_quantization** 参数，用于自动跳过 YOLO 后处理的部分 OP。

问题 11: IGIE 是否支持非官方模型?

目前 IGIE ModelZoo 已经支持 120+ 个模型，CV 类包括但不限于：torchvision、MMClassification、MMDetection 的大多数模型，并且在持续扩充中。

在适配过程中，IGIE 展现了较强的通用性，即使模型并非官方模型，相信 IGIE 也可以覆盖。唯一需要注意的是，当模型中存在 Dynamic Input/Output 时，IGIE 的支持还不是那么完善。

另外，您也可以使用 igie-exec 工具来快速验证模型的支持情况，详情请参考“igie-exec 模型快速部署和验证工具使用指南”。

问题 12: 如何解决 ONNX infer shape 报错?

首先解释下为何会出现该报错：igie-exec 内置自动修改模型 Batch 的功能，但是并非支持 ONNX 模型中的所有 OP。因此，修改后的模型在调用 ONNX shape inference 接口时可能会出现 [ShapeInferenceError] Inferred shape and existing shape differ in dimension 0: (64) vs (2)。

由于 ONNX shape inference 仅用作模型中间层 Tensor Shape 的推断，不是模型推理的必要执行条件，并且 IGIE 自身存在 Shape 推断逻辑，通常注释掉该接口调用并不影响模型推理，可以注释掉 rewrite_batch_size 函数中的 onnx.shape_inference.infer_shapes 调用。

另外，igie-exec 自动 Batch 修改并非适用于所有模型，如果 ONNX 模型来自于 PyTorch 框架，您可以尝试指定 Batch 或将 Batch 维度设置为 dynamic_axes。

问题 13: IGIE 能否提供 Dynamic Shape 支持?

在 Transformer 模型上，往往会遇到 Dynamic Shape 的情况，目前 IGIE 提供了一定范围的 Dynamic Shape 支持。具体的做法是：将 Dynamic Shape 的范围分档，编译多个引擎文件，对输入进行 Pad 以匹配对应的引擎文件，实现一定程度的支持。

问题 14: 在 ARM 设备上编译模型出现 Compilation Error，如何解决?

受 GCC 限制，目前在 ARM 设备上编译和导出超过 2GB 的模型时，会出现 Compilation Error。您可以通过以下方式解决该问题：

1. 将 engine 和 params 分开导出：

```
# Export model
graph, lib, params = relay.build(mod, target, params=params, precision="fp32")

lib.export_library("deploy_lib.so")

with open("deploy_graph.json", "w") as fo:
    fo.write(graph)

with open("deploy_param.params", "wb") as fo:
    fo.write(relay.save_param_dict(params))
```

2. 加载导出的 engine 和 params:

```
# Load model
lib = tvn.runtime.load_module("deploy_lib.so")
graph = open("deploy_graph.json", "rb").read()
params = bytearray(open("deploy_param.params", "rb").read())

dev = tvn.device("iluvatar", 0)
module = graph_executor.create(graph, lib, dev)
module.load_params(params)
module.run()
```

3. 使用 C++ 接口加载导出的 engine 和 params:

```
// Load lib
tvm::runtime::Module lib = tvm::runtime::Module::LoadFromFile(module_file);

// Parser graph json
std::ifstream json_in(json_file.c_str());
if(json_in.fail())
{
    throw std::runtime_error("could not open json file");
}

const std::string json_data((std::istreambuf_iterator<char>(json_in)),
                             std::istreambuf_iterator<char>());
json_in.close();

// Parser params
std::ifstream params_in(params_file.c_str(), std::ios::binary);
if(params_in.fail())
{
    throw std::runtime_error("could not open params file");
}

const std::string params_data((std::istreambuf_iterator<char>(params_in)),
                               std::istreambuf_iterator<char>());
params_in.close();

TVMByteArray params_arr;
params_arr.data = params_data.c_str();
params_arr.size = params_data.length();

// Create Module
DLDevice dev{kDLILUVATAR, 0};
const auto runtime_create =
    ↪ *tvm::runtime::Registry::Get("tvm.graph_executor.create");
tvm::runtime::Module gmod = runtime_create(json_data, lib,
    ↪ static_cast<int>(dev.device_type), dev.device_id);

// Load params
tvm::runtime::PackedFunc load_params = gmod.GetFunction("load_params");
load_params(params_arr);
```

11 附录：IGIE 框架算子支持列表

11.1 IGIE 算子支持 - Relay 算子

算子	备注
abs	Get absolute value of the input element-wise.
acos	Take acos of input x.
acosh	Take acos of input x.
adaptive_avg_pool1d	1D adaptive average pooling operator.
adaptive_avg_pool2d	2D adaptive average pooling operator.
adaptive_avg_pool3d	3D adaptive avg pooling operator.
adaptive_max_pool1d	1D adaptive max pooling operator.
adaptive_max_pool2d	2D adaptive max pooling operator.
adaptive_max_pool3d	3D adaptive max pooling operator.
add	Generic add operator.
adv_index	Numpy style advanced indexing.
affine_grid	affine_grid operator that generates 2D sampling grid.
all	Computes the logical AND of boolean array elements over given axes.
all_class_non_max_suppression	Non-maximum suppression operator for object detection, corresponding to ONNX NonMaxSuppression and TensorFlow combined_non_max_suppression.
any	Computes the logical OR of boolean array elements over given axes.
arange	Return evenly spaced values within a given interval.
argmax	Returns the indices of the maximum values along an axis.
argmin	Returns the indices of the minimum values along an axis.
argsort	Performs sorting along the given axis and returns an array of indices having same shape as an input array that index data in sorted order.
argwhere	Find the indices of elements of a tensor that are non-zero.

算子	备注
asin	Compute elementwise asin of data.
asinh	Take asinh of input x.
atan	Take atan of input x.
atanh	Take atanh of input x.
avg_pool1d	1D average pooling operator.
avg_pool2d	2D average pooling operator.
avg_pool2d_grad	Gradient of 2D average pooling operator.
avg_pool3d	3D average pooling operator.
batch_flatten	BatchFlatten.
batch_matmul	Compute batch matrix multiplication of tensor_a and tensor_b.
batch_norm	Batch normalization layer .
batch_to_space_nd	Reshape the batch dimension into spatial dimensions.
bias_add	add_bias operator.
bind	Bind an free variables in expr or function arguments.
bitpack	Tensor packing for bitserial operations.
bitserial_conv2d	2D convolution using bitserial computation.
bitserial_dense	Bitserial Dense operator.
bitwise_and	bitwise AND with numpy-style broadcasting.
bitwise_not	Compute element-wise bitwise not of data.
bitwise_or	bitwise OR with numpy-style broadcasting.
bitwise_xor	bitwise XOR with numpy-style broadcasting.
broadcast_to	Return a scalar value array with the same type, broadcasted to the provided shape.
broadcast_to_like	Return a scalar value array with the same shape and type as the input array.
cast	Cast input tensor to data type.
cast_like	Cast input tensor to data type of another tensor.
ceil	Compute element-wise ceil of data.
clip	Clip the elements in a between a_min and a_max.

算子	备注
<code>collapse_sum_like</code>	Return a scalar value array with the same shape and type as the input array.
<code>collapse_sum_to</code>	Return a summation of data to the specified shape.
<code>comm_reducer</code>	Create a commutative reducer for reduction.
<code>concatenate</code>	Concatenate the input tensors along the given axis.
<code>const</code>	Create a new constant with specified name and dtype
<code>conv1d</code>	1D convolution.
<code>conv1d_transpose</code>	One dimensional transposed convolution operator.
<code>conv2d</code>	2D convolution.
<code>conv2d_backward_weight</code>	The gradient of conv2d with respect to weight.
<code>conv2d_transpose</code>	Two dimensional transposed convolution operator.
<code>conv3d</code>	3D convolution.
<code>conv3d_transpose</code>	3D transpose convolution.
<code>copy</code>	Copy a tensor.
<code>copy_shape_func</code>	Shape function for copy op.
<code>correlation</code>	Applies correlation to inputs.
<code>cos</code>	Take cos of input x.
<code>cosh</code>	Take cosh of input x.
<code>contrib_conv2d_gemm_weight_transform</code>	Weight Transformation part for 2D convolution with gemm algorithm.
<code>contrib_conv2d_gemm_without_weight_transform</code>	2D convolution with gemm algorithm.
<code>contrib_conv2d_nchw</code>	Variant of 2D convolution.
<code>contrib_conv2d_winograd_nnpack_weight_transform</code>	Weight Transformation part for 2D convolution with winograd algorithm.
<code>contrib_conv2d_winograd_weight_transform</code>	Weight Transformation part for 2D convolution with winograd algorithm.
<code>contrib_conv2d_winograd_without_weight_transform</code>	2D convolution with winograd algorithm.
<code>contrib_conv3d_winograd_weight_transform</code>	Weight Transformation part for 3D convolution with winograd algorithm.

算子	备注
contrib_conv3d_winograd_without_weight_transform	3D convolution with winograd algorithm.
contrib_dense_pack	Dense operator.
contrib_depthwise_conv2d_nchw	Variant of 2D depthwise convolution.
crop_and_resize	Crop input images and resize them.
cross_entropy	CrossEntropy without logits.
cross_entropy_with_logits	CrossEntropy with logits.
cumprod	Numpy style cumprod op.
cumsum	Numpy style cumsum op.
decl_tensor_intrin	Declare a tensor intrinsic function.
deformable_conv2d	Deformable 2d convolution.
dense	Dense operator.
depth_to_space	Convert channels into spatial blocks.
dilate	Dilate data with given dilation value .
dilation2d	Morphological Dilation 2D.
div	Compute a / b as in C/C++ semantics.
divide	Division with numpy-style broadcasting.
dropout	Applies the dropout operation to the input array.
dropout_raw	Applies the dropout operation to the input array.
einsum	Evaluates the Einstein summation convention on data
equal	Broadcasted elementwise test for (lhs == rhs).
erf	Take gauss error function of the input x.
exp	Take exponential of input x.
expand_dims	Insert num_newaxis axes at the position given by axis.
extern	Compute several tensors via an extern function.
extern_primfunc	Compute tensors via a schedulable TIR PrimFunc
fast_softmax	Computes softmax.
fifo_buffer	FIFO buffer to enable computation reuse in CNNs with sliding indow input

算子	备注
fixed_point_multiply	Fixed point multiplication between data and a fixed point constant expressed as multiplier * 2 ^(-shift) , where multiplier is a Q-number with 31 fractional bits
floor	Take floor of float input x.
floor_divide	Floor division with numpy-style broadcasting.
floor_mod	Floor mod with numpy-style broadcasting.
fmod	Return the remainder of x divided by y with the same sign as x.
full	Fill array with scalar value.
full_like	Return a scalar value array with the same shape and type as the input array.
gather	Gather values along given axis from given indices.
gather_nd	Gather elements or slices from data and store them to a tensor whose shape is defined by indices.
get_pad_tuple1d	Common code to get the 1 dimensional pad option :param padding: Padding size :type padding: Union[int, Tuple[int, ...]]
get_pad_tuple2d	Common code to get the pad option :param padding: Padding size :type padding: Union[int, Tuple[int, ...]]
get_pad_tuple3d	Common code to get the pad option :param padding: Padding size :type padding: Union[int, Tuple[int, ...]]
get_valid_counts	Get valid count of bounding boxes given a score threshold.
global_avg_pool1d	1D global average pooling operator.
global_avg_pool2d	2D global average pooling operator.
global_avg_pool3d	3D global average pooling operator.
global_max_pool1d	1D global maximum pooling operator.
global_max_pool2d	2D global maximum pooling operator.
global_max_pool3d	3D global maximum pooling operator.
gradient	Perform reverse-mode automatic differentiation.
greater	Broadcasted elementwise test for (lhs > rhs).
greater_equal	Broadcasted elementwise test for (lhs >= rhs).
grid_sample	Applies grid sampling to input feature map.

算子	备注
group_norm	Group normalization normalizes over group of channels for each training examples.
if_then_else	Conditional selection expression.
indexdiv	Compute floor(a / b) where a and b are non-negative.
indexmod	Compute the remainder of indexdiv.
instance_norm	Instance Normalization Applies instance normalization to the n-dimensional input array.
invert_permutation	Computes the inverse permutation of data.
isfinite	Check if input value is finite.
isinf	Check if input value is infinite.
isnan	Check if input value is Nan.
l2_normalize	Perform L2 normalization on the input data
layer_norm	Layer normalization .
layout_transform	Transform the layout of a tensor.
leaky_relu	This operator takes data as input and does Leaky version of a Rectified Linear Unit.
left_shift	Left shift with numpy-style broadcasting.
less	Broadcasted elementwise test for ($lhs < rhs$).
less_equal	Broadcasted elementwise test for ($lhs \leq rhs$).
load_param_dict	Load parameter dictionary to binary bytes.
log	Take log of input x.
log_softmax	Computes log softmax.
log10	Take log10 of input x.
log2	Take log2 of input x.
logical_and	logical AND with numpy-style broadcasting.
logical_not	Compute element-wise logical not of data.
logical_or	logical OR with numpy-style broadcasting.
logical_xor	logical XOR with numpy-style broadcasting.
logsumexp	Compute the log of the sum of exponentials of input elements over given axes.

算子	备注
lrn	This operator takes data as input and does local response normalization.
matmul	Matmul operator.
max	Create a max expression over axis.
max_pool1d	1D maximum pooling operator.
max_pool2d	2D maximum pooling operator.
max_pool2d_grad	Gradient of 2D maximum pooling operator.
max_pool3d	3D maximum pooling operator.
max_value	maximum value of dtype
maximum	Maximum with numpy-style broadcasting.
mean	Computes the mean of array elements over given axes.
mean_std	Computes the mean and standard deviation of data over given axes.
mean_variance	Computes the mean and variance of data over given axes.
meshgrid	Create coordinate matrices from coordinate vectors.
min	Create a min expression over axis.
min_value	minimum value of dtype
mirror_pad	MirrorPadding
mod	Mod with numpy-style broadcasting.
multibox_prior	Generate prior(anchor) boxes from data, sizes and ratios.
multibox_transform_loc	Location transformation for multibox detection
multiply	Generic multiply operator.
nearbyint	Round elements of the array to the nearest integer.
negative	Compute element-wise negative of data.
nll_loss	Negative log likelihood loss.
non_max_suppression	Non-maximum suppression operator for object detection.
not_equal	Broadcasted elementwise test for (lhs != rhs).

算子	备注
one_hot	Returns a one-hot tensor where the locations represented by indices take value on_value, and other locations take value off_value.
ones	Fill array with ones.
ones_like	Returns an array of ones, with same type and shape as the input.
pad	Padding
placeholder	Construct an empty tensor object.
popcount	Count the number of set bits in input x.
power	x power y
prelu	This operator takes data as input and does Leaky version of a Rectified Linear Unit.
prod	Computes the products of array elements over given axes.
proposal	Proposal operator.
reduce_axis	Create a new IterVar for reduction.
reinterpret	Reinterpret input tensor to data type.
relu	Rectified linear unit.
repeat	Repeats elements of an array.
reshape	Reshape the input array.
reshape_like	Reshapes the input tensor by the size of another tensor.
resize1d	Image resize1d operator.
resize2d	Image resize2d operator.
resize3d	Image resize3d operator.
reverse	Reverses the order of elements along given axis while preserving array shape.
reverse_reshape	Reshapes the input array where the special values are inferred from right to left.
reverse_sequence	Reverse the tensor for variable length slices.
right_shift	Right shift with numpy-style broadcasting.
roi_align	ROI align operator.
roi_pool	ROI pool operator.

算子	备注
round	Round elements of the array to the nearest integer.
rsqrt	Take reciprocal of square root of input x.
scan	Construct new tensors by scanning over axis.
scatter	Update data at positions defined by indices with values in updates.
scatter_add	Update data by adding values in updates at positions defined by indices.
scatter_nd	Scatter values from an array and update.
searchsorted	Find indices where elements should be inserted to maintain order.
segment_sum	Computes the sum along segment_ids along axis 0.
sequence_mask	Sets all elements outside the expected length of the sequence to a constant value.
shape_of	Get shape of a tensor.
ShapeVar	A helper which constructs a type var of which the shape kind.
sigmoid	Quick function to get sigmoid
sign	Compute element-wise absolute of data.
sin	Take sin of input x.
sinh	Take sinh of input x.
size_var	Create a new variable represents a tensor shape size, which is non-negative.
slice_like	Slice the first input with respect to the second input.
sliding_window	Slide a window over the data tensor.
softmax	Computes softmax.
sort	Performs sorting along the given axis and returns data in sorted order.
space_to_batch_nd	Divide spatial dimensions of the data into a grid of blocks and interleave them into batch dim.
space_to_depth	Convert spatial blocks into channels.

算子	备注
sparse_add	Computes the matrix addition of dense_mat and sparse_mat, where dense_mat is a dense matrix and sparse_mat is a sparse namedtuple with fields data, indices, and indptr.
sparse_dense	Computes the matrix multiplication of dense_mat and sparse_mat, where dense_mat is a dense matrix and sparse_mat is a sparse namedtuple with fields data, indices, and indptr.
sparse_fill_empty_rows	Fill rows in a sparse matrix that do not contain any values.
sparse_reshape	Reshape a sparse tensor.
sparse_to_dense	Converts a sparse representation into a dense tensor.
sparse_transpose	Computes the fast matrix transpose of x, where x is a sparse tensor in CSR format .
split	Split input tensor along axis by sections or indices.
sqrt	Take square root of input x.
squeeze	Squeeze axes in the array.
stack	Join a sequence of arrays along a new axis.
std	Computes the standard deviation of data over given axes.
stft	The STFT computes the Fourier transform of short overlapping windows of the input.
strided_set	Strided set of an array.
strided_slice	Strided slice of an array.
subtract	Generic subtract operator.
sum	Create a sum expression over axis.
tag_scope	The operator tag scope.
take	Take elements from an array along an axis.
tan	Take tan of input x.
tanh	Take hyperbolic tanh of input x.
tile	Repeats the whole array multiple times.
topk	Get the top k elements in an input tensor along the given axis.

算子	备注
trace	Trace tensor data at the runtime.
transpose	Permutates the dimensions of an array.
trilu	Given a 2-D matrix or batches of 2-D matrices, returns the upper or lower triangular part of the tensor.
trunc	Get truncated value of the input.
trunc_divide	Trunc division with numpy-style broadcasting.
trunc_mod	Trunc mod with numpy-style broadcasting.
truncdiv	Compute the truncdiv of two expressions.
truncmod	Compute the truncmod of two expressions.
unique	Find the unique elements of a 1-D tensor.
unravel_index	Convert a flat index or array of flat indices into a tuple of coordinate arrays.
upsampling	Upsampling.
upsampling3d	3D Upsampling.
var	Create a new tvm.relay.Var.
where	Selecting elements from either x or y depending on the value of the condition.
yolo_reorg	Yolo reorg operation used in darknet models.
zeros	Fill array with zeros.
zeros_like	Returns an array of zeros, with same type and shape as the input.
regular_non_max_suppression	Regular non-maximum suppression operator for object detection.
dft	Discrete Fourier transform.
matrix_set_diag	Returns a tensor with the diagonals of input tensor replaced with the provided diagonal values.
threefry_key	Create a new Threefry random number generator key.
threefry_generate	Generate an array of random bits (uint64) using the Threefry algorithm.
threefry_split	Split an existing Threefry key into two new ones.

11.2 IGIE 算子支持 - Relax 算子

Relax 算子在接口命令上更为规范，向 Python API 看齐，您可以参考 [API specification --- Python array API standard 2023.12 documentation \(data-apix.org\)](#) 进一步了解 Relax 接口。

在 Python array API 标准手册中，把一些操作按大类进行划分，这些划分方法在各热门框架 (如 Numpy, TensorFlow, PyTorch 等等) 中是一致的。

以线性代数为例，抽象出如下 4 个接口：

- `matmul (x1, x2, /)`: Computes the matrix product.
- `matrix_transpose(x, /)`: Transposes a matrix (or a stack of matrices) x.
- `tensordot (x1, x2, /, *, axes=2)`: Returns a tensor contraction of x1 and x2 over specific axes.
- `vecdot (x1, x2, /, *, axis=-1)`: Computes the (vector) dot product of two arrays.

详情请参考 [Linear Algebra Functions --- Python array API standard 2023.12 documentation \(data-apix.org\)](#)
<https://data-apix.org/array-api/latest/API_specification/linear_algebra_functions.html>。

11.3 框架模型算子支持

IGIE 支持主流框架模型直接导入。

IGIE 支持的主流框架模型包括：

框架模型	使用接口	说明
ONNX	<code>relay.frontend.from_onnx()</code>	导入 ONNX 模型，转换为 IGIE relay Op
PyTorch	<code>relay.frontend.from_pytorch()</code>	导入 PyTorch 模型
TensorFlow	<code>relay.frontend.from_tensorflow()</code>	导入 TensorFlow 模型
PaddlePaddle	<code>relay.frontend.from_paddle()</code>	导入 Paddle 模型
TFlite	<code>relay.frontend.from_tflite()</code>	导入 TFlite 模型
MXNet	<code>relay.frontend.from_mxnet()</code>	导入 MXNet 模型
Caffe	<code>relay.frontend.from_caffe()</code>	导入 Caffe 模型
Keras	<code>relay.frontend.from_keras()</code>	导入 Keras 模型
Darknet	<code>relay.frontend.from_darknet()</code>	导入 Darknet 模型

Tip

relay 是 IGIE graph 和 op 的集合，所有算子的定义都在 relay package 下面，因此 igie 的算子有时候也会称为 relay op。

11.3.1 ONNX 模型算子的支持

Identity
Optional
OptionalHasElement
OptionalGetElement
Affine
BitShift
BitwiseAnd
BitwiseNot
BitwiseOr
BitwiseXor
ThresholdedRelu
ScaledTanh
ParametricSoftplus
Constant
ConstantOfShape
FC
Scale
MeanVarianceNormalization
Upsample
SpatialBN
Add
Sub
Mul
Div
Neg
Abs
Reciprocal
Floor
Ceil
Round
IsInf
IsNaN
Sqrt
Relu
Celu
LeakyRelu
Selu
Elu
Gelu
FastGelu
BiasGelu
Mish
LayerNormalization
EmbedLayerNormalization
SkipLayerNormalization
Attention
QAttention

Exp
Greater
GreaterOrEqual
Less
LessOrEqual
Log
Acos
Acosh
Asin
Asinh
Atan
Atanh
Cos
Cosh
Sin
Sinh
Tan
Tanh
Pow
PRelu
Sigmoid
HardSigmoid
HardSwish
Max
Min
Sum
Mean
Clip
Softplus
Softmax
LogSoftmax
OneHot
Hardmax
Shrink
Softsign
Gemm
MatMul
MatMulInteger
MatMulInteger16
Mod
Xor
AveragePool
LpPool
GlobalLpPool
MaxPool
MaxUnpool
Conv
ConvTranspose
GlobalAveragePool
GlobalMaxPool

BatchNormalization
InstanceNormalization
Dropout
Flatten
LRN
RNN
LSTM
GRU
MaxRoiPool
RoiAlign
NonMaxSuppression
ReduceMax
ReduceMin
ReduceSum
ReduceMean
ReduceProd
ReduceLogSumExp
ReduceLogSum
ReduceSumSquare
ReduceL1
ReduceL2
ArgMax
ArgMin
TopK
Cast
CastLike
Reshape
Expand
Concat
Split
Slice
Transpose
DepthToSpace
SpaceToDepth
Gather
GatherElements
GatherND
Compress
Size
Scatter
ScatterElements
ScatterND
EyeLike
Squeeze
Unsqueeze
Pad
Shape
Sign
Equal
Not

And
Tile
Erf
Where
Or
Resize
NonZero
Range
CumSum
Unique
Einsum
Trilu
GridSample
Loop
If
ATen
QuantizeLinear
DequantizeLinear
DynamicQuantizeLinear
ReverseSequence
QGemm
QLinearConv
QLinearConcat
QLinearAdd
QLinearMatMul
QLinearMul
QLinearSigmoid
QLinearSoftmax
ConvInteger
QLinearAveragePool
QLinearGlobalAveragePool
QLinearLeakyRelu
Bernoulli
RandomNormal
RandomNormalLike
RandomUniform
RandomUniformLike
Multinomial
NegativeLogLikelihoodLoss
SoftmaxCrossEntropyLoss
Adagrad
Adam
Momentum
Scan
LinearRegressor
DFT
SequenceConstruct
SequenceEmpty
SequenceErase
SequenceInsert

SequenceLength
ConcatFromSequence
SplitToSequence
SequenceAt

11.3.2 PyTorch 模型算子支持

aten::is_floating_point
aten::pixel_shuffle
aten::device
aten::sub
aten::max
aten::min
aten::maximum
aten::minimum
aten::amax
aten::amin
aten::stft
aten::mul
aten::pow
aten::lerp
aten::arange
aten::meshgrid
aten::div
aten::floor_divide
aten::true_divide
aten::fmod
aten::remainder
aten::addcdiv
aten::addcmul
aten::ones
aten::ones_like
aten::zeros
aten::zero_
aten::zeros_like
aten::new_zeros
aten::new_ones
aten::full
aten::full_like
aten::new_full
aten::fill_
aten::linspace
aten::reciprocal
aten::repeat
aten::repeat_interleave
aten::to
aten::squeeze

```
aten::unsqueeze
aten::cat
aten::concat
aten::slice
aten::narrow
aten::split
aten::tensor_split
aten::split_with_sizes
aten::select
aten::take
aten::where
aten::topk
aten::relu
aten::relu6
aten::prelu
aten::leaky_relu
aten::elu
aten::celu
aten::gelu
aten::selu
aten::silu
aten::glu
aten::log_sigmoid
aten::adaptive_avg_pool1d
aten::adaptive_avg_pool2d
aten::adaptive_avg_pool3d
aten::adaptive_max_pool1d
aten::adaptive_max_pool2d
aten::adaptive_max_pool3d
aten::max_pool2d
aten::max_pool2d_with_indices
aten::max_pool1d
aten::max_pool3d
aten::hardtanh
aten::_convolution
aten::softmax
aten::threshold
aten::contiguous
aten::batch_norm
aten::instance_norm
aten::layer_norm
aten::group_norm
aten::transpose
aten::t
aten::numpy_T
aten::flatten
aten::unflatten
aten::addmm
aten::size
aten::view
```

```
aten::view_as
aten::reshape
aten::reshape_as
aten::clone
aten::log_softmax
aten::sigmoid
aten::softplus
aten::avg_pool1d
aten::avg_pool2d
aten::avg_pool3d
aten::linear
aten::dropout
aten::feature_dropout
aten::alpha_dropout
aten::mean
aten::chunk
aten::unsafe_chunk
aten::matmul
aten::bmm
aten::baddbmm
aten::expand
aten::Int
aten::ScalarImplicit
aten::pad
aten::constant_pad_nd
aten::reflection_pad1d
aten::reflection_pad2d
aten::replication_pad1d
aten::replication_pad2d
aten::replication_pad3d
aten::permute
aten::sum
aten::prod
aten::argmin
aten::argmax
aten::norm
aten::frobenius_norm
aten::std
aten::var
aten::var_mean
aten::abs
aten::neg
aten::cos
aten::cosh
aten::sin
aten::sinh
aten::tan
aten::tanh
aten::acos
aten::asin
```

```
aten::atan
aten::log
aten::log2
aten::log10
aten::log1p
aten::exp
aten::erf
aten::trunc
aten::sign
aten::sqrt
aten::rsqrt
aten::square
aten::tril
aten::triu
aten::ceil
aten::floor
aten::round
aten::isfinite
aten::isinf
aten::isnan
aten::clamp
aten::clamp_min
aten::clamp_max
aten::detach
aten::upsample_bilinear2d
aten::upsample_bicubic2d
aten::upsample_nearest2d
aten::upsample_trilinear3d
aten::upsample_nearest3d
aten::expand_as
aten::broadcast_tensors
aten::broadcast_to
aten::lt
aten::gt
aten::le
aten::ge
aten::ne
aten::eq
aten::logical_not
aten::logical_xor
aten::bitwise_not
aten::bitwise_xor
aten::bitwise_and
aten::Bool
aten::Float
aten::rsub
aten::embedding
aten::one_hot
aten::mm
aten::add
```

```
aten::stack
aten::getitem
aten::len
aten::type_as
aten::gather
aten::index_select
aten::index
aten::logsumexp
aten::unbind
aten::and
aten::logical_and
aten::logical_or
aten::_shape_as_tensor
aten::nonzero
aten::nonzero_numpy
aten::scatter
aten::scatter_add
aten::scatter_reduce
aten::index_put
aten::scalar_tensor
aten::__interpolate
aten::IntImplicit
aten::tensor
aten::numel
aten::empty
aten::empty_like
aten::new_empty
aten::randn
aten::bincount
aten::not
aten::hardswish
aten::hardsigmoid
aten::cumsum
aten::masked_fill
aten::masked_select
aten::argsort
aten::sort
aten::_unique2
aten::nll_loss
aten::nll_loss2d
aten::nll_loss_nd
aten::cross_entropy_loss
aten::l1_loss
aten::mse_loss
aten::flip
aten::rnn_tanh
aten::rnn_relu
aten::gru
aten::lstm
aten::all
```

```
aten::any
aten::searchsorted
aten::bucketize
aten::roll
aten::einsum
aten::dot
aten::mv
aten::grid_sampler
aten::ior
aten::iand
aten::ixor
aten::lshift
aten::rshift
aten::multinomial
aten::_weight_norm
aten::copy_
aten::swapaxes
aten::linalg_vector_norm
aten::scaled_dot_product_attention
aten::tile
```

11.3.3 TensorFlow 模型算子支持

```
Abs
Acos
Acosh
Add
AddN
AddV2
All
Any
ArgMax
ArgMin
Asin
Asinh
Assert
Atan
Atanh
Atan2
AvgPool
AvgPool3D
BatchMatMul
BatchMatMulV2
BatchNormWithGlobalNormalization
BatchToSpaceND
BiasAdd
Bincount
```

BroadcastTo
BroadcastArgs
Cast
Ceil
CheckNumerics
ClipByValue
Concat
ConcatV2
Conv2D
Conv2DBackpropInput
Conv3D
Conv3DBackpropInputV2
Cos
Cosh
CropAndResize
DecodeJpeg
DenseBincount
DepthToSpace
DepthwiseConv2dNative
Dilation2D
Einsum
Elu
Equal
Erf
EuclideanNorm
Exp
ExpandDims
Expm1
Fill
Floor
FloorDiv
FloorMod
FusedBatchNorm
FusedBatchNormV2
FusedBatchNormV3
Gather
GatherNd
GatherV2
Greater
GreaterEqual
Identity
IdentityN
InvertPermutation
IsFinite
IsInf
IsNan
LeakyRelu
LeftShift
Less
LessEqual

Log
Log1p
LogicalAnd
LogicalNot
LogicalOr
LogSoftmax
LRN
LSTMBlockCell
MatMul
Max
Maximum
MaxPool
MaxPool3D
Mean
Min
Minimum
MirrorPad
Mod
Mul
Neg
NonMaxSuppressionV2
NonMaxSuppressionV3
NonMaxSuppressionV4
NonMaxSuppressionV5
CombinedNonMaxSuppression
NoOp
NotEqual
OneHot
Pack
Pad
PadV2
Pow
Prod
Range
Rank
RealDiv
Relu
Relu6
Reshape
ResizeBicubic
ResizeBilinear
ResizeNearestNeighbor
ReverseV2
RightShift
Rint
Round
Rsqrt
Select
SelectV2
Selu

Shape
Sigmoid
Sign
Sin
Sinh
Size
Slice
Softmax
Softplus
Softsign
SpaceToBatchND
SpaceToDepth
SparseToDense
SparseTensorDenseMatMul
SparseFillEmptyRows
SparseReshape
SegmentSum
SparseSegmentSum
SparseSegmentSumWithNumSegments
SparseSegmentSqrtN
SparseSegmentSqrtNWithNumSegments
SparseSegmentMean
SparseSegmentMeanWithNumSegments
SparseTensorDenseAdd
Split
SplitV
Sqrt
Square
SquaredDifference
Squeeze
StopGradient
StridedSlice
Sub
Sum
Tan
Tanh
TensorArrayConcatV3
TensorArrayGatherV3
TensorArrayReadV3
TensorArrayScatterV3
TensorArraySizeV3
TensorArraySplitV3
TensorArrayV3
TensorArrayWriteV3
Tile
TopKV2
Transpose
TruncateMod
Unique
UniqueWithCounts

Unpack
UnravelIndex
Where
ZerosLike

11.3.4 Paddle 模型算子支持

abs
acos
addmm
arg_max
arg_min
argsort
asin
assign
assign_value
atan
batch_norm
bicubic_interp_v2
bilinear_interp_v2
bmm
brelu
cast
ceil
clip
concat
conv2d
conv2d_transpose
conv3d
cos
cosh
cumsum
depthwise_conv2d
depthwise_conv2d_transpose
dist
dot
dropout
elementwise_add
elementwise_div
elementwise_floordiv
elementwise_max
elementwise_min
elementwise_mod
elementwise_mul
elementwise_pow
elementwise_prod
elementwise_sub

elu
equal
erf
exp
expand_v2
expand_as_v2
eye
feed
fill_any_like
fill_constant
fill_constant_batch_size_like
fill_zeros_like
flatten_contiguous_range
floor
floor_mod
flip
gather
gather_nd
gaussian_random
gelu
greater_equal
greater_than
grid_sampler
group_norm
hard_shrink
hard_sigmoid
hard_swish
index_select
instance_norm
isfinite_v2
isinf_v2
isnan_v2
layer_norm
leaky_relu
less_equal
less_than
linspace
log
log2
log10
log1p
logical_and
logical_not
logical_or
logical_xor
logsigmoid
log_softmax
logsumexp
lookup_table_v2
matmul

matmul_v2
meshgrid
mish
mul
mv
nearest_interp_v2
norm
not_equal
one_hot_v2
p_norm
pad1d
pad2d
pad3d
pixel_shuffle
pool2d
pool3d
pow
prelu
range
relu
relu6
reshape2
round
roi_align
reciprocal
reduce_all
reduce_any
reduce_max
reduce_min
reduce_prod
reduce_sum
reduce_mean
rnn
rsqrt
scale
scatter
scatter_nd_add
selu
set_value
shape
sigmoid
sign
silu
sin
sinh
size
slice
softmax
softmax_with_cross_entropy
softplus

```
softsign  
softshrink  
split  
stack  
strided_slice  
sqrt  
square  
squeeze2  
swish  
take_along_axis  
tan  
tanh  
tanh_shrink  
top_k  
thresholded_relu  
tile  
top_k_v2  
transpose2  
unique  
unsqueeze2  
unstack  
where  
where_index  
dequantize_linear  
quantize_linear
```

11.3.5 MXNet 模型算子支持

```
_copy  
relu  
broadcast_add  
broadcast_plus  
broadcast_sub  
broadcast_minus  
broadcast_mul  
broadcast_div  
broadcast_mod  
broadcast_maximum  
broadcast_minimum  
broadcast_power  
arccos  
arcsin  
arctan  
arccosh  
arcsinh  
arctanh  
broadcast_equal
```

```
broadcast_not_equal
broadcast_greater
broadcast_greater_equal
broadcast_lesser
broadcast_lesser_equal
broadcast_logical_or
broadcast_logical_and
broadcast_logical_xor
broadcast_to
broadcast_like
logical_not
_equal
_not_equal
_greater
_greater_equal
_lesser
_lesser_equal
elemwise_add
elemwise_sub
elemwise_mul
elemwise_div
_maximum
_minimum
flatten
Flatten
square
rsqrt
cbrt
rcbrt
pow_scalar
_power_scalar
rsub_scalar
_rminus_scalar
rdiv_scalar
_rdiv_scalar
rpow_scalar
add_scalar
_plus_scalar
sub_scalar
_minus_scalar
mul_scalar
_mul_scalar
div_scalar
_div_scalar
log2
log10
log1p
expm1
_equal_scalar
_not_equal_scalar
```

`_greater_scalar`
`_greater_equal_scalar`
`_lesser_scalar`
`_lesser_equal_scalar`
`_maximum_scalar`
`_minimum_scalar`
`mean`
`max`
`min`
`sum`
`max_axis`
`min_axis`
`sum_axis`
`argmax`
`argmin`
`_ones`
`softmax`
`log_softmax`
`Softmax`
`softsign`
`softmin`
`hard_sigmoid`
`reciprocal`
`Reshape`
`reshape`
`Cast`
`amp_cast`
`amp_multicast`
`clip`
`transpose`
`UpSampling`
`add_n`
`_zeros`
`FullyConnected`
`Activation`
`Convolution`
`Convolution_v1`
`Deconvolution`
`Pooling`
`Pooling_v1`
`Dropout`
`BatchNorm`
`BatchNorm_v1`
`_contrib_SyncBatchNorm`
`InstanceNorm`
`LayerNorm`
`GroupNorm`
`LRN`
`L2Normalization`
`slice`

slice_like
slice_axis
SliceChannel
split
_split_v2
SwapAxis
expand_dims
Concat
concat
stack
dot
batch_dot
LeakyReLU
_arange
_full
repeat
tile
pad
Pad
take
gather_nd
reverse
SequenceReverse
squeeze
broadcast_axis
broadcast_axes
BlockGrad
shape_array
Embedding
argsort
topk
_unravel_index
SequenceMask
SoftmaxOutput
SoftmaxActivation
LinearRegressionOutput
LogisticRegressionOutput
smooth_l1
make_loss
_contrib_div_sqrt_dim
_contrib_arange_like
one_hot
depth_to_space
space_to_depth
Correlation
_contrib_BilinearResize2D
_contrib_MultiBoxPrior
_contrib_MultiBoxDetection
_contrib_ROIAlign
ROIpooling

```

_contrib_Proposal
_contrib_MultiProposal
_contrib_box_nms
_contrib_box_decode
_contrib_DeformableConvolution
_contrib_AdaptiveAvgPooling2D
GridGenerator
BilinearSampler
RNN
_rnn_param_concat
_contrib_interleaved_matmul_selfatt_qk
_contrib_interleaved_matmul_selfatt_valatt
_cond
Crop
ring_buffer
_contrib_quantize_v2
_contrib_quantized_concat
_contrib_quantized_ring_buffer
_sg_mkldnn_conv
_contrib_quantized_flatten
_contrib_dequantize
_contrib_quantized_act
_contrib_quantized_pooling
_contrib_quantized_batch_norm
_sg_mkldnn_fully_connected
_np_transpose
_npi_transpose
_npi_pad
_npi_concatenate
_npx_reshape
_np_copy
_npi_copy
_npi_power
_npi_power_scalar
_npi_multiply
_npi_multiply_scalar
_npi_add
_npi_add_scalar
_npi_subtract
_npi_subtract_scalar
_npi_where_rscalar
_npi_less
_npi_less_equal
_npi_tanh
_npi_true_divide_scalar
_npi_stack

```

商标声明

- 天数智芯、天数智芯 logo、Iluvatar CoreX 等商标、标识、组合商标为上海天数智芯半导体有限公司之注册商标或商标，受法律保护。
- CentOS 标识为 Red Hat 公司的商标。
- Docker 为 Docker 公司在美国和其他国家的商标或注册商标。
- Linux 为 Linus Torvalds 在美国和其它国家的注册商标。
- NVIDIA、CUDA、GeForce 和 GeForce GTX 为 NVIDIA 公司在美国和/或其它国家的商标和/或注册商标。
- PyTorch 为 Facebook 公司的商标。
- TensorFlow 为 Google 公司的商标。
- Ubuntu 为 Canonical 公司的注册商标。

除上述明确列出的商标外，本内容中可能存在的任何其他未明示的产品名称、服务名称、企业名称、标识、口号、域名或其他商业标记 (无论是否标注 ™ 或 ® 符号)，均假定归属于其各自合法的商标权人所有。